Project Submission Form

UNIVERSITY OF LONDON - MSc INFORMATION SECURITY

A copy of this form (or a typed or computer-generated version) must be completed by each student and attached to each hard copy of the project report that is submitted to the University.

STUDENT	**SUPERVISOR**
Name **David Stent**	Name **Guillermo Suarez-Tangil**
Number (SRN) **130157887**	

ANTI-PLAGIARISM DECLARATION

You are reminded that all work submitted as part of the requirements for any examination of the University of London must be expressed in your own words and incorporate your own ideas and judgements.

Using another person's thoughts or words as though they were your own work is considered to be copying or plagiarism. This is not allowed. You must clearly identify direct quotations from the published or unpublished work of another person by placing those quotes inside quotation marks, and also provide a full reference to their source.

Whether you use a series of short quotations from several different sources or a single long quotation from a single source, they must all be identified clearly. Equally, if you summarise another person's ideas or judgements, you must refer to that person in your text, and include the work referred to in your bibliography. Failure to observe these rules may result in an allegation of cheating.

I declare that this dissertation is all my own work, and that I have acknowledged all Quotations from the published or unpublished works of other people.

I declare that I have also read the statement on plagiarism in the General Regulations for Awards at Graduate and Masters Levels for the MSc in Information Security and in accordance with it I submit this project report as my own work.

Please sign here to show that you have read the above:

_____ Date: 27/03/2017

A Comparison of Polymorphic and Metamorphic Forensic Techniques

Author:	David Stent
Email Address:	david@stentsure.com
Registration Number (SRN):	130157887
Date:	March 2017
Supervisor:	Guillermo Suarez-Tangil

Submitted as part of the requirements for the award of the MSc in Information Security of the University of London.

Table of Contents

1 Executive summary 6
2 Introduction 7
2.1 Aims 7
2.2 The impact of malware on organisations 8
2.3 The dangers of malware to an organisation 8
2.4 Malware complexity is increasing 9
2.5 Objectives 10
2.6 Scope 11
3 Packaging payloads 12
3.1 Packers 12
3.2 Stubs 14
3.3 Binders and joiners 15
3.4 Crypters 16
4 Malware classification 17
4.1 Types of crypters 17
4.2 Oligamorphic malware 18
4.3 Polymorphic malware 19
4.4 Metamorphic malware 21
4.5 Comparison of malware types 22
4.5.1 Renaming or encoding variables, subroutines and strings 23
4.5.2 Adding jumps across code 23
4.5.3 Adding junk code 23
4.5.4 Adding garbage code 24
4.5.5 Using permutation techniques 24
4.5.6 Exchanging equivalent code 25
4.5.7 Splitting data structures 26
4.5.8 Adding anti-virtualisation and anti-debugging protection 26
5 Cryptographic malware antivirus evasion 26
5.1 Signature-based detection 26
5.2 Heuristic-based detection 27
6 Malware analysis and using a development lab 28
6.1 Why do we need a lab? 28
6.2 Infrastructure of the lab 28
6.3 Building the lab 30
6.4 Risks to consider whilst operating in the lab 30
7 Case study one – manual deconstruction of polymorphic malware 31

7.1 Background 31
7.2 Scope 32
7.3 Reverse engineering the sample 33
7.3.1 Examination of the sample's behaviour 33
7.3.2 Unpacking a sample's outer layer of protection 36
7.3.3 Baiting the sample 39
7.3.4 EPO techniques 41
7.3.5 Stub unpackers 42
7.3.6 Analysis of the stub 42
7.3.7 Polymorphic functions in the stub 44
7.3.8 Obtaining the unencrypted payload 44
8 Case study two – compilation and analysis of metamorphic malware 46
8.1 Background 46
8.2 Scope 47
8.3 Compilation of Lexotan32 47
8.4 Analysis of Lexotan32 50
8.4.1 Re-generation 53
8.4.2 Mutation 53
8.4.3 Permutation 54
8.4.4 Garbage code creation 54
8.4.5 Relocations 55
8.4.6 New genotype creation 56
9 Case study three – mobile malware packed by online platforms 56
9.1 Background 56
9.2 Scope 57
9.3 Building a sample 58
9.4 Packing the sample 59
9.4.1 ProGuard vs ALAN 59
9.4.2 APK Protect 60
9.4.3 Bangcle 61
9.5 Analysis of a packed sample 62
9.5.1 Basic static analysis 62
9.5.2 Anti-debugging 63
9.5.3 Dumping memory using Android-unpacker 64
9.5.4 Using Volatility 65
9.5.5 Obfuscation techniques observed 67
10 Comparisons, conclusion and reflections 67
10.1 Comparisons 67

10.2 Conclusion 68
10.3 Reflections 71
11 Bibliography 72
11.1 References cited 72
11.2 Documents used as additional sources 74
12 Appendix 77
12.1 The use of VMware ESXi for hosting the malware lab 77
12.2 Using Ollydbg to extract a payload from memory 77
12.3 Case study three development environments 77
12.4 Using Bangcle to protect an Android application 77
12.5 Malware samples used as part of the study 78
12.5.1 Virut.ce 78
12.5.2 Lexotan32 79

1 Executive summary

I have spent the first ten years of my career as a Systems Administrator, during which it has always surprised me when a malware infection has been detected and the decision has been taken to either clean up the infection or destroy the infected device. Often, the server or workstation has been rebuilt without the organisation investing the time to work out how the infection occurred or what the malicious program has executed on the corporate device.

Since then, I have moved to the role of Security Analyst and I have continued to notice the same behaviour, although I have started to realise that it isn't always financially viable for an organisation to invest resource in investigating each incident. Instead, they save time by accepting the risk. This feels counter-intuitive to me, as the cost of investigating an incident could be balanced against the level of assurance provided by investigating the issue and then building more accurate documentation and methodologies that could support future practical examination of malware.

Malware samples that often afflict businesses targeted by organised crime and other malicious parties are obfuscated to make them difficult to analyse. To the best of my knowledge, no single place exists that documents the techniques and tooling required to investigate obfuscated malware designed to target an organisation and be resistant against forensic analysis.

The primary purpose of this report is to provide information regarding the techniques used by obfuscated malware designed to evade antivirus software and forensic examination. This study provides practical examples of the techniques and tooling used to quickly investigate and obtain a malware sample's payload during an incident. The concepts used to investigate the malware samples that are presented in this study can be utilised to aid similar investigations carried out by an organisation and to make a forensic analysis of a sample more appealing to stakeholders. It is hoped that more organisations will realise that investigating a malware infection can actually be feasible and can also be a positive investment of time as it reduces the unknown risk associated with a malware infection.

To summarise, this paper makes the following contributions.

- It provides a case study focused around the forensic analysis of a polymorphic sample during a simulated major incident.
- It delivers a practical investigation of how to profile the tools and techniques used by a malware writer, which is a highly desirable skill when looking at a sample that is suspected to have been built to target an organisation.
- It examines in detail the techniques used by an innovative metamorphic malware sample.
- It demonstrates the use of an online platform for protecting Android applications using a lab-based example.
- Advanced techniques that are used to protect Android applications are explained.
- The custom samples and tools that have been developed as part of this project have been made open source, and they are publicly available on GitHub: https://github.com/devopscrazy.
- Practical techniques and tools used as part of this project have been filmed and are available publicly on YouTube: https://www.youtube.com/channel/UC1wecqGMvSU-Q-gxV2yCkcg.
- By providing a repository of tools and videos that demonstrate their use, an analyst, systems administrator or engineer can start a forensic investigation without having the overhead of researching appropriate tooling.

2 Introduction

2.1 Aims

This report is designed to provide documentation and tooling for security engineers, security analysts and systems administrators who may be faced with the option to conduct forensic examination of malware during or after a major incident. Currently, documentation to aid forensic analysis during an incident where there is a requirement to analyse polymorphic and metamorphic malware affecting personal computers or mobile devices is either difficult to find and collate, or it is incomplete. This study looks at different types of malware across various devices and analyses some of the techniques already documented by previous studies that are relevant to this investigation.

Some of the techniques and experiments use methodologies and research that have been documented in papers cited throughout this report. Here, these methodologies are carried out using a fresh approach, focusing on an agile analysis of malware samples during a security incident. The techniques used by the malware samples to obfuscate their payload are documented within this project, as well as the forensic techniques used to extract the payload itself. During initial research into the subject, it became evident that there was a deficit in documentation that collated techniques and tools that an analyst would need as part of a forensic analysis during a major incident. There remains examples of these obfuscation techniques in use by malware in the wild that are designed to target specific organisations and individuals.

The obfuscation detailed within this project is focused on techniques that utilise the encryption and substitution of code; however, it is important to acknowledge that there are more sophisticated obfuscation techniques that are possible when hiding malware within a sample. One such technique is the use of *steganography* to hide malicious code inside a program's asset. The concept of steganography has been around for a long time and it historically involved hiding messages within images; similarly, *stegomalware* [1] can be used to hide a malicious payload within a sample's asset. The techniques for hiding a malicious payload in this manner are more sophisticated and have been tested to evade detection by the protection mechanisms in place as part of submitting an application to a mobile marketplace; however, their use in the wild has not been conclusively proven. Therefore, due to the limited time available, this project has focused on well-established techniques which has resulted in easily obtaining live malware samples for the case studies. The malware samples used in the case studies are many years old and due to their success, the techniques they use will more than likely be utilised by future malware writers. It is hoped that the forensic techniques demonstrated in this project will aid in the examination of future samples that utilise polymorphic and metamorphic obfuscation techniques.

Any custom tools developed and used as part of this project are open source and can be accessed from the following link: https://github.com/devopscrazy. The GitHub repository also contains a final copy of this project, providing easy access to all content within one location. As well as open source tools developed as part of this study, the practical sections of the three case studies have been recorded and are available on YouTube. These recordings have been designed to help aid systems administrators and security analysts tasked with malware forensic analysis during major incidents. Links to the videos that have been uploaded to YouTube can be tracked from this repository on GitHub: https://github.com/devopscrazy/MalwareAnalysis. Individual videos are cited throughout this document by direct links and in sections that are relevant to their recorded content. If any of the content gets updated or is removed, or if its hosting provider is migrated, then the links in the GitHub repository will provide a central point for easy maintenance and reference. As such, this repository should be referred to if the links to the videos in this document are no longer available.

2.2 The impact of malware on organisations

The term malware describes any program that performs a malicious activity to further compromise a machine [2, page 6]. These programs are often designed to compromise or disrupt systems by using vulnerabilities inherent in Operating Systems and in the software that has been installed on the machine. Malware is no longer confined to simply infecting traditional computer systems; instead, it often infects mobile devices, such as smart phones, as well as embedded hardware, such as what is commonly referred to as the *Internet of Things*. Mobile devices often have full Operating Systems present; on Android devices these are based on *Linux*, and on Apple devices the underlying Operating System is based on a variant of *Unix*. These types of mobile devices are often referred to as smart phones.

Both traditional computer systems and smart phones allow for custom applications to run on the device. There are no strict industry recommended regulations in place that application writers must comply with when submitting software to online marketplaces. When device owners visit an online marketplace, they can conveniently and easily purchase or download applications for free. However, they are not provided with any assurance that the application has been implemented securely, since mobile applications are often being rushed to online marketplaces to gain an advantage over the competition. These applications have the potential to provide an attacker with limited or even full access to the mobile device, allowing them to control and instruct the device to carry out some form of malicious activity.

Sometimes, this malicious activity can involve using the infected device as a node in a botnet to target an organisation, rather than attempting to steal credentials or personal information from the infected device and targeting the user directly. These nodes are referred to as zombie nodes, and the activity that the malware carries out is often completely undetected by the device owner. The malicious activity carried out by a zombie node is controlled by a remote attacker, and is often targeted towards organisations that have no connection with the owner of the device. A *botnet* is a network of zombie nodes that are controlled by a central command-and-control server. The owner of the infected device is often unaware that their device is carrying out these operations, such as sending network traffic to a target, because the small amount of network traffic sent to the target is not enough to affect the resources on the infected device. If enough zombie nodes are present in a botnet, it can have an exponential effect on the target of the attack. This type of attack is known as a *Distributed Denial of Service* (DDOS) attack and is becoming increasingly common [3].

DDOS attacks are commonly used to hold a business to ransom and can threaten them with disruptions to their systems if a ransom is not paid. The malicious party sometimes disrupts a target's system long enough to affect the business financially and then attempts to extort money from the target to cease any further attack.

Therefore, malware is an attractive commodity to organised crime; it provides a resource that could result in a good return of investment to the attacker if it succeeds in extorting money from the target organisation [4, section 2].

2.3 The dangers of malware to an organisation

When malware is executed on a system, the payload itself conducts certain operations, such as infection and privilege escalation within the device. The payload can also contain operations to evade antivirus tools that may be present on the system. Payloads are often packaged within malware using layers of obfuscation that try to prevent security analysts from easily reverse engineering the malware and identifying what its payload is doing. On initial execution of the sample, the de-obfuscated payload is extracted and the instructions within the payload are run.

If the infection has resulted in data being compromised or a system outage, then this could have a financial as well as a reputational impact on an organisation. This type of damage to an organisation changes the classification of an incident to "major". The payload may have been present on the workstation undetected, and sometimes it can take antivirus vendors up to four weeks to provide signatures to detect such payloads [5, page 8]. If the infected device contains or has access to business-critical information, then it is essential for the organisation to understand the capabilities of the malicious payload and what it may have done during the time it remained undetected. Answers to the following questions may be important to ascertain the level of risk that the business may have been exposed to during an incident.

- Have user accounts been leaked and compromised?
- Has the asset been used as a pivot point for further attacks on the corporate network?
- Have confidential documents on a user's workstation been leaked to other sources?
- If so, who has the data been transferred to and where are they located?

Alerting when malware is detected and simply cleaning it up is no longer an effective solution for some organisations due to the possibility of their internal policies not providing adequate or enough protection of their data. Attackers with more focused malicious intent, such as organised crime, now have easy access to resources to create malicious payloads. They have the option of anonymously hiring malware developers with the skills that they require. Organisations can no longer assume that malware merely contains pranks devised by mischievous youths. Individuals with malicious intent can build or obtain payloads that are designed to attack a narrow target, and a good example of this is the *Stuxnet* worm, developed specifically to target hardware used to control centrifuges that were part of a nation state's nuclear enrichment process [6, page 3].

Conducting *black-box* testing by simply running the malware and observing how it executes and communicates provides very little confidence of the full functionality of the payload. Therefore, reverse engineering malware is becoming a necessity to understand the risks that the malware may pose to an organisation. This can aid an analyst to develop a signature to detect the malware, resulting in them being able to fine-tune a pattern-based *Intrusion Detection System* (IDS). Analysing the malware may also provide a useful indication of any information that has been exposed within the organisation.

2.4 Malware complexity is increasing

As a malware sample is obfuscated the resulting payload is often larger and contains more instructions that are designed to confuse any attempt at obtaining the original sample, therefore, malware is becoming increasingly complex and difficult to investigate. Increased complexity of a payload can also be introduced to a sample using binders and packers, which can obfuscate payloads and combine them with other applications. These obfuscation techniques change the payload's signature and mask the true purpose of the underlying code [7, page 8].

Simpler techniques, such as compressing the payload or obfuscating a payload's code, are less effective than they once were. As they produce a single signature, antivirus vendors require less resource to add the sample to a signature database. As such, they have a limited life expectancy when targeting users who have effective signature-based antivirus software. Increasingly, short-lived malware campaigns are being developed that predominantly focus on harvesting data, such as banking credentials; an example of which is *Dridex* [8]. These payloads do not need to maintain long-term access to an infected device once they have accomplished their task. Therefore, the developers of this malware often use binders and packers to run a week-long campaign and obtain a healthy return of investment. However, campaigns that are directed at an individual business may actually require payloads to remain undetected for months while information is harvested, and so traditional binders and packers are ineffective at supporting *advanced persistent threats* (APT).

The next generation of viruses, which remain undetected for longer, spread using more intelligent packers called crypters. These crypters package the payload using encryption and attach code to the payload that is used to unencrypt it on infection, known as a stub. The stub utilises obfuscation techniques of its own to modify itself on re-infection [5, page 17]. The malware produces a newly encrypted payload on each infection using unique keys, and this technique makes it more difficult for antivirus software to develop a signature for detecting new infections from a single strain. These types of malware are polymorphic in nature and, in some cases, are capable of evading detection at rest for longer periods of time, as well as being resistant to dynamic analysis.

Nevertheless, polymorphic malware can still leave a signature as it contains an encrypted payload that could possibly be identified and blocked by stringent security policies. Another type of malware obfuscation instead uses metamorphic techniques to modify the entire code of the malware on re-infection [5, page 20]. This type of malware is more difficult to detect as it does not need to use encryption to change the malware's signature.

In summary, the malware technique landscape is constantly changing and the ways in which these techniques are implemented are improving. There are more advanced techniques being developed, such as stegomalware. However, there are inconclusive indications of these techniques being used in the wild. Polymorphic and metamorphic techniques have been used in the past and will continue to be used in the future by APTs due to their success in protecting historic samples.

2.5 Objectives

When reviewing the existing literature, there are several journal articles and documents detailing the cryptographic obfuscation techniques already in use by malware. However, as already mentioned, there exists a gap in the presentation of practical examples of polymorphic and metamorphic malware forensic analysis. One of the objectives of this project is to document the processes that are utilised by polymorphic and metamorphic malware to modify themselves on re-infection to evade detection by antivirus products. Furthermore, it aims to provide a single document that can serve as a reference during a major incident that requires the forensic evaluation of obfuscated malware.

Virtual Private Networks (VPNs) are used to pair bring your own devices (BYODs) to corporate networks, and now mobile devices are being paired in businesses using the same technology [9, page 11]. These devices provide a bridge directly into corporate systems and their data. As such, this project devotes a case study clearly detailing the techniques used by malware that has been designed specifically to target mobile devices. The study demonstrates the tools that can be used to extract the payloads from mobile devices and how to interpret the output from these tools for forensic analysis.

As each layer of obfuscation used by a packer or binder is identified, this document demonstrates these techniques using malware samples where possible. The incremental steps required to make polymorphic and metamorphic malware are dissected and the tools and any scripted code used to accomplish this are included in the case studies. There are three case studies that have been carried out to demonstrate either the deconstruction, compilation or packing of live or custom built malware samples. The live samples have been obtained from the virus repositories found on *VirusShare* (https://virusshare.com) and *Offensive Computing* (http://offensivecomputing.net/) websites.

The polymorphic and metamorphic malware samples used in the case studies are not current. The polymorphic sample has been infecting devices since 2000 and the metamorphic sample since 2004. Although these samples are over a decade old, their obfuscation techniques can still be considered as resourceful as those used today. In fact, the metamorphic sample used in this project packages its malicious code in such a unique way that it would be surprising if the code has not been the

inspiration for many current malware samples. The two live samples used in this project have been heavily scrutinised in other papers and therefore it has been easier to support the findings in this project's case studies. Some authors from the cited papers were also available to consult with when attempting to use new tools to demonstrate forensic techniques. Not only did contacting the authors of other papers aid with the examination of the samples, but it also helped to demonstrate collaboration as one of the most important tools to employ when conducting malware forensic analysis.

2.6 Scope

This document demonstrates the tools and techniques required to reverse engineer a polymorphic malware sample. It also dissects and compiles the source code and techniques used by a sample of metamorphic malware that was once common on a large number of infected devices during the ten years that it remained difficult to detect in the wild. The techniques used by these two types of malware to evade detection by antivirus software are broken down and presented in this project. From the viewpoint of a malware analyst, the case studies demonstrate the following three topics.

- Deconstruction of malware.
- Compilation of malware.
- Forensic analysis of malware.

This document provides practical demonstrations of how to peel back the layers of obfuscation used by malware developers in the production of polymorphic and metamorphic malware.

The polymorphic malware sample used within case study one is *Virtu.ce*. The case study provides a detailed breakdown of the tools and techniques used to reverse engineer this polymorphic malware to its de-obfuscated payload. The malicious activity conducted by the de-obfuscated payload itself is not detailed within this report. Instead, the focus of this report is to investigate how to obtain the original payload's Assembly code, which can then be used for future analysis by an analyst.

The metamorphic malware sample investigated in case study two is *Lexotan32*. This document demonstrates both the compilation and metamorphic techniques used by *Lexotan32*. Packing and re-infection techniques used by a metamorphic sample can be complex and cause code expansion. Therefore, rather than trying to peel back the layers of obfuscation used by a sample that could be many generations old and could have suffered a great deal of code expansion, this case study examines the metamorphic techniques used by this sample, examining its source code both prior to and during compilation.

The convenience of mobile devices allows their popularity to surpass that of computer systems. Therefore, this document also looks at the different types of cryptographic malware that are designed to circumvent antivirus solutions and infect mobile devices. In the third case study, the techniques and tools used to forensically examine a malware sample installed on a mobile device are examined. How to protect a sample from antivirus detection during the sample's development is also demonstrated.

In summary, this document is supported predominantly by three case studies to demonstrate the tools, techniques and forensic analysis of malware samples that classify as polymorphic, metamorphic or oligamorphic.

- In the first case study a sample is reverse engineered to examine the polymorphic techniques it has implemented to protect its payload. As part of this case study, the tools used to reverse engineer the sample are demonstrated.
- The second case study investigates how to compile and research the techniques used by a metamorphic sample. This includes some techniques used to profile the malware developer.
- The final case study details how to use a freely available *Platform as a Service* (PaaS) that claims to protect mobile applications. A custom sample is built and then protected using a PaaS. The protected sample is then reverse engineered to demonstrate how to forensically analyse a packed mobile application and to examine what techniques the PaaS has used to protect the mobile application.

It is important to understand that there are more complex forms of obfuscation that can be used by malware other than polymorphic and metamorphic techniques. However, this project's time and resources are limited which has unfortunately restricted its scope. Thus, this document is designed to provide a solid foundation of how to analyse complex samples that may rise in popularity in the future.

3 Packaging payloads

3.1 Packers

The *payload* of a malware sample contains the malicious commands that are performed on the infected machine. The payload often contains information that has been hard coded into it, and as such, it is not easily changed. Examples of this can include the address of the malicious party's command-and-control servers, or the location of any captured data that the malicious party intends to use for later analysis. As this data is hard coded into the sample, changing these references can be actioned in the same way as patching a piece of software, and often requires re-infecting a target with an updated payload. If a payload was not obfuscated, then the hard-coded information could easily be obtained, resulting in antivirus vendors being able to automate the process of extracting this information, and reporting malicious parties to the relevant authorities. Authorities, such as law enforcement agencies, have the power to block malicious resources. An example of a malicious resource is a command-and-control server, and depending on the impact and threat that the malware is causing, the process of blocking the malicious resource can be expedited. There can be a long lead time between a malware sample being released and antivirus vendors creating a signature for its detection. If the cost of creating the malware is higher than the revenue generated, either before it is detected by antivirus software or before its command-and-control servers are taken down by law enforcement, then there is no financial gain for the developer to use malware that is not obfuscated.

Consequently, malicious parties are seeking quick and cost-effective ways to encapsulate their payloads into a hardened software envelope. A software envelope can protect the information within a payload from being extracted by automated tools used by antivirus vendors, such as command-and-control server addresses.

It is not just malicious parties that look to package payloads in software envelopes; companies also use these techniques to protect their intellectual property. Therefore, blocking the execution of payloads packaged in software envelopes is not a common practice as it would prevent the execution of legitimate software.

Packers offer the ability to encapsulate a payload inside a software envelope. A common and freely available packer is the *Ultimate Packer for eXecutables* (UPX). The first case study demonstrates the

use of UPX and a recording of its use has been published on YouTube and can be found via the links in section 2.1. UPX uses both compression and simple obfuscation when protecting a payload. A stub is packaged inside the software envelope along with the obfuscated payload. Upon execution of the software envelope, the stub will be executed first instead of the payload, which unpacks and decompresses the payload before it then executes it. When viewing the contents of a payload in a hex editor prior to packing, hard-coded items in its content are easily visible. After packing a sample, these hard-coded items are no longer easily identifiable in a hex editor. Packers can therefore be thought of as tools to help prevent the reverse engineering of code produced by both software vendors that want to protect their products and malicious parties wanting to hide the purpose of their payloads. Packers take a payload as their input and, after processing, output a payload that is surrounded by a protective shell.

Tools used to pack payloads, such as UPX, can often offer the ability to unpack the payload using the same tool. Malicious parties have found ways to further protect samples that have been packed by modifying the software envelope to prevent the tool from being able to unpack it. One technique to accomplish this is to remove identifiable flags within the software envelope that are used by UPX to confirm that a payload was originally packed by it. Consequently, when trying to unpack the software envelope using the tool, it will fail. The stub within the software envelope is still able to unpack the payload and continue its execution, however, using the original tool fails to locate the payload to be able to unpack it successfully.

The payload can sometimes be modified by the packing process, such as obfuscating identifiable names. However, these changes do not affect the execution and functionality of the original payload, and instead provide a greater challenge when carrying out forensic analysis of the obfuscated payload.

The output produced by the processes of packing a payload is referred to in this project as a *malware sample*. Within the malware sample, an unpacking engine is referred to as a *stub* and the code that it unpacks is referred to as the payload. A *software envelope* consists of the stub and the payload packaged together. Figure 3.1 demonstrates the workflow of a payload being packed into a software envelope. The stub and the payload are referred to separately in the case studies as the two are not always kept together within an infected sample. Polymorphic and metamorphic samples tend to vary the location of where they keep the components of a software envelope to help mask their presence from antivirus software.

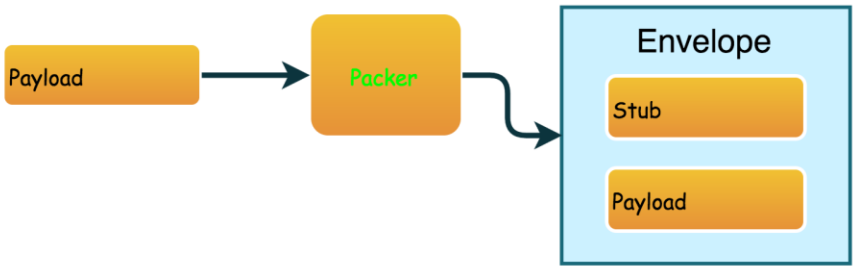


Figure 3.1: Workflow of a packer protecting a payload and producing a software envelope.

This project focuses on more specialised types of packers called binders and crypters; these packers utilise more complex methods to obfuscate a payload. However, case study one observes some of the less complex techniques that are used by packers, such as payload compression. Crypters generate software envelopes containing stubs that use specialised techniques to decrypt the

payload and re-package it on re-infection, and these techniques are the focus of the first case study. The combination of a compressor and a crypter is often referred to as a protector, although case study one does not focus on a sample that combines these functions. Instead, the sample used in case study one can be thought of as an "onion" of security, and as such, it contains many distinct layers of protection wrapped around the payload located at its centre.

Some samples contain many executables as part of their payload and therefore they require a bundler to join them together. Although the case studies in this project do not contain multiple executables that require a bundler to produce a payload, case study two does involve pre-processing several files together that are required to compile a sample. The concept of a bundler is briefly covered in section 3.3.

3.2 Stubs

As detailed in section 3.1, the stub is a component of the software envelope that extracts, executes and provides a layer of protection to the payload whilst it is in an obfuscated state. There are several methods that a stub can use to extract a payload from its obfuscated state. When extracting the payload prior to execution, the stub can either:

- save the extracted payload to disk
- extract the payload and inject it directly into memory.

During the process of extracting the payload to disk, there is always the possibility that it can be scanned by any available antivirus software on the system. Antivirus software that provides on-access scanning often reads the files being written to disk and compares the contents against a signature's database. Therefore, when developing a stub, this method of payload extraction is undesirable.

Alternatively, injecting the payload into a process in memory provides more protection from being detected by antivirus software. Some stubs can pause a legitimate program already resident in memory, write the payload to a memory stack and adjust the paused program so that the payload is executed when the program is resumed. This method does not involve writing files to disk, and so any on-access scanning that is being conducted by antivirus software does not get triggered.

Some packers encrypt a payload by combining it with a key using a bitwise *XOR* operation. It is this use of encryption that differentiates a crypter from a standard packer, which only uses simple compression techniques to obfuscate a payload. The encryption key is often hard coded into the stub to allow it to decrypt the payload. By hard coding this data in the stub, it allows the key to be extracted by an analyst during the forensic analysis of a sample. There are alternative techniques that can be implemented which allow a key to be used and not packaged with the stub. The available methods for obtaining the encryption key and decrypting a payload are the following.

- The key can be hard coded into the stub and used directly by the stub when decrypting the payload.
- The stub can generate the key programmatically and then use the key to decrypt the payload without exposing the key that has been used.
- A seed can be hard coded into the stub; this seed can be passed through a series of pseudo-random algorithms to derive the encryption key that can then be used to decrypt the payload without exposing the key during the process.
- The stub can run an algorithm to brute force its own key and, when the correct key is found, it can be used to decrypt the payload.

The paper by Deque entitled *Handbook about Crypters and Binders* [5, page 16] provides an example of a virus called *Solixious* that hard coded its encryption key into the stub. This document by Deque is a great source of information when researching crypters and was originally sourced via a forum hosted on the *Tor* network. Content on Tor is dynamic and constantly moving or being removed, and unfortunately the document by Deque has been removed since conducting this project. Thus, the copy cited in this document is hosted on the GitHub account previously mentioned in section 1.

Hard coding a key into a stub means that an analyst need only search for strings in an executable program to find the key. Using a function in the stub to generate a key makes it more difficult for an analyst to derive the key, although this does not prevent them from deriving it by executing the same function themselves.

Hard coding a seed into the stub and then passing it through a pseudo-random generator to derive the encryption key only slows down the forensic analysis of a payload. An analyst can still extract the logic of the pseudo-random generator from the stub and replay it themselves to convert a seed into a key to decrypt the payload. The addition of more programmatic steps means that an analyst will need to interpret more code when analysing a stub. To protect an encrypted payload, the most effective of the methods listed above is to make the stub brute force its own key. This requires the stub to be programmed to iterate through all the possible combinations of keys until it reaches the one required to decrypt the payload. A drawback to this method though is that the stub needs to run for longer and it will also consume system resources while it attempts to brute force the decryption key. However, on systems where there are enough available resources for processing the stub, its execution can be left to run as a background task.

Antivirus software vendors often submit samples to automated systems for unattended analysis. If the stub is trying to brute force its own key before decrypting and exposing the payload, then these automated systems may simply time out. In doing so, this may then prevent the sample from being flagged as malicious by antivirus software. By deterring antivirus vendors from finishing any automated analysis of the sample, it will result in the sample lasting longer in the wild before detection.

Some crypters place markers in the software envelope to enable the stub to locate the start and end of the payload for decryption. Unfortunately, this provides a signature that antivirus software can utilise to identify a payload. Instead of using markers, another option is to amend the stub with the addresses and size of the payload. The stub can then use these hard-coded variables to locate and unpack the payload without providing a pattern that could allow antivirus software to develop a signature for detection.

3.3 Binders and joiners

To further mask a malicious payload's signature from being detected by antivirus software after it has been packed, a common method of diluting its signature is to hide the software envelope inside another program. Binders and joiners are used to combine an obfuscated payload and its corresponding stub with other programs to not only mask their presence but also aid the re-infection of new programs. Figure 3.2 illustrates the workflow of a binder combining a stub and payload with a target program, and the diagram shows that the payload is then extracted from the infected program on its execution.

Binders can be used to package a payload inside a file, such as an image file, instead of using a program. The stub will still need to be combined with a file that can execute it so that when the stub is run, it can then extract the payload from the image. If there is free space available at the end of an executable file, then a binder can also hide an entire software envelope at the end of the executable file. A drawback of this is that antivirus software can easily detect this method of appending an already executable software envelope to the end of an existing executable file.

Antivirus software does not flag every file that has been joined together in this manner as malicious because legitimate software designers also use these processes as they provide a convenient method for packaging any of the software's pre-requisites.

Binders can also be used to hide an obfuscated payload inside its stub, allowing for one executable that contains both the payload and the stub that is required to extract it.

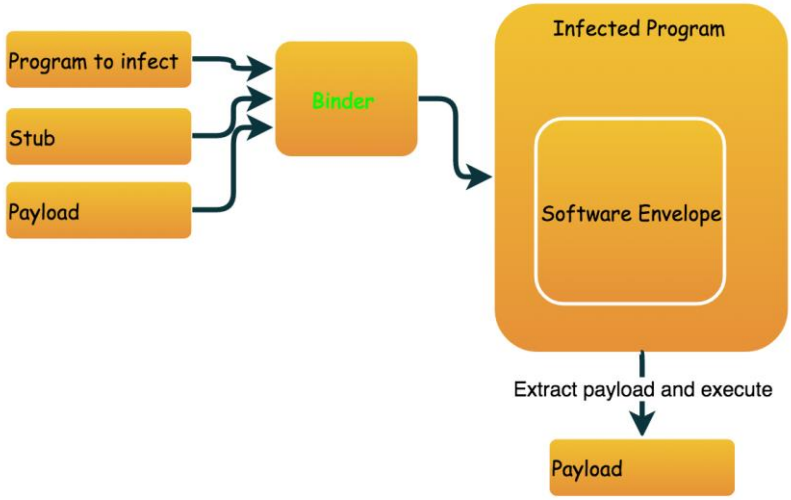


Figure 3.2: Infection of a program using a binder.

As well as a binder combining a software envelope with another program, it also needs to modify the infected program so that the stub is executed, otherwise the infected program will simply contain hidden content that will never be run. Without any alteration, the original portion of the executable will still be run by default, and without the infected program being modified to execute the newly inserted content, the malicious portion of the infected file will remain inert. For the infected program created in figure 3.2 to extract and execute the payload, the infected program must be modified to do one of the following.

- The infected file can be modified so that the first thing that is executed is the stub. Once the stub has finished executing, the original program prior to infection is then executed.
- The infected file can be modified so that during its execution it context switches to call the stub. Once the stub has finished executing, the program will resume.

Some binders add new sections to the infected program to store the software envelope, for instance, the software envelope can be stored in the resource section of a file. This portion of a file is where the strings and images are commonly stored. Therefore, discovering portable executable content stored in this section is enough of an indicator to justify further investigation of a sample's content when conducting a forensic analysis of a sample.

3.4 Crypters

Thus far, packers that allow a payload to be compressed and obfuscated using straightforward techniques have been examined, such as encoding and the *XOR* of a payload's content. The techniques and reasons for using binders and joiners to combine a payload and stub with another executable program have also been described. Using any one of these techniques have so far produced a single sample that will result in the same signature when infecting a new file. Once antivirus software vendors create a signature for the sample or the packer itself, further investment of time is required for the malware developer to modify the packer, stub or payload to evade detection by the antivirus signature.

A crypter can be used to secure a payload using encryption and it can also encrypt the stub in some circumstances. This results in a software envelope that is more resistant to antivirus vendors developing signatures that can detect the packer that was used to package the sample instead of detecting the sample itself. By only encrypting the payload and continuing to use the same stub, a point of entropy is provided which antivirus software vendors can then use to develop a signature for detection. Therefore, crypters also obfuscate the stub that is packaged within the payload so that stubs generated by the same crypter vary between different samples.

The main purpose for the usage of crypters is to make the payload resilient enough to remain undetectable by antivirus software. The ultimate goal of a malware writer would therefore be to build a *Fully Undetectable* (FUD) payload [10]. There are two types of crypters that try to provide a FUD sample.

- *Run-time* crypters that write to disk.
- *Scan-time* crypters that write to memory.

These techniques provide names to the concepts outlined in section 3.2. A run-time crypter produces a packed sample that, when running, extracts the payload to disk and then executes it. This has the same drawbacks as previously detailed in that it is scanned by antivirus software on writing to disk. Scan-time crypters produce samples that inject their payload directly into memory, and by not writing to disk, it is hoped that antivirus software will not scan the payload.

Both techniques utilise the same key derivation techniques described in section 3.2 to decrypt the payload, such as hard-coding encryption keys into the stub, generating the key by some form of computational process, or running a brute force algorithm "on the fly" to generate the key.

4 Malware classification

4.1 Types of crypters

Three types of crypters that increase in the complexity of their implementation and level of protection are described in this project. Each type provides a robust level of obfuscation when protecting a sample. These crypters modify a sample so that any new files that are infected do not trigger existing malware signatures present in antivirus software. If a signature is added to antivirus databases to detect the original sample, then subsequent infections carried out by the malware sample will remain robust enough to not be detected by the same signature. Section 2.4 described that if a malicious party has financial motives, they will require a return of investment. A crypter enables a sample to remain undetected for a longer period of time, and as a result, it enables a campaign to achieve greater financial gain for a relatively small degree of effort required to secure the sample.

The first type of crypter that is discussed is the simplest and it produces an oligamorphic malware sample. Oligamorphic samples produce an encrypted payload and a stub to decrypt it. The payload is encrypted using a new key when it infects a new file. This provides a payload that is constantly changing as it builds a new generation, and so antivirus software is unable to develop a signature to detect just the contents of the payload. The oligamorphic engine that allows the malware sample to infect new files has a pool of possible stubs that it can select from when building the software envelope for the next generation of the sample. For an antivirus vendor to develop a signature that can detect all generations of the sample, they must therefore consider all variations of the stub that are being used throughout the lifetime of the malware sample. This process is more time-consuming than developing a signature for a sample that has simply been packed, and therefore oligamorphic samples can remain undetected for longer in the wild.

Polymorphic malware builds on the same concepts used to build oligamorphic malware. This type of malware uses the same method when encrypting the payload, but it reduces the risk of detection by antivirus software by improving how the stub is obfuscated. Rather than rotate between a pool of stubs, a polymorphic engine mutates the stub when infecting a new file. By mutating the stub, more combinations are possible when compared to just rotating between a pre-determined number. As there are too many resulting variants for antivirus vendors to be able to develop a signature, it therefore requires a greater degree of effort and a longer timeframe for antivirus vendors to build a signature that has a high enough detection rate for all generations of the sample.

The final type of crypter that this study explores is metamorphic in nature and builds upon the morphing ability used by polymorphic malware to modify the stub. Whereas polymorphic malware encrypts its payload, a metamorphic malware sample does not. Just as polymorphic malware mutates its stub, metamorphic malware uses similar techniques to protect its entire payload. Rather than encrypting a payload using a new key and then mutating a stub upon re-infection, the metamorphic engine mutates the entire sample's code base. When infecting a new file, the code used as part of the payload is mutated as well as the metamorphic engine, and thus the resulting sample has a different code base to the last generation of the sample but it accomplishes the same results when it is executed.

A sample does not have to fulfil a rigid list of criteria for it to fall into one of the types of crypters detailed above. The variation in the methods and approaches of malware writers means that setting a threshold whereby a sample must operate or be structured in a certain way to classify as a type of crypter is too ambiguous. As an example, a malware sample could mutate all of the components within the software envelope when infecting another file, and therefore this could result in a payload that has the same end result as the original sample, thereby classifying it as metamorphic. Another sample could also classify as metamorphic, but instead of the mutated code conducting the same end results, the metamorphic engine could be used to extract clean code from the sample that is then executed to provide the same end result as the original sample.

4.2 Oligamorphic malware

As mentioned in section 4.1, an oligamorphic malware sample uses one of a limited number of possible stubs upon infecting new files. When creating the next generation, a sample can either have a pool of possible stubs to select from or it can modify the stub in certain ways to make it unique on the next infection. In either case, the possible stub combinations that a sample can use are limited and this will eventually result in the same stub being used by more than one generation, or in it becoming feasible for an antivirus vendor to produce enough variants of the stub to develop a signature that can cover the entire strain.

The cryptographic algorithms used by the stub can also be rotated on new infections, and the methods of encryption can also be combined to form layers of obfuscation. Examples of common cryptographic algorithms include *AES*, *3DES* or even the older algorithm *DES*. Antivirus software signatures that look for specific content encrypted with one of these algorithms can be evaded by rotating the algorithm used. The stub will still use the same workflow for decrypting the payload, but the algorithm used to decrypt the content is changed. The pool of possible decryption routines is small, and therefore varying the method of encryption only adds to the possible combinations of stubs and does not provide sole protection against signatures being developed by antivirus vendors.

The payload itself is often encrypted, and the key to decrypt it is stored in the stub using a method of obfuscation as previously detailed in section 3.2. When the stub decrypts the payload, it will produce an identical executable component for each generation of the sample. By encrypting components within the sample, an analyst must use more detailed forensic analysis techniques to obtain the payload when developing signatures of their own.

The main drawback to oligamorphic malware is that it is not particularly difficult to build antivirus signatures, and it is also feasible to automate the creation of these signatures.

4.3 Polymorphic malware

The biggest drawback to oligamorphic crypters is that there is a limited pool of potential stubs that a malware sample can be packed with, and as such, the automated systems of antivirus vendors are merely delayed in building a signature to detect its variants. Ideally, the malware sample needs to be resilient enough so that automated tools are either considerably delayed or they are prevented from developing a signature. Polymorphic malware addresses this limitation by increasing the possible number of distinct stubs, meaning that the creation of a signature that can detect all possible generations of the sample will be unlikely. Similar to oligamorphic samples, the payload in a polymorphic sample is encrypted, and in some cases, it is encapsulated within multiple layers that use different types of encryption algorithms. Both the encryption routines and the keys used are rotated on infecting new files.

The stub conducts many of the complicated instructions when unpacking the payload and therefore it has the potential to be the largest footprint for antivirus software to detect its presence on a system. Often, the stub itself can also be encrypted before being stored in the sample, and if this is the case, then a pre-process is used to decrypt the stub before it is executed. This reduces the overall effectiveness of hiding the stub's footprint as the pre-processor can often be used by antivirus vendors to produce a signature. The first case study in this document demonstrates that the use of a pre-processor can vary between the generations of a sample. Rather than rotating between a limited number of stubs on infecting new files, polymorphic malware mutates the stub. Within the sample exists a polymorphic engine that uses some of the following techniques to mutate the stub when infecting new files. It can:

- use random function names
- encode portions of the entire payload
- encrypt strings
- use a multitude of mutating techniques to ensure that a stable antivirus signature is not developed.

The workflow shown in figure 4.3 demonstrates a sample mutating as it infects a new file and builds its next generation. The payload contains a polymorphic engine that allows it to infect new files. The next generation re-encrypts the payload using a new key, and the stub is generated using a combination of multiple mutating techniques that are picked at random. The change in colour from generation to generation represents a change in signature; however, it is clear from the diagram that the payload maintains the same contents when extracted by each generation of the sample.

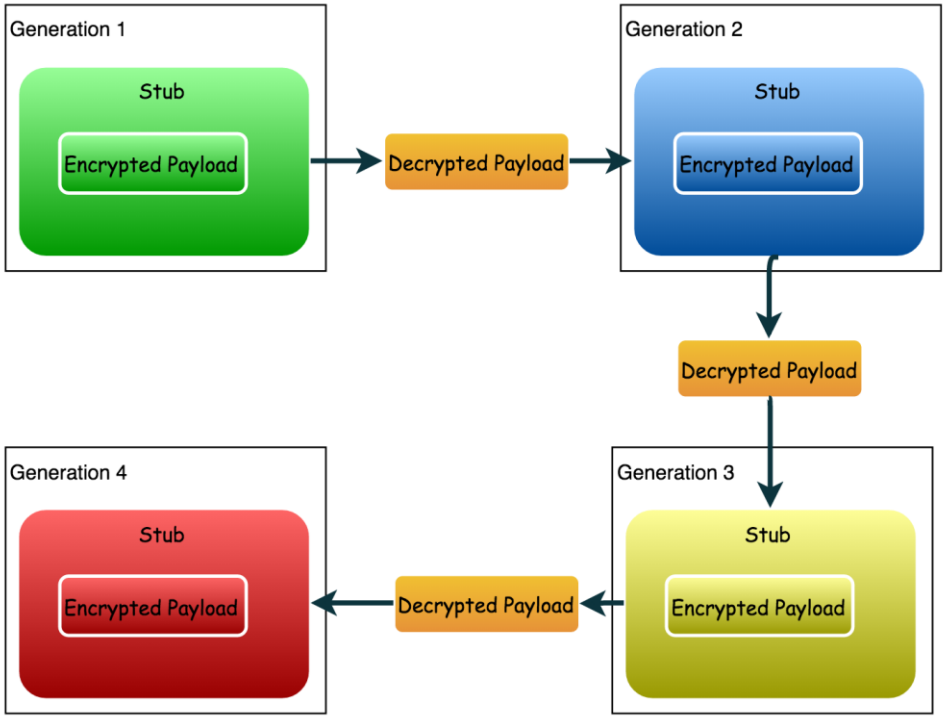


Figure 4.3: Demonstration that when a new generation is built by a polymorphic sample, it creates a different signature that, when executed, extracts an identical payload each time.

Some of the polymorphic engines that have been researched as part of this project have allowed a sample to vary where it stores the malware's stub within the infected file. The polymorphic engine can often randomly choose whether to store the stub:

- with the encrypted payload
- elsewhere in the infected file.

The polymorphic engines observed during this study were also noted to have the option to modify how the stub is then executed within a newly infected file, and the chosen option could be dependent upon the type of file being infected and the specific privileges that the sample may have on the infected device. When the sample has greater privileges in a system, it may decide to use *Entry Point Obscuring* (EPO) techniques. EPO techniques are described in case study one under section 7.3.4. A polymorphic sample can adapt its stub when infecting new files depending on the environment that it is infecting, and these different ways of infecting a file can help to impede both a forensic analysis and the creation of an antivirus signature. If an analyst does not map all of the possible jumps used by a sample, they may fail to observe the steps carried out on certain types of systems.

It is in these more advanced types of malware that anti-virtualisation and anti-debugging techniques are implemented. An efficient way for an analyst to obtain an obfuscated payload is to execute the sample and then dump the payload out of the device's memory once it is decrypted. By using anti-virtualisation and anti-debugging techniques, it is hoped that an analyst is prevented from obtaining the payload in this manner. Analysts often use virtualised environments to forensically analyse a sample, thus protecting their development environment (also referred to as a *lab*) from infection and accelerating the clean-up process involved in returning a lab back to a re-usable condition for analysing further samples. Often, extracting a machine's memory can be easier to accomplish from a virtual machine, as demonstrated in section 7.3.8. Malware writers that prevent their work from manifesting itself correctly on a virtual machine can therefore make it difficult for an analyst to

extract a sample's payload using a virtualised lab. Analysts also use debuggers to either execute a sample or connect to a running sample, and they can then dump a payload from memory or watch its execution in more detail. This can prevent a debugger from connecting to a running process or cause the debugger to crash when executing a sample, consequently slowing an analyst's investigation.

4.4 Metamorphic malware

The two types of malware that have been covered so far focus predominantly on protecting a payload by using encryption. The fact that a sample has an encrypted payload could provide sufficient information for a signature to be developed. Metamorphic malware does not use an encrypted payload, and a broad range of techniques can be combined to protect a metamorphic sample. For instance, case study two investigates a sample that uses techniques inspired by nature to extract a clean payload from an obfuscated sample that can modify itself on each new infection. Alternative samples may use techniques inspired by other phenomena that are of interest to the malware writer.

Metamorphic malware samples consist of both the metamorphic engine and the payload itself, and in this project, this is referred to as the malware body. The entire malware body is then mutated between infections, leaving very little that remains the same for a signature to be developed that is capable of detecting all the possible generations of a sample.

As mentioned in section 4.1, different types of metamorphic malware will have their metamorphic engines implemented in different ways. Some samples do not need to extract clean code from the malware body, as shown earlier in figure 4.3; instead, when mutating the code to create the next generation, they can build malware bodies that are distinct from one another but can still conduct the same tasks when executed.

Figure 4.4 demonstrates the workflow of the metamorphic malware sample as it mutates to form the next few consecutive generations of itself when infecting new files. This is shown in detail in case study two. The colour change between generations represents the malware body code mutating and this results in a distinct sample from the last generation. The change in shape represents the fluctuating size of the sample. As instructions in the code are replaced with different sections that perform the same semantic operations, it can result in the sample *bloating* in size. Alternatively, the sample can also shrink in size as operations are exchanged with shorter instructions. Regardless of whether or not the malware body has changed in size, the sample used in case study two results in clean code being extracted in each of the sample's consecutive generations.

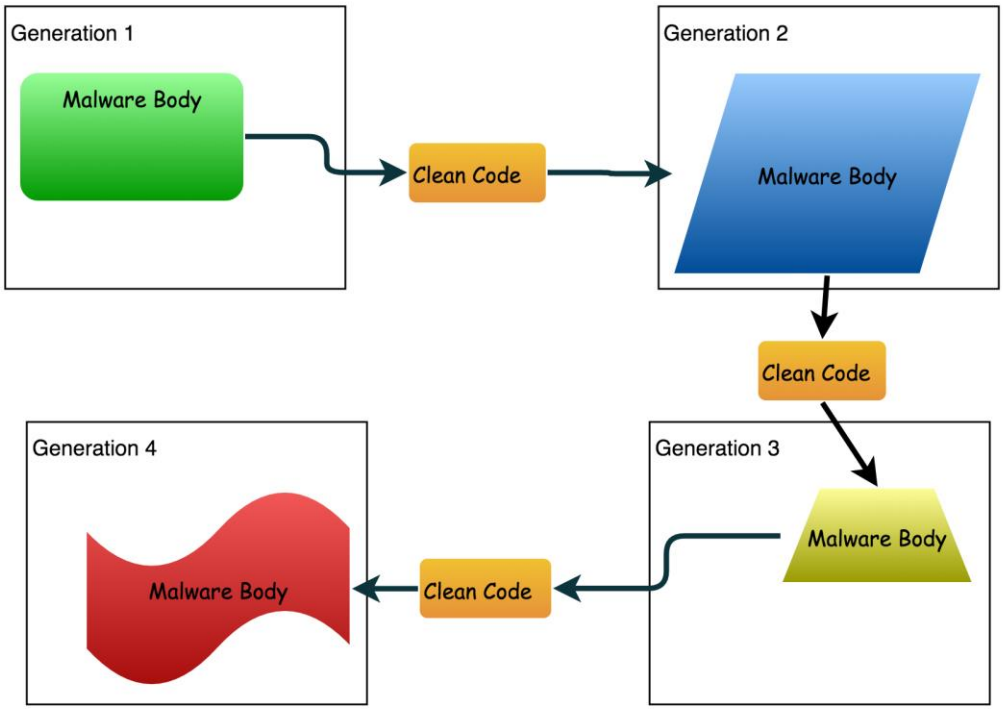


Figure 4.4: Metamorphic sample showing clean code extracted from differing generations.

If a metamorphic malware sample has been developed in a high-level language, then to mutate itself, it will need a compiler to re-build its source code. Ordinary programs do not require compilers to be used, so this unusual activity provides antivirus software with a threat to detect a malicious sample's activity. If a compiler is not present on a system, then a malware sample may install one or be packaged with the components it requires to recompile itself. This provides a signature for antivirus software to detect its presence. Alternatively, if the malware sample is developed exclusively in a low-level language, such as Assembly, then a metamorphic engine can swap out instructions without recompilation. This activity is observed in the second case study that focuses on a sample developed exclusively in a low-level language by an experienced malware writer, and as such, it does not require a compiler present to use effective mutation techniques when infecting new files.

4.5 Comparison of malware types

The different types of cryptographic malware that are focused on in this project have been compared to one another as if the techniques implemented by each one have built upon the last. Oligamorphic malware uses the least complex of techniques, and therefore it only has a limited number of combinations that it can use to hide its stub from antivirus software. Polymorphic malware is similar to oligamorphic malware in that its payload is encrypted in much the same way, but it addresses the weaknesses of oligamorphic malware by improving the obfuscation techniques used by the stub to evade antivirus software. No common features are shared between metamorphic and oligamorphic malware; metamorphic malware doesn't need to encrypt its payload and it does not require a selection of stubs to rotate between. Both polymorphic and metamorphic malware implement the same techniques to mutate portions of their code; however, polymorphic samples contain an encrypted payload and metamorphic samples do not need to do so.

It has been noted that polymorphic and metamorphic malware share common abilities to mutate portions of their code. Polymorphic malware mutates code in its stub and metamorphic malware

mutates its entire payload. Both accomplish this by implementing a combination of obfuscation and permutation techniques. Techniques can include but are not limited to [5, page 22]:

- renaming or encoding variables, subroutines and strings
- adding jumps across code
- adding junk code that is never executed
- adding garbage code that is unnecessary to the program
- using permutation techniques
- exchanging equivalent code
- splitting data structures
- adding anti-virtualisation and anti-debugging protection.

Each of the techniques listed above are further outlined below.

4.5.1 Renaming or encoding variables, subroutines and strings

The names given to both subroutines and variables after compilation could be used as indicators of the type of programming language that has been used to develop them. If programmed in a low-level language, such as *Assembly*, these names are most likely selected by the programmer and could provide a programming style to correlate with other samples produced by the same malware developer. Obfuscating this information by either renaming or simply re-encoding variables can aid in masking this data from being identified during static analysis of the sample.

Any string used in the sample, such as keys or copyright messages, could also be used to decrypt payloads or to develop an antivirus signature based on any of the developer's programming styles identified in the sample. Obfuscating the strings in a sample helps to protect the payload from analysis and also increases the chances of a sample remaining undetected for longer.

4.5.2 Adding jumps across code

If a sample's code follows the workflow in a linear fashion, then an analyst will easily be able to follow its logic and map out its purpose. By moving subroutines in the code to different locations, it causes an analyst to have to constantly adjust their position in a non-linear manner, and mapping such a network of jumps across a code base can prove time-consuming to conduct manually. The mapping process for a non-linear code sample can be automated using tools such as *IDA Pro*, and so this obfuscation method is most powerful when combined with other methods.

4.5.3 Adding junk code

Figure 4.5 depicts an example of two functions that, when passed through a process that inserts junk code, have resulted in a section of code that is more challenging to analyse. In this example, a new subroutine has been inserted, however it is never executed when the program is run. The discreet code in the example has bloated in size, and when this process is carried out on a large-scale section of code, it increases the time required to carry out a manual analysis of the sample.

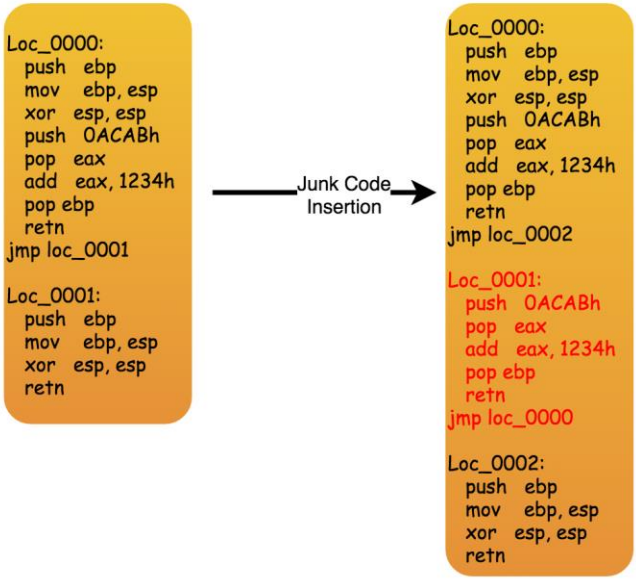


Figure 4.5: Insertion of junk that will not be run, confusing manual analysis.

Adding junk code provides a challenge for an analyst as they will need to filter through this additional content, and in some cases, they may end up being fooled into attempting to understand what the new sections of junk code are trying to accomplish, wasting time in the process. Tools such as IDA Pro can be useful when trying to automate this task as they only map subroutines together that actually call one another.

4.5.4 Adding garbage code

Similar to junk code, adding garbage code involves the insertion of code within a subroutine that tries to confuse the overall understanding of what the subroutine is doing. Figure 8.5 in the second case study illustrates this process and demonstrates the insertion of junk code during the mutation of a sample.

Although junk code is never run and can be easily filtered out, garbage code is executed and so it is included in the output of tools such as IDA Pro. The garbage code is executed as part of the subroutine but it doesn't affect the normal operation of the subroutine prior to the garbage code insertion process. The garbage code usually consists of common calculations that do not affect the subroutine but instead form an additional set of functions that need to be examined by an analyst before being discarded.

4.5.5 Using permutation techniques

By taking a simple piece of code and then moving sections of it around using jumps across the code whilst maintaining the original flow, the overall syntactic logic of the code is unaffected. As a result, mapping the code manually becomes more difficult for an analyst. Figure 4.6 shows an example of a simple piece of code that has been reordered whilst maintaining the original flow using links.

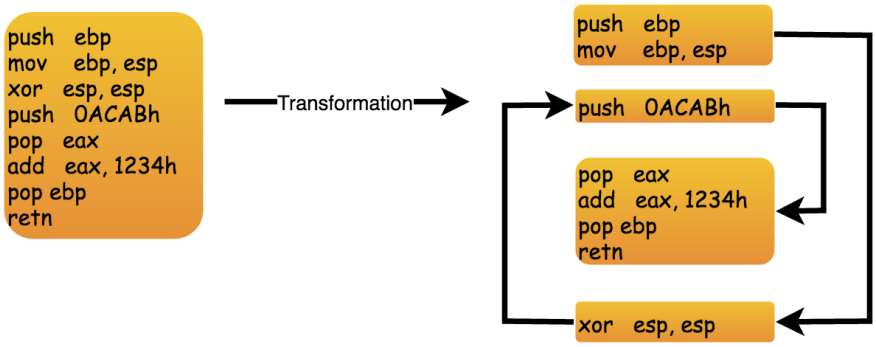


Figure 4.6: The re-ordering of code to confuse manual analysis [11, section 6].

The flow of the resulting code is difficult to follow when trying to map it out manually or when using tools such as IDA Pro to try and automate the process. What initially started as a simple piece of code to understand is now a series of jumps that are more difficult to digest, and this is further compounded when examining a large-scale sample.

4.5.6 Exchanging equivalent code

This process identifies sections of the code that can be exchanged for different pieces of code that can accomplish the same operations. Figure 4.7 shows the workflow of such a process. In this example, an operation has been identified as containing other ways of accomplishing the same operation. An alternative way of expressing the operation is randomly selected and swapped into place.

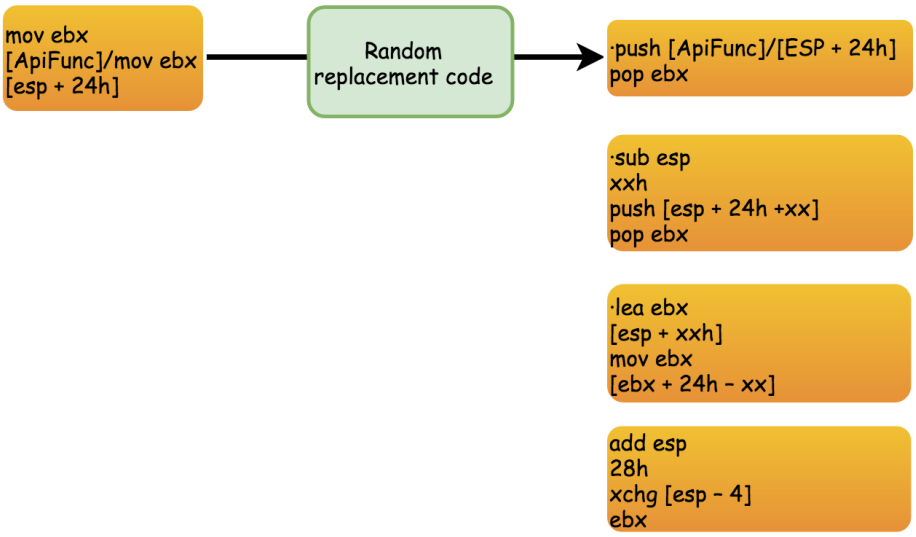


Figure 4.7: Illustration of the substitution of equivalent processes for what is functionally the same code [12, section 6, para 11].

This process can result in the original code bloating in size due to repeated generations of the sample being built using the previous generation's code. Over time, common sections of the code are replaced with functions that are then repeatedly replaced themselves. If this process is not monitored between generations, it can cause older generations of the code to become inefficient and essentially slower. To ensure that the size of the sample is maintained, either an algorithm is required that compresses the code or tight management should be conducted over what functions are replaced.

4.5.7 Splitting data structures

Antivirus software can often form signatures based on the regular expressions within a sample. These signatures may be based on variables or subroutine constructs. By splitting these items into smaller sections and then linking them together, they can still execute as normal, but the signature is unable to get a full pattern match any longer.

4.5.8 Adding anti-virtualisation and anti-debugging protection

As described in section 4.3, malware writers will attempt to prevent analysts from obtaining the code from an obfuscated payload. If a malware writer is seeking to only harvest bank details and log-in information for websites that have been visited on an infected device, then these operations are likely to be carried out using a physical device rather than using a virtual machine. It is becoming increasingly common for malware analysts to rely on the convenience of virtual machines to simulate a lab environment. Therefore, if malware contains code that prevents it running on virtual devices, then, in this scenario, the sample will continue to run on targeted devices and, at the same time, it will make it difficult for an analyst to examine the de-obfuscated payload.

When conducting a dynamic analysis of a sample, analysts will often use a debugger to run a malware sample or attach it to a running process to interpret the instructions being executed by the sample. Similar to the use of anti-virtualisation techniques, the user being targeted in this scenario is not likely to run programs on their system using a debugger. Thus, by blocking the use of debuggers, the malware developer is also reducing the risk of an analyst interrogating their sample. In case study three, a sample is protected using an online platform and as a result is provided with anti-debugging capabilities. The anti-debugging techniques that are added to the sample cause a limit to be reached when it is executed, thereby preventing an analyst from attaching a debugger to the running process and dumping out its de-obfuscated payload from the memory of a device.

5 Cryptographic malware antivirus evasion

Antivirus vendors do not often provide a great deal of detail surrounding the inner working of their products, such as how their engines look for patterns in samples. Therefore, this section does not contain an analysis of the technology used by antivirus vendors as it would add little to the available documentation and academic papers that already cover this area in sufficient detail.

This project has so far focused on the techniques used by malware writers to evade antivirus software, and therefore the experiments that have been carried out in the case studies have involved understanding what the samples are trying to accomplish when attempting to evade detection. This section details both *signature-based* and *heuristic-based* techniques that have been observed as part of this project and in its case studies.

5.1 Signature-based detection

When data is stored and processed at rest in a device, such as when writing data to disk, it often remains static. This provides a stable period during which a signature can be scanned for and a malicious sample can be detected. Signature-based engines can look at static content that has already been written to disk or they can look at the content as it is being written to a storage medium. This is the simplest and most common way of detecting malicious samples on a system by antivirus software.

A signature-based approach is more efficient to conduct on a system in terms of resource. One technique that is used is to calculate a unique hash for each file being scanned and then compare it against a database containing the hashes of known malicious files. Calculating a hash of a file requires little resource from a system, and as such, it is not detrimental to the system's performance to perform this check on every file as it is written to or already present on a device. This mechanism is therefore a common component of antivirus software.

During case study one, the sample was detected as malicious when submitted to an online site called *VirusTotal*; this site compares a sample across the signature databases from multiple antivirus vendors. The sample was already many years old at the time, but when a new file that had been newly infected by the sample was submitted to VirusTotal, a positive match for the malware sample was still detected. As the sample was polymorphic, a positive match was unexpected. The malware component should have resulted in a unique hash being generated for the file due to the newly infected file utilising obfuscation techniques, and it would therefore not be in the signature database of antivirus software. The detection of the newly infected file was due to a partial signature being developed by the antivirus vendor to detect elements of the polymorphic malware sample. This demonstrates that another ability of antivirus engines is to search for regular expressions within the files themselves and look for partial signatures.

The sample used for the second case study was written in pure Assembly, and it too was successfully detected by the VirusTotal website. This is because the antivirus vendor's signature-based detection not only looks for strings in their regular expressions but also searches for recognisable bite sequences to build a partial signature.

Each antivirus vendor clearly uses different methods to detect samples, as most of the databases used by VirusTotal did not detect the samples that were used in this project's case studies, and those vendors that did successfully detect the samples had unique names for the variant.

It can take up to four weeks after the release of a malicious sample for it to be included in the signature databases of antivirus vendors [5, page 8]. The samples used in this project's case studies use more complex techniques to evade detection, and thus they have the potential to remain undetected for longer. A signature-based antivirus database can eventually be updated to detect polymorphic and metamorphic malware samples. However, in the short term, this approach offers very little protection against polymorphic and metamorphic malware samples.

5.2 Heuristic-based detection

It has already been noted that both a polymorphic and metamorphic malware sample's payload is protected when at rest, and so examining a protected sample offers little insight into what malicious activities are carried out when the sample is executed. Heuristic-based detection engines look at the types of activities being carried out by a sample when it is being executed. It can examine these types of activities by looking at the instructions and calls to external resources that are being executed and alerting if possible malicious activity is taking place, such as a malicious program making a call to an Operating System *Application Programming Interface* (API). Alternatively, a heuristic method of detection can also block known malicious activity as it examines a sample's execution on a device.

Heuristic engines estimate and predict the impact of an instruction being carried out by a sample as well as its malicious intent. As this method involve allowing a process to be executed so that it can be examined in greater detail than the sample being examined at rest, its resource requirements are greater than a signature-based method of detection. Sometimes, the heuristic analysis requires the disassembly of a sample for investigation [5, page 9]. Or, an even more resource-intensive requirement is to run the program in a sandbox environment before allowing its execution on a host. The workflow of sandboxing a process is to first load up a sandbox, run the sample in the new sandbox and then examine any calls it attempts to make to the Operating System, using regular expressions and signatures on the processes that are being executed. Once the sample successfully passes through this process without triggering any alarms, it is then executed on a system.

This process, however, could cause a delay that is noticeable to the end user, and they could become frustrated if they have to wait sixteen seconds for their email client to start up while it is being examined in a sandbox. Although antivirus vendors can build this functionality into their

software products, it is a balancing act to provide enough protection without impacting the user unnecessarily. Both signature- and heuristic-based antivirus detection techniques are ineffective against *zero-day vulnerabilities* [5, page 8]. As an example, a zero-day vulnerability may involve exploiting a piece of software that has legitimate access to an Operating System API, and therefore any calls made by the malware that egress through the vulnerable software will not be in any existing antivirus signature. Antivirus vendors could err on the side of caution and use *behaviour blockers* [5, page 10] when any activity is detected that is suspected to have become malicious, prompting users to confirm that they are happy with the access being granted. However, this has the same drawbacks as a false positive in that when a malicious program eventually triggers an alarm, the user may already be desensitised due to having to acknowledge such alerts on a regular basis, and by habit, they may allow the malicious activity to continue anyway.

6 Malware analysis and using a development lab

6.1 Why do we need a lab?

Performing a forensic analysis on malware involves an element of risk. Often, dynamic analysis of a sample is necessary and this involves executing either a part of or the entire malware sample. Alternatively, tools may be obtained from unreliable sources to analyse the sample's obfuscation techniques. An analyst's workstation may contain personal information pertaining to the user themselves or to the organisation owning the device. Infecting the analyst's device could therefore expose this personal information outside of the organisation.

As a result, a degree of separation is required to analyse samples and the *principle of least privilege* should be observed when providing access to resources for the segregated environment, such as access to internet resources. It may be necessary to execute a sample and monitor any online resources that it attempts to access on the internet. It is also desirable for the sample to have enough resource so that it can unpack its payload, but not enough to succeed with accessing resources that could expose the analyst's development environment to unnecessary risk or provide the malware developer with an indication that their sample is being analysed.

The book *Practical Malware Analysis* [13] presents a good design for a malware analysis lab. This design has been adapted specifically for use within these case studies; the basic principles are followed and a virtualised environment has been used.

It is possible to use an environment consisting of physical hardware. This has its advantages if the malware samples that are being analysed look for virtualised hardware and, if they are detected, result in the malware sample remaining inert. Most analysts are reluctant to edit a malware sample to disable anti-virtualisation techniques. Therefore, their only options are to run a hardware lab or adapt their virtual lab to evade detection. A hardware lab has the added overheads of requiring more physical space, more power consumption and a higher overall cost. Every time the environment needs reverting to an uninfected state, the hardware lab would need to be wiped and re-imaged. Even then, there is no guarantee that a malware sample has not already infected any hardware storage, such as the firmware. Being able to restore a virtualised lab from a snapshot in a few minutes is far more convenient. Furthermore, by using a virtual lab, the analyst also gains flexibility and portability of the lab itself.

6.2 Infrastructure of the lab

A full description of the architecture for the lab used in this project, along with demonstrations of how to install it, are available on the author's YouTube channel at: https://youtu.be/Ddiytw0nUYU. A top-level description of the lab's architecture is also included in this document.

Figure 6.1 shows a top-level diagram of the lab used as part of this study. A virtualised environment is being used instead of a hardware lab. The types of malware being analysed in this project's case

studies did not implement anti-virtualisation techniques that prevented the use of a virtualised lab. Therefore, there was no need to run the malware samples on physical hardware to force the unpacking of their payloads.

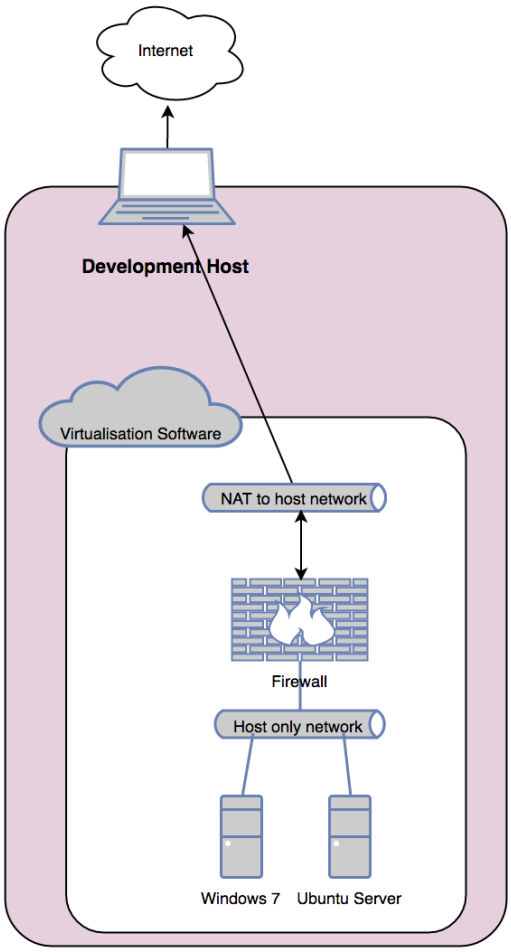


Figure 6.1: Overview of the lab used in this project.

Running the lab in a virtualised environment means that the workflow followed in this study allows for a snapshot of the environment to be taken at regular intervals, and so as dead ends in the investigation occur, the lab can then be restored to a previous state and the investigation can continue with little interruption. As an example, it is easier to execute an infected file, conduct initial dynamic analysis of the sample and when complete, restore the lab back to a desired state prior to infection. Overall, this can save an analyst some time because the machine does not need to be rebuilt after each test infection. An analyst can be a bit more cavalier and run malware payloads more often without the need to forward plan if the lab needs to remain clean for any future stages of the investigation.

Some malware samples have anti-debugging capabilities that prevent an analyst from attaching a debugger to a malware process running on a machine and obtaining a memory dump for further static analysis. In these situations, it is a great time-saver to be able to snapshot a machine and then use the memory dump from the snapshot for quick analysis.

6.3 Building the lab

There are several software vendors that can be considered when selecting appropriate virtualisation software to host the lab, such as *Oracle*, *VMware* and *Parallels*. Free software, such as Oracle's *VirtualBox*, can be used although this software proved unstable at times when running the case studies in this project. During a security incident, it is important that the tools being used do not slow down the analysis of a sample. Paid-for software solutions, such as Parallels and VMware products, could therefore be a more efficient option.

The studies conducted as part of this project were carried out using a lab hosted on *VMware Workstation* and *VMware Fusion*. When dynamic analysis of malware is prevented by a sample implementing anti-virtualisation techniques, being able to snapshot a device and interrogate the snapshot's memory can provide a quick solution, and the snapshots by VMware provide this quick and easy access to a device's memory.

The use of a hypervisor to host the lab was also tested and provided the same functionality as using VMware on a workstation. Appendix 12.1 provides a brief description of the benefits and drawbacks to this approach.

The virtual machines running on the lab used as part of this study were the following.

- *Firewall-1* – this machine was a *Sophos UTM Home* firewall. The software is free and includes useful functionality for future analysis of other malware.
- *Forensic-1* – a Windows 7 desktop machine was used rather than older/newer versions of the Operating System. Windows XP is end-of-life and some software no longer supports it, whilst Windows 10 includes built-in antivirus protection that needs to be disabled before conducting any forensic analysis of malware. Windows 7 instead allows an analyst to simply build a machine from evaluation images or fully licenced copies, and then copy the tools and malware samples on to the lab machine before starting an investigation.
- *Forensic-2* – an *Ubuntu* desktop machine was used to host software, such as *Inetsim*, allowing for malware DNS and service requests to be spoofed via the Ubuntu machine. Other Linux specific tools were also tested on this host and in case study three this machine was used to compile tools.

A video demonstrating the set-up of the lab used as part of this project is available via the YouTube link referenced in section 6.2.

6.4 Risks to consider whilst operating in the lab

There are inherent risks with running malware in the lab environment. Caution must be exercised and focus maintained, especially when managing devices that contain multiple snapshots of a machine, as mistaking the state of a machine can affect the results obtained from tests being conducted on a sample. As an example, if an analyst mistakenly reverts a lab to a state where the machine is still infected and they believe that the lab is in a state prior to infection, then any tools used to analyse the sample may themselves be infected and could cause inaccurate results.

Ideally, the development device hosting the lab architecture should be on a different platform to the virtual machines being infected as part of any investigation. In this way, if a malicious payload is capable of spreading like a worm, then the host machine has a reduced chance of infection if the malware sample is not designed to target the host machine's Operating System. Therefore, the experiments conducted as part of this study used an *Apple MacBook Pro* (Mac) for the host machine because the malware samples in case study one and two specifically target Microsoft Windows devices. If, as a result of company policy, there is no choice other than to use the same Operating

System as the machines hosted in the lab environment, then the host machine should have all available security software patches, and at the very least, it should have antivirus software installed. In this study, even though a Mac was used for the host development machine, it still had *ESET* installed to help protect against infections breaking out of the lab environment.

Most of these virtualisation software offerings can share a folder between the host machine and the lab devices to allow for fast file sharing between the host development machine and the virtual machines. This option should be used with caution. It is likely that any malware samples copied to shared folders that are accessible by the host machine will be quarantined or deleted by its antivirus software. If any malware samples can encrypt or remove files on the lab devices, then they could potentially do the same to any file storage that is shared with the host development machine.

In the studies conducted as part of this project, files were copied to the virtual machines from a *Dropbox* (http://www.dropbox.com) account. Any artefacts obtained during the investigations were also copied to this online file share for later analysis and archiving, providing an area of separation for files shared between the lab devices and the host machine.

Some malware may specifically target virtualised hosts in the hope that the virtualisation software or hypervisors in use are unpatched, meaning that they can exploit any unpatched vulnerabilities that have the potential to break out of the virtualised environment. Again, caution should be exercised when investigating such malware. One option to prevent this is to execute a malware sample in a virtual machine that has been provisioned on a host that is sandboxed from other devices and sacrificed only for this type of testing. Once the malware sample passes this initial quarantine measure, it can then be signed off as safe to be examined in the lab environment.

7 Case study one – manual deconstruction of polymorphic malware

7.1 Background

Section 4 has so far detailed what it is that can make malware polymorphic in nature and what differentiates it from an oligamorphic sample. The section also outlined and provided some diagrams detailing the techniques used to pack a payload and make it polymorphic.

This case study examines a complex polymorphic sample that was once commonly found on infected devices and was widespread due to its polymorphic techniques that aided it in evading antivirus software. The sample used for this study was a strain of the *Virut* malware, which was very popular during 2000–2010. Studies by Kaspersky Labs [12, section 2, para 5] show that between 2009 and 2010, the *Virut.ce* strain was one of the most popular viruses found on infected hosts. Therefore, the *Virut.ce* strain was chosen as part of this case study to examine its polymorphic techniques.

The techniques used to both create the lab environment and obtain the de-obfuscated payload for the sample are documented in this study and have also been recorded, with links to the videos found via the GitHub link provided in section 2.1.

As polymorphic samples can vary due to their changing signatures, the *Virut.ce* samples were obtained from two different sources and compared with one another. The samples were obtained from two virus repositories called Offensive Computing and VirusShare[1]. Offensive Computing is widely used by analysts to obtain samples, but it was not solely trusted for collecting the samples used in this project due to its open community. As such, a site called VirusShare was also used to obtain the primary sample that was to be analysed. VirusShare is a member-only site and requires its members to recommend others before they can also register an account. Therefore, samples

[1] https://github.com/devopscrazy/MalwareAnalysis/tree/master/virtu.ce/
(SHA-256 hash = 57243f589c28a012f9bfaef3504cf5cec1f19f14e5ff4793a21d00e034c77755, Samples' password = "infectedIYM011")
For more information refer to appendix 12.5.1

from this site could be perceived as having a greater reputation than those from sites that allow samples to be submitted by people with a degree of anonymity.

Before investing time in analysing a sample, it was submitted to the website VirusTotal to confirm it was a *Virut.ce* malware strain. This site compares samples against a signature database for a multitude of antivirus vendors. Using VirusTotal confirmed that the samples being examined were indeed *Virut.ce*. Interestingly, some of the major antivirus vendors, such as *ESET*, did not detect the malware. This was later confirmed by copying the payload onto a machine protected by *ESET*. The lab used in this case study had Sophos home edition installed on the host environment. It was therefore tested to make sure that the sample used in this case study triggered an alert if it was copied onto the host environment, so that any infection that escaped the lab environment would be detected.

7.2 Scope

This paper has been designed with the intention of aiding the forensic analysis of malware in organisations. An exhaustive explanation of the actions carried out by the payload once it has been unpacked and executed is therefore out of scope for this study. Instead, the primary focus of this investigation is to reverse engineer the protection used by the *Virut.ce* malware sample and to examine how to identify specific techniques used by malware to obfuscate themselves, using this knowledge to unpeel further layers of security that have been wrapped around the malicious payload.

The sample of *Virut.ce* will be examined predominantly in Assembly because it is straightforward to disassemble and examine Assembly language in a debugger. Many malware samples are written in languages that are compiled to a *portable executable* (PE). Examples of these languages are *C* and *C++*. Scripting languages, such as *Visual Basic* (VB) and *Bash* or *Shell* scripts, are sometimes found within a malware's payload but rarely constitute the main portion of the malware's obfuscation layers. When a program is compiled to a PE, it is stored in machine-readable instructions known as Assembly. As documented in section 4, when looking at malware obfuscation techniques, Assembly can be subjective in how it is interpreted by a device. Simply adding a line to a program, such as a jump in the code, could cause it to be decompiled to something that appears completely different. As a result, malware writers often write their samples directly in Assembly as it is easier to write non-standard and unobvious code that reaches the same syntactic conclusion as a piece of code that would be easy for an analyst to interpret. This means that malware is often bloated and difficult to follow by eye as it can jump from section to section of the payload, and in some places it may even do nothing and is designed to simply waste an analyst's time. Assembly has powerful access to an Operating System's APIs and it can allow a malware writer to detect signatures of their own to identify information about the host that it is executed on. This means that a malware writer can look for hosts that are virtual or that are running in a debug mode, as well as identify hosts that are running a certain Operating System to then exploit a known vulnerability. As detailed in section 6.1, the malware may be used as a platform for organised crime, and as such, may only be interested in harvesting personal information, therefore workstations would be the best place to locate this personal information, such as online banking details and credit card details. As a result, these APIs are used to identify information that could indicate the sample is being reverse engineered.

Disassembling the sample and then performing an extensive search of all obfuscation techniques used by the sample in the Assembly code is not possible as part of this investigation. Different samples require different techniques to extract their payloads, and so an extensive explanation of the obfuscation techniques used by this strain provides little help when investigating another type of malware. There is no "one size fits all" when it comes to reversing malware and there is no methodology for completely analysing such a sample. Therefore, this case study will document the main steps taken to extract the payload and identify some obfuscation techniques used by this

strain. The scope of the documented steps will include those that are most common when examining polymorphic malware.

7.3 Reverse engineering the sample

7.3.1 Examination of the sample's behaviour

The first step is often to open the sample in a disassembler of the analyst's choice and examine the Assembly used by the stub of the program. IDA Pro is a fantastic tool for disassembling a program. It not only provides the Assembly instructions performed by the program but also examines the jumps and calls in the program, rearranging the view into a logical flow. A program rarely executes its instructions in a linear order and when observed by an analyst, a program could start at an instruction in the middle of the code and proceed to jump up and down through the code using calls. Figure 7.1 shows that when opening the *Virut.ce* sample in IDA Pro, the disassembler warns that the sample has most likely been packed and that instructions carried out during the execution of the program would not necessarily reflect the Assembly being shown.

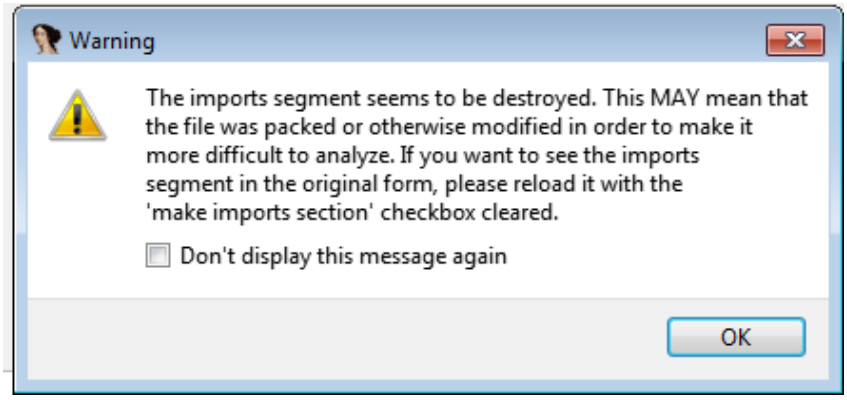


Figure 7.1: IDA Pro disassembler warning of a packed sample.

When viewing the sample in a disassembler, the stub itself looks small and it contains loops that load registries and then conduct *XOR* operations. This indicates that a key is being loaded into memory and is then being used to unpack a payload using multiple *XOR* operations. Figure 7.2 shows an extract from the sample.

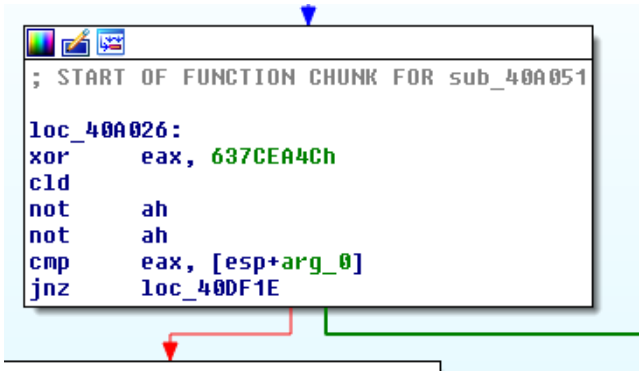


Figure 7.2: An XOR operation that suggests the payload may be packed within the sample.

This can be further confirmed by passing the Assembly through a de-compiler to display in a high-level language such as *C*. IDA Pro offers plugins that can render the currently viewed Assembly into an easy-to-read *pseudo code*. Figure 7.3 shows an extract of the pseudo code rendered using *IDA Pro's Hexrays plugin*. In the sample, this pseudo code was used to ascertain that the stub uses a loop that has all the hallmarks of a standard packer. However, these types of instructions are not only

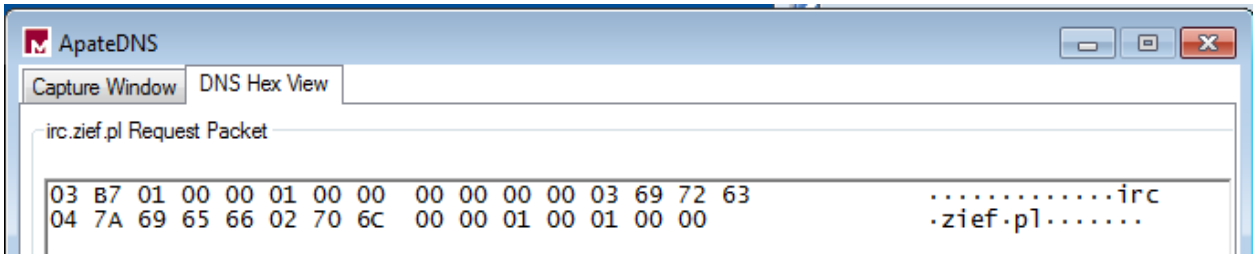
present when polymorphic malware is decrypting its content but also when programs are packaged using standard packers. Therefore, further testing should be conducted to identify whether other packers have been used. Often, if code is being executed from the *.rsrc* or *.reloc* sections of a program, it is a good indicator that a file has been infected with a malicious payload.

```
20  v15 = -1;
21  v8 = (int *)(a1 + v7);
22  v9 = -1;
23  do
24  {
25    v10 = *v8;
26    ++v8;
27    ++v9;
28    v11 = (_BYTE *)(v10 + a1);
29    JUMPOUT(v9, v6[6], nullsub_2);
30    v5 = 0;
31    do
32    {
33      v15 = v5;
34      v14 = _InterlockedExchange((volatile signed __int32 *)&v15, 16 * v5);
35      v15 -= v14;
36      v15 -= (unsigned __int8)*v11++;
37      v5 = v15;
38    }
39    while ( *v11 );
40  }
41  while ( (v15 ^ 0x637CEA4C) != a2 );
42  v2 = a1 + v6[8];
43  v3 = a1 + v6[7];
44  v4 = ~((unsigned int)v6 + 18);
45  result = *(unsigned __int16 *)(v2 + 2 * v9);
46  v12 = *(_DWORD *)(v3 + 4 * result);
47  BYTE1(result) = BYTE1(v4);
48  return result;
```

Figure 7.3: Extract of pseudo code generated by IDA Pro's Hexrays plugin showing that a loop is used, similar to most packers.

An overview of the malware sample and details of its test execution within the lab has been recorded and is available online via the GitHub link referenced earlier in this paper or via the direct link at: https://youtu.be/Ozxd3KFPXdg.

When executing the sample to examine its behaviour in *Wireshark* (https://www.wireshark.org/) and *ApateDNS* (https://www.fireeye.com/services/freeware/apatedns.html), it is noted that the sample attempts to make a request to a website at the domain rus-pazari.com, but more interestingly, it also attempts to connect to a resource at an *Internet Relay Chat* (IRC) address. Figure 7.4 shows the IRC connection attempt that was collected by ApateDNS.



7.4: The malware sample attempts an IRC connection to *irc.zief.pl*.

Another behaviour observed in the sample is that while it is executed on a device, it can infect other programs that are present. A report by Avira [14, page 1] details that the *Virut.ce* sample hooks into the *ntdll.dll* API. This provides the malware with access to infect any .exe file that is executed with an

argument for *CreateProcess* or *CreateFile* APIs. If a standard program that comes with the Operating System, such as the *CMD.exe* file, is copied to the desktop and executed while the malware is being run, then the *CMD.exe* file will be altered. As detailed in section 7.3.3, this change that is visible in the file is because the malicious sample's polymorphic payload has been appended to the file. Figure 7.5 shows the resulting hash change of the *CMD.exe* binary as it is infected by the polymorphic sample called malware.exe that is running in the background.

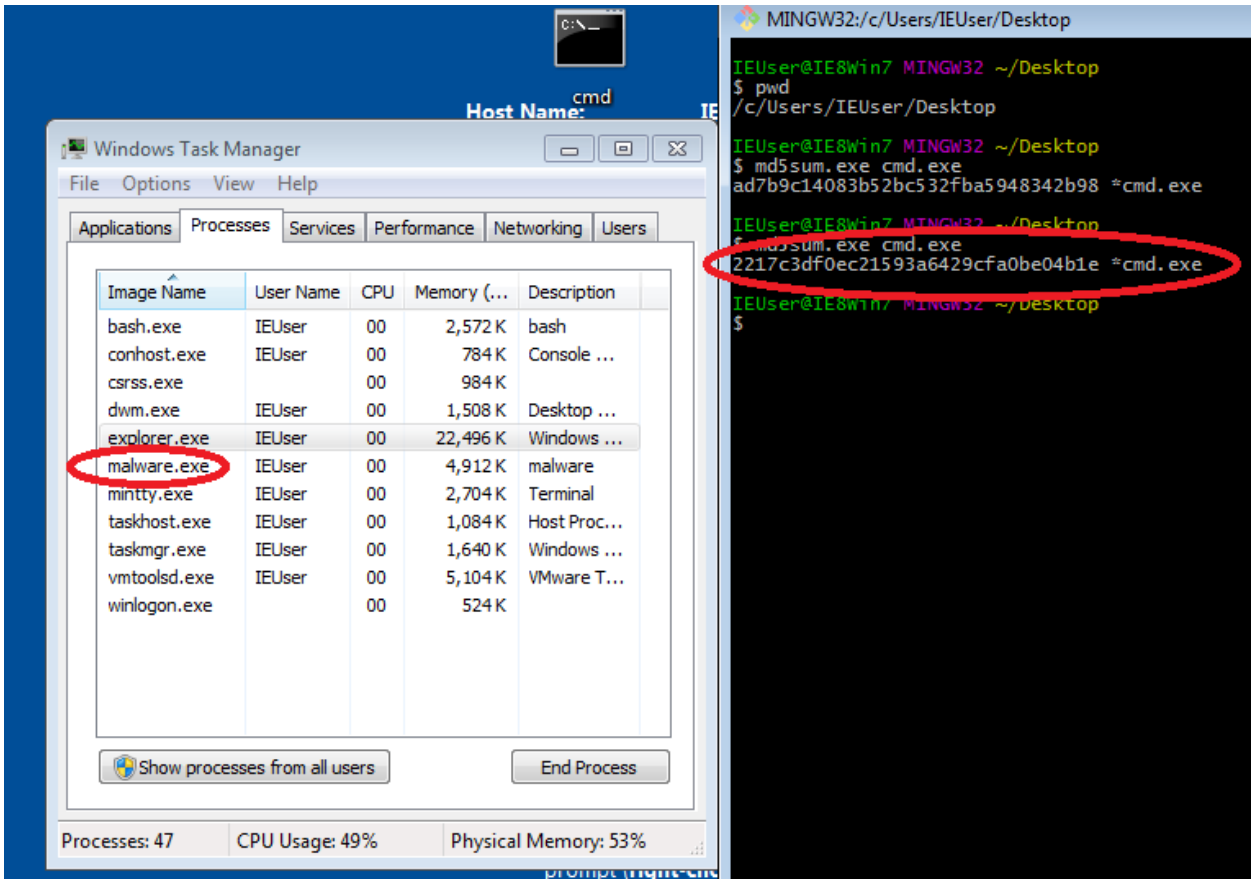


Figure 7.5: A copy of *CMD.exe* being infected by the malware sample, showing a hash change.

At this stage, it is now known that the malware tries to contact command-and-control servers, one possibly being an IRC service, and that the malware is capable of infecting other files on the device. Initial static analysis has also identified that the sample's outer layer is packed.

It is useful to pass samples through programs that are specially designed to detect if a program is packed as it can allow the possible identification of the packing software that has been used. Utilising these tools can speed up an analyst's tasks and provide confidence that a sample is indeed packed before investing resources to unpack it further. Many of the tools used in this case study could detect if the samples are packed as well as indicate what it has been packed with. By confirming that a sample is packed, it provides an analyst with a direction to proceed in and they can then attempt to unpack the sample manually or use an appropriate tool. Multiple tools should be used to scan a sample for any obfuscation that may be in use, as older versions of tools often do not correctly detect that more recent packers have been used, or they incorrectly report the piece of software that is believed to have been used to obfuscate a sample. *PEiD* has been used to scan the sample in this case study; this is an older and unsupported tool for detecting packers, but it is still a powerful piece of software and includes plugins that were utilised in section 7.3.2. The sample being investigated as part of this study was not detected as being packed by PEiD, as shown in figure 7.6. This is most likely due to the age of the tool and the lack of active development for some time

resulting in new packers not being included in the tool's signature database. Alternatively, it may not have been detected due to the sample being manually packed in some way.

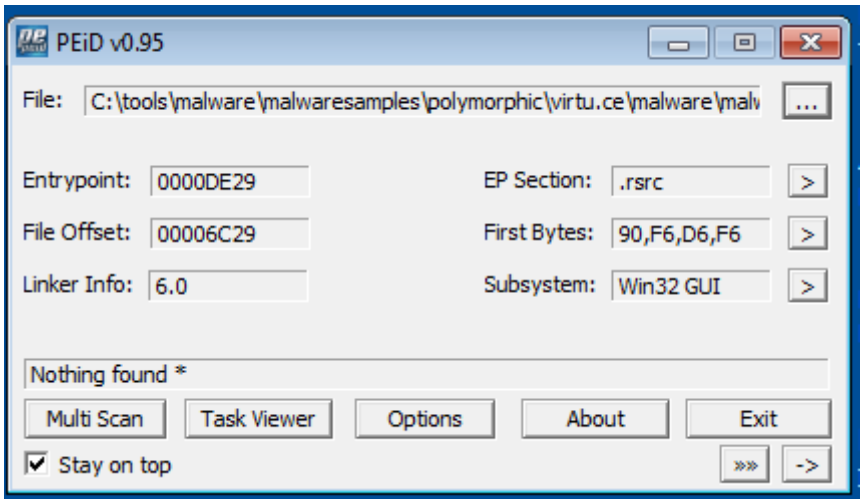


Figure 7.6: PEiD being used to check for a packed sample.

Opening the sample in *RDG Packer Detector* (RDG) showed that the sample was not only packed but also that it was packed with UPX, and furthermore, the version used to pack it was *v2.00*. Figure 7.7 shows this tool being used, and it is worth re-iterating here that sometimes it can be risky to use a new tool as it may also contain malware that can be used to infect an analyst's development environment. The program should always be run in the lab environment or on a machine with up to date and effective antivirus software installed. Preferably, snapshot the lab environment prior to running the tool and then copy out whatever resources or evidence needed for further investigation before restoring the lab back to its last good state.

Figure 7.7 RDG successfully detecting that UPX was used to pack the sample.

All the above steps of initial examination have been recorded; the video is available at: https://youtu.be/Od3EAcm-mdM.

7.3.2 Unpacking a sample's outer layer of protection

From this point onwards in the case study, most of the techniques that are outlined involve executing the sample to attempt to extract its payload, and therefore an analyst should exercise caution as they proceed. Ensure that the lab environment used is set up and segregated in the same way that is documented in section 6.2. Make sure that the lab is separated from other machines on

the network and that the security methods detailed in section 6.4 have been observed before embarking on any active examination of the samples.

As stated above, running automated tools to try and unpack samples will involve their execution and so extreme caution was exercised while attempting to unpack the sample in this study. As RDG detected that UPX was used to pack the sample, it was sensible to check if that packer was readily available and, if so, that it had an option to unpack the sample. Luckily, UPX is available to download and so it is easy to obtain and check its options by running it and passing the parameter "-h" for help. This shows an option "-d" to decompress a program, which is then used to decompress the sample. Comparing a hash of the original sample with the one after the sample was decompressed with UPX indicates that the sample was modified by the process of decompressing, and that it was unpacked to the next stage of execution. However, the payload is not unpacked very far, and although more code is now available for analysis, it is far from what had initially been expected. A good indicator of this was to use IDA Pro to search for strings in the unpacked sample, however, there were no extra strings available and so more layers of unpacking were needed.

It is worth noting that although an analyst could try and manually step through the sample's stub, either by using the code alone or using a debugger, and eventually work out how to unpack the sample, this is a time-consuming task and may involve working around any anti-debugging techniques used by the packer. In a major incident, it may make a huge difference in time to peel off the first few layers of a sample quickly and then spend more time on working out what the underlying payload is doing, rather than wasting time trying to unpack layers of obfuscation manually that could be more efficiently carried out using readily available tools.

As using UPX itself did not remove all the layers of unpacking necessary to start examining the polymorphic portion of the sample, it was necessary to look at other tools. Although PEiD had not originally detected the packed program, interestingly, it has a plugin for generic unpacking as shown in figure 7.8. This plugin was effective at unpacking most of the outer layer of protection used by the sample.

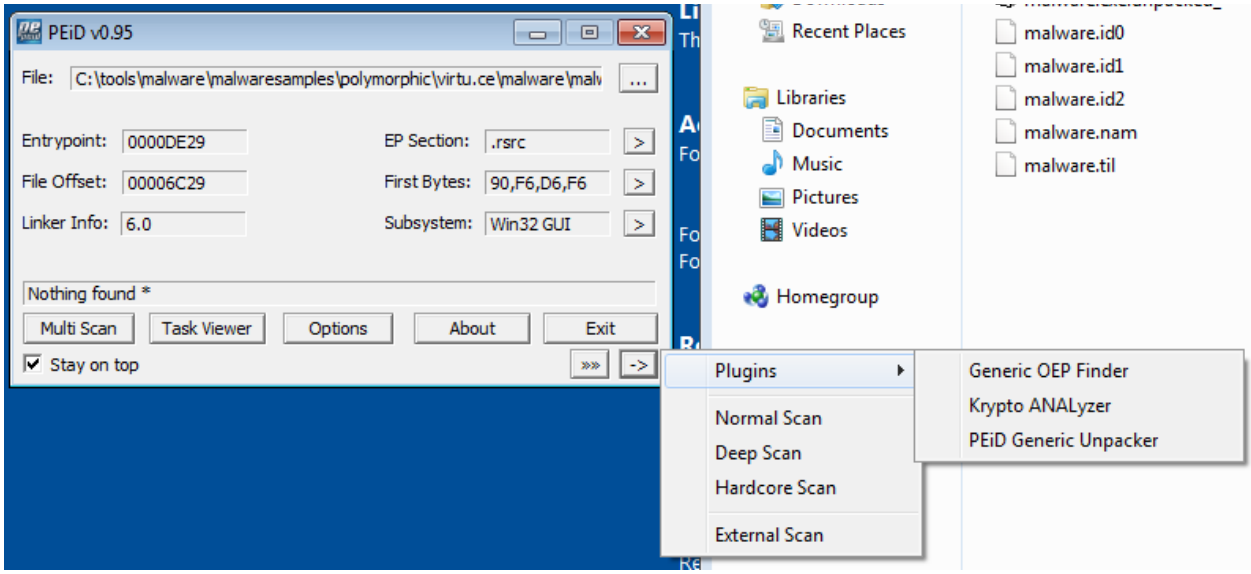


Figure 7.8: PEiD includes plugins, one of which is a generic unpacker.

When comparing the unpacked sample that had been extracted using UPX against the result of unpacking the sample with the PEiD plugin, the difference was noticeable, as shown in figure 7.9. Far more functions were now available in the unpacked sample that used PEiD. Also, more complex

subroutines were now visible, most of which run complex procedures that appear to aid in simply running *XOR* commands across the encrypted payload.

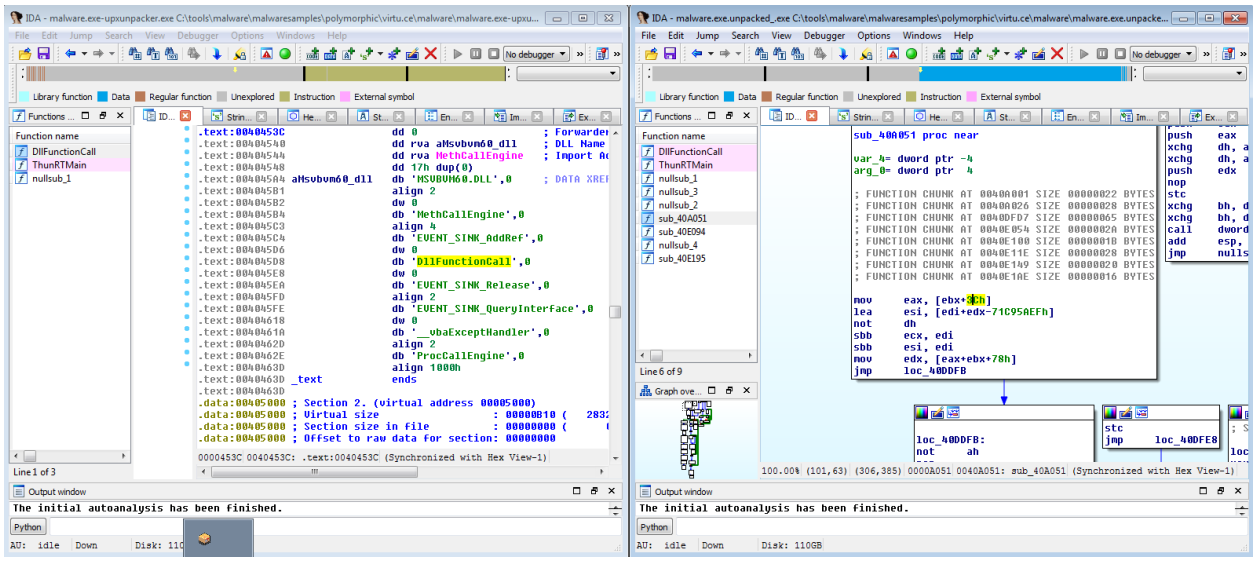


Figure 7.9: Comparison of a PEiD unpacked sample shown on the right and a UPX unpacked sample shown on the left.

Examining the unpacked sample in IDA Pro shows constructs that suggest a VB program is being executed. This is not the behaviour that is expected from this sample as the unpacked polymorphic section was expected to be in a low-level language such as *C* or pure Assembly. Figure 7.10 shows the results of running the sample through RDG, once again confirming that the sample is indeed a Microsoft VB program. It must therefore execute further obfuscated code somewhere in the program.

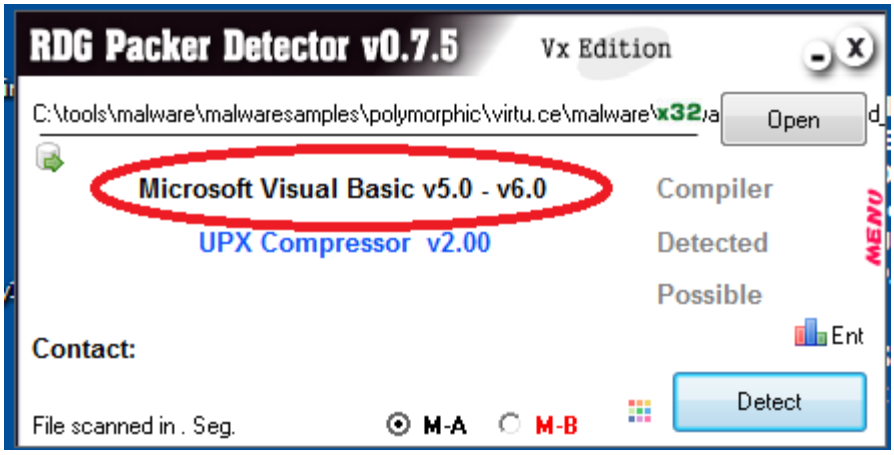


Figure 7.10: Microsoft VB code detected after unpacking the sample with PEiD.

Using a VB decompiler to examine the sample that has been unpacked with PEiD shows that the stub is a different piece of malware, and figure 7.11 contains an extract of this code. It uses VB to utilise the machine's browser and bypass security controls to download and install a malicious payload called *ZangoSA*. As this download is no longer accessible, the malware gets stuck in a sleep cycle which it never exits. This explains why a portion of the malware causes a DNS call to the domain *www.rus-pazari.com*. Although this is interesting, it is not the desired polymorphic payload and shows that the sample is packaged with other pieces of malware. The VB portion of the payload does not perform any IRC requests, and so this is a good fingerprint to look for when confirming that

the polymorphic portion of the malware payload has been unpacked fully. Ideally, it would be easier to examine the techniques used by the polymorphic portion of the sample if it is separated from the other malicious sections that are packaged with the sample.

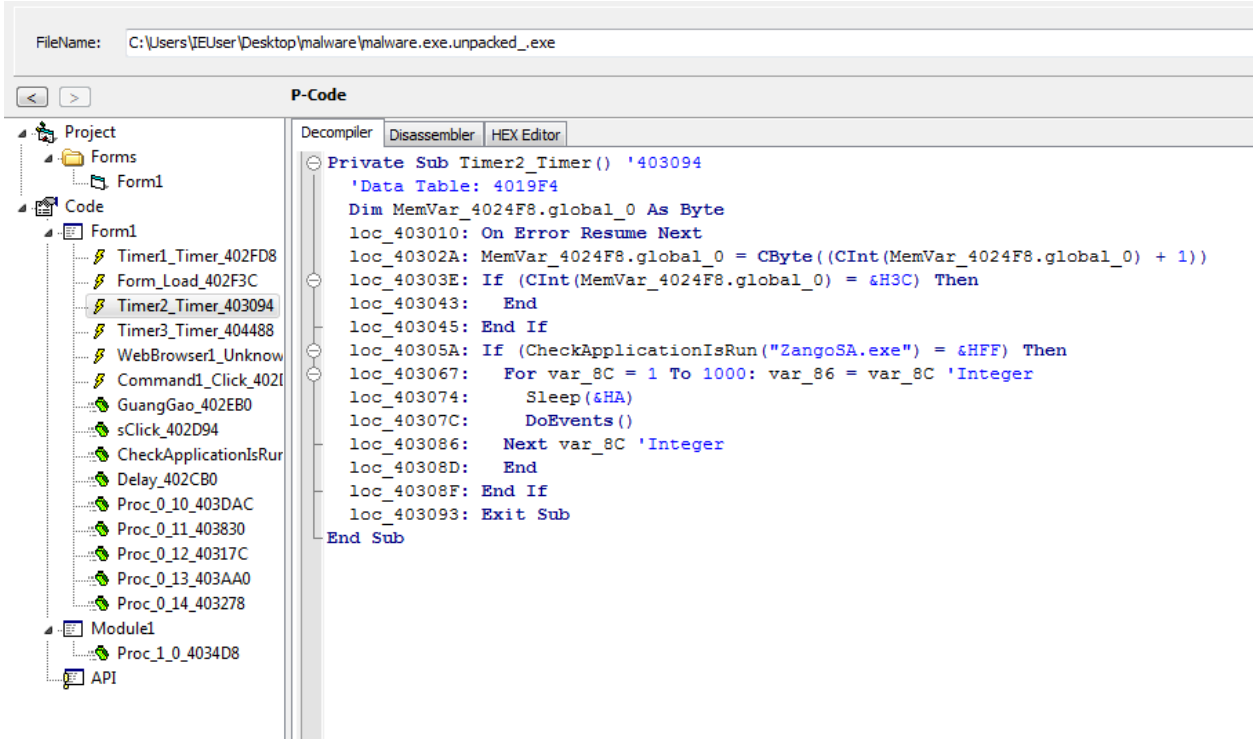


Figure 7.11: The unpacked stub contains constructs to install *ZangoSA* malware.

When the sample was executed and its behaviour observed in section 7.3.2, it was noted that the sample was capable of infecting other files on the device. Once the first layer of unpacking is completed by PEiD, the sample is no longer capable of infecting new files. It can still run the *ZangoSA* payload but it can no longer execute the polymorphic part of the sample that is of interest in this study. Therefore, examining the polymorphic code responsible for infecting new files is not possible when the initial layers of packing is removed. This could be some form of protection that the malware uses against an analyst attempting to unpack the payload, or it could be that the initial entry point to the polymorphic stub is damaged during the unpacking process.

The steps carried out in this section can be observed in the recorded video at: https://youtu.be/Od3EAcm-mdM.

7.3.3 Baiting the sample

The sample obtained from VirusShare contains too many other malicious packages. During the forensic examination, it is desirable that only the polymorphic payload is present. Dumping the sample out of memory when executing it will also contain the extra payloads, and it may still be missing parts of the original payload due to obfuscation techniques. Due to anti-debugging techniques being implemented by the sample, an analyst is unable to use a debugger to step through the unpacking process and obtain the unpacked payload at a point that it is no longer protected.

The solution used in this case study was to build a custom program and bait the malware into infecting it. Consequently, the known sections of the custom program can be discarded during analysis leaving just the polymorphic malware portion of the sample. Later analysis simply involves tracing the program and disregarding any known instructions from the original program. It is simple

to write a sample program in Assembly that displays a message box, and the custom sample used in this case study is shown in figure 7.12. By displaying a message box, it ensured that the sample would stay open while running further tests, such as taking a memory dump or connecting a debugger to the executing process for analysis of the unpacked polymorphic code.

```
.686p
.mmx
.model flat


; Segment type: Pure code
; Segment permissions: Read/Execute
_text segment para public 'CODE' use32
assume cs:_text
;org 401000h
assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing



public start
start proc near
push    40h             ; uType
push    offset Caption  ; "PROGRAM!"
push    offset Text     ; "Goodbye world!"
push    0               ; hWnd
call    MessageBoxA
push    0
call    $+5
jmp     ds:ExitProcess
start endp
```

Figure 7.12: Custom sample code used as a sacrifice for infection.

It is then simply a case of running the sample malware and executing the test program to see if it will infect it. The test program can then be loaded into IDA Pro and it can be confirmed that the polymorphic stub has replaced the start function with one that will also execute the stub's decryption routines.

After infecting the custom-built sample, it is possible to see the obfuscated polymorphic malware portion of the program when viewing the infected sample in a hex editor and comparing it against the original custom sample. This is demonstrated in figure 7.13.

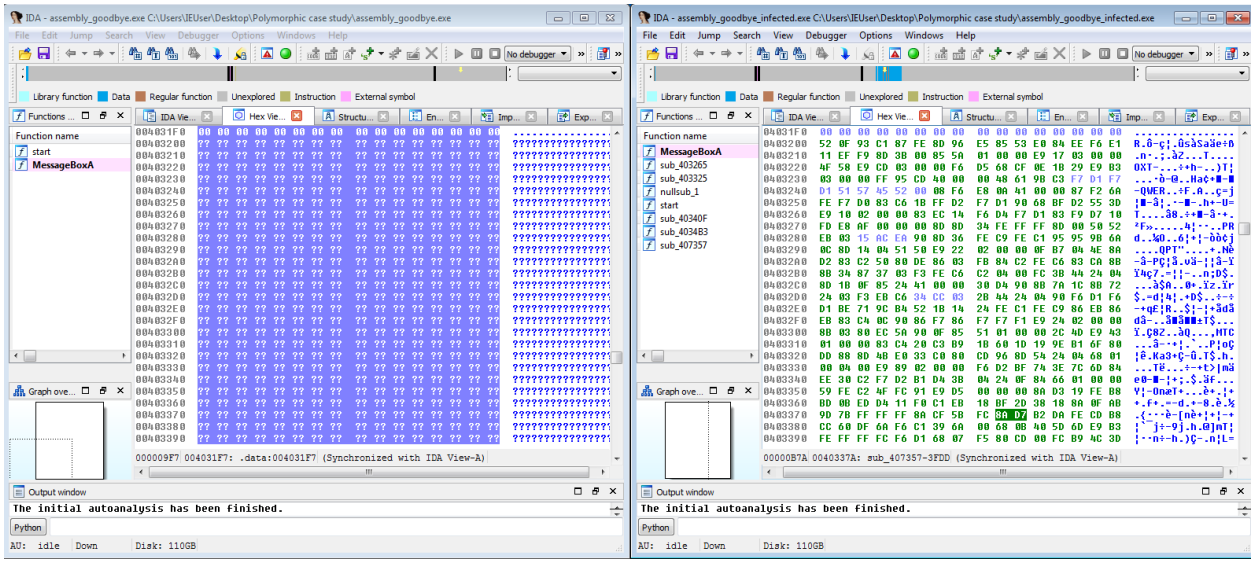


Figure 7.13: The malware portion of the infected custom sample is visible in the window on the right.

How the *Virut.ce* sample executes and unpacks its payload can be broken down into two parts. The first is by using EPO techniques to make any infected program initially execute the malware sample's stub. The second part of the process involves the sample's stub carrying out the main decryption of the payload.

The steps to build the sample program and bait the malware to infecting it have been recorded and a video can be found at: https://youtu.be/Goqd2RJXcaA.

7.3.4 EPO techniques

When infecting the test program used in this case study, the following modifications were carried out by the malware sample.

- The malware's encrypted payload was injected to the end of the test program's file.
- The initial entry point address in the test program was modified to point to the first instruction in the malware's stub.
- The malware stub was located at the start of the new section added to the end of the file.

The first instruction called by the infected test file switches to a new procedure that contains a jump to the main body of the stub, as shown in figure 7.14. By doing this, the malware evades being detected by antivirus software when the final jump to the malware stub is executed.

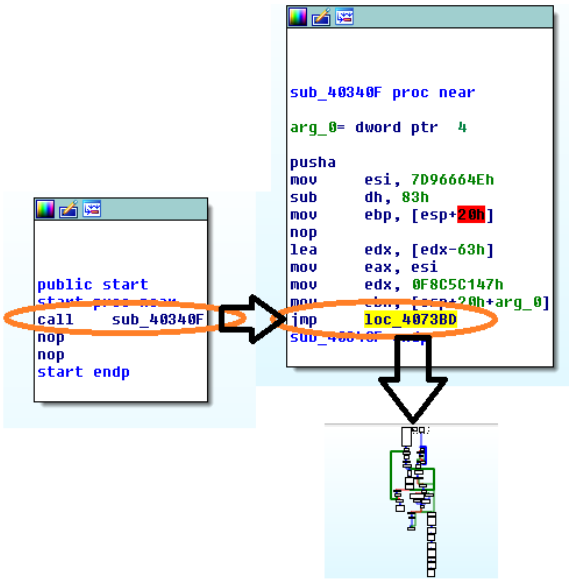


Figure 7.14: The first procedure calls a jump to the main body of the malware.

A whitepaper by Avira [14, pages 18-19] describes that there are only four ways that the *Virut.ce* variant can modify an infected file and ensure its execution. In all methods, the encrypted malware payload is placed at the end of the file.

1. Modify the Entry Point, so that the entry point of the infected program points directly to the malware stub located at the end of the file.
2. If suitable access to the *kernel32.dll* is available, then the malware can get a random call patched into the sample and point to the new malware instructions. This means an entry point to the code conducts an EPO jump to the malware's stub at the end of the file.
3. An initial decryptor can be copied to the end of the .text section, and the entry point is modified to execute it first so that it can then decrypt the main stub. Once the main stub is

unpacked by the initial decryptor, it then executes the malware's stub to decrypt the payload.

4. The final method also involves getting a random *kernel32.dll* call similar to option 2, but this time it uses the random call to point to an initial decryptor similar to the one used in option 3. Essentially, the entry point runs the legitimate file's code and then initiates an EPO jump to the initial decryptor. The initial decryptor unpacks the stub in the malware body at the end of the file and then passes control to the malware stub to then decrypt the payload.

The sample used during this case study seemed to exhibit the behaviour detailed in the final method documented above. When unpacking the first layer of obfuscation, the underlying VB program was lined up to be executed. The EPO jump was, however, corrupted as part of the automated unpacking process, but it would be expected that during the execution of the VB program, an EPO jump to an initial decryptor would have occurred, and that would have unpacked the stub of the malware itself.

Upon infection of the custom program in the previous section of this case study, the first of the methods was observed to infect the program. When loading the infected sample into IDA Pro and switching to a hex view, it can be observed that the first part of the program to be executed is in fact within the malware portion that was inserted at the end of the file.

7.3.5 Stub unpackers

The *Virut.ce* malware infection can contain two layers of encrypted instructions, as per methods 3 and 4 that have already been detailed in section 7.3.4 on EPO techniques. Using this nested approach appears to provide further obfuscation to help thwart antivirus static signature analysis. The malware payload will still be encrypted after the first layer of encryption is removed. The paper from Kaspersky Labs [12, section 4, para 5] regarding this strain of the sample provides a good overview of the processes conducted by the initial decryptor when it unpacks the stub from the sample.

The initial decryptor contains between *0x100* and *0x99* bytes of code and is tasked to:

- write the size of the current encrypted section to the register
- perform *XOR* operations on the encrypted code section using a key
- adjust pointers to align with the encrypted code section
- iterate through the *XOR* function until the code is decrypted.

After attempting to unpack the original sample, it did not exhibit the initial decryptor behaviour anymore, perhaps as a result of the unpacking process possibly damaging this capability. The infected test program did not use an initial decryptor. This can be confirmed by checking the differences in the first portion of the program up until the virus body, before and after infection. If the first portions of the program have been altered, then it is an indication that the initial decryptor was present and an EPO jump is most likely being used in the original program.

7.3.6 Analysis of the stub

The stub contains three main sections.

- A section that restores the original entry point for the payload after the stub has been executed.
- A decryptor that unpacks the payload.
- The payload executor that will run the payload after it is unpacked.

It is the stub that contains anti-debug and anti-emulation tasks. There are calls to items such as *CPUIDs*, shown in figure 7.15, that suggest a check is being conducted to detect virtual sandbox environments to then sinkhole the execution of the decryption routines.

```
; START OF FUNCTION CHUNK FOR sub_407357

loc_40344B:
070 inc     al
070 push    ebx
074 nop
074 cpuid
074 jmp     loc_40335B
```

Figure 7.15: A call to the *CPUID* indicating steps to prevent running the sample in virtualised environments for debugging purposes.

The stub's code is scattered throughout the stub's executable sections, and even using the graph view in IDA Pro does not help simplify the view enough to observe a workflow that has been implemented by the stub. This behaviour further obfuscates the code from both manual and static analysis by antivirus vendors. These elements in the stub are mutated on infecting new files and therefore provide few static elements for which a signature can be developed. The static portions of the malware are within the encrypted payload, which means that they remain out of scope for building a signature at this stage.

Figure 7.16 shows IDA Pro's hex view and highlights in green the executable portion of the stub before the encrypted payload section, shown in blue. This view further strengthens the hypothesis that an initial decryptor detailed in options 3 and 4 of section 7.3.4 was not used during the infection of the test program, otherwise the entire malware stub section would be encrypted and shown in blue.

```
00403520  76 5E BE C6 90 8D 95 26  FE FF FF 3D 06 86 01 00  v^+¦..ò&¦··=.å..
00403530  B0 25 E9 C9 FC FF FF 8D  8E B2 29 84 F1 80 CA F7  ¦%T+n··.Å¦)ä±Ç-■
00403540  83 EA 6F 87 FA 8B 95 89  FF FF FF B5 E0 FC 8D 85  âOoç·ïòë···¦an.à
00403550  F8 01 00 00 28 FD 83 C1  11 8B F9 29 FF BE 24 2E  °...(²â-.ï·)·+$.
00403560  D4 22 8D 70 FC F7 D6 C3  31 FE FF FF F6 D1 80 D1  +".pn■++1¦··÷-Ç-
00403570  B3 4E 86 ED 8B FF E9 8A  00 00 00 E8 D7 3D 00 00  ¦NåFï·Tè...F+=..
00403580  B8 41 57 81 1B 01 B5 D1  40 00 00 E8 D5 FC FF FF  +AW...¦-@..F+n··
00403590  85 C0 E9 77 FF FF FF 8D  71 BC FC 0F 85 88 FC FF  à+Tw···.q+n.àên·
004035A0  FF 39 CA 83 E0 00 B1 9D  46 64 FF 35 34 00 00 00  ·9-âa.¦.Fd·54...
004035B0  8A C8 81 D9 C0 42 13 CA  8A E9 CD 2E E9 5F FC FF  è+.++B.-èT-.T_n·
004035C0  FF FC F8 87 C9 18 FD FE  C5 8F 02 21 E9 F6 C1 23  ·n°ç+.²¦+..!T÷-#
004035D0  86 ED 89 42 08 FE C9 19  F1 B9 03 25 A2 B0 89 42  åFëB.¦+.±¦.%ó¦ëB
004035E0  04 E9 AD 3D 00 00 48 31  C1 20 EA E8 67 3D 00 00  .T.=..H1--OFg=..
004035F0  FF D6 8A F1 85 C0 8B F8  8D 37 0F 84 4B FE FF FF  ·+è±à+ï°.7.äK¦··
00403600  E9 2E FC FF FF AF 64 88  BB 1C 4A C3 78 36 28 F1  T.n··»dê+.J+x6(±
00403610  E9 72 59 5B 7C F9 73 02  90 0B 38 45 FB C7 37 84  TrY[|·s...8Ev¦7ä
00403620  D7 58 9D B7 3E 7F 05 51  2D 0B AD 8A AE BA 9A 93  +X.+>..Q-..è«¦Üô
00403630  37 CF 16 25 08 1B DB CA  64 D5 BE 8E 83 FE 63 B1  7-.%..¦-d++Ïâ¦c¦
00403640  FB 36 73 E0 B9 BC 29 B0  A7 CC 66 F5 92 84 3E EC  v6sa¦+)¦º¦f)æä>8
00403650  FA BA 74 73 F8 37 67 E9  62 F1 9F 82 3B 58 98 9D  ·¦ts°7gTb±■é;Xÿ.
00403660  8C C5 B8 99 A7 A3 28 3E  39 E6 23 BD CE 35 AC EF  î++Ö°ú(>9µ#++5¼n
00403670  04 B3 E9 75 CA 21 EC 97  AE CE CF F1 F8 74 43 6B  .¦Tu-!8ù«+-±°tCk
00403680  98 EB 27 EE 9B 45 9A 5E  DF AA 71 6F 8F 7E F2 03  Ÿd'e¢EÜ^¯¬qo.~=.
00403690  83 F7 D8 4B 2F EF A2 15  18 44 E0 E9 3E 64 62 E6  â■+K/nó..DaT>dbµ
004036A0  F6 7C 7A 97 19 04 76 AE  32 ED FC 18 61 B9 99 0A  ÷|zù..v«2fn.a¦Ö.
004036B0  BE A9 27 8C 72 C5 01 33  1B 47 77 DB 12 49 6D D7  +¬'Îr+.3.Gw¦.Im+
004036C0  3B 1E B5 90 07 83 1E B5  B3 C2 DD 86 6A B7 C1 4D  ;.¦_.â.¦¦-¦åj+-M
004036D0  D9 69 34 DF 8D 0C B2 D7  96 75 89 65 CB 75 D6 EF  +i4¯..¦+ûuë-u+n
004036E0  FF E8 61 51 7C 16 EC 57  55 8D 76 A4 7D 8D 79 BE  ·FaQ|.8WU.vñ}.y+
```

Figure 7.16: The encrypted payload is shown in blue and the malware stub is shown in green.

7.3.7 Polymorphic functions in the stub

The stub implemented by *Virut.ce* has six main stages that can vary between new infections and can help create enough possible combinations to classify the malware sample as polymorphic [12, section 6, para 6].

1. Call an address with a small increment.
2. Store the original contents of the registers.
3. Optionally add an address to point to *kernel32.dll* in the *EBX* register.
4. Calculate a pointer to the address for the call detailed in point 1.
5. Perform an arithmetic operation on the output of point 4.
6. Finally, replace the *RETN* instruction with a *CALL* instruction to decrypt the main payload.

At re-infection, the malware replaces instructions in the above logical processes to make them either more or less complex. The *Virut.ce* strain used in this study was documented by Kaspersky Labs [12] as having multiple ways of implementing each process, and in turn, exponentially increasing the number of possible combinations that the malware can mutate to when infecting new files.

Each of the first five points listed in the main stages of infection above implement similar exchanges to their instructions, resulting in the new sample's code differing from the last but still accomplishing the same end results.

Point 3 has multiple options to choose from that can conduct the task of adding an address to the *EBX* register. Below are some examples of how this can be accomplished, and which have been taken from the Kaspersky Labs white paper [12, section 6, para 11].

- *MOV EBX, [ApiFunc]/MOV EBX, [ESP + 24h];*
- *PUSH [ApiFunc]/[ESP + 24h]; POP EBX;*
- *SUB ESP, xxh; PUSH [ESP + 24h + xx]; POP EBX;*
- *LEA EBX, [ESP + xxh]; MOV EBX, [EBX + 24h – xx];*
- *ADD ESP, 28h; XCHG [ESP – 4], EBX;*

Point 5 is responsible for decrypting the main payload, and as such, has less possible permutations to accomplish this task and remain undetected. The main decryption of the payload is hidden from manual analysis because the stub uses a *RETN* instruction that points to a random address. The *RETN* instruction is changed to a *CALL* to the decryptor before its execution, and an analyst would need to follow a sample's workflow up until this point to then observe the decryption routine used by the stub. The decryption algorithms themselves vary, as does the key, which helps to diversify the malware and harden it against detection.

The section titled "Decrypting the Main Body" of the Kaspersky Labs breakdown document [12, section 7] of *Virut.ce* provides a subset of the possible decryption algorithms that are selected by this strain on infecting a new file.

In addition to the obfuscation techniques listed so far, the malware sample consists mostly of junk code to try and thwart manual interpretation of the code using simple tooling and help scatter the code through the sample to prevent a static signature being derived.

7.3.8 Obtaining the unencrypted payload

The techniques identified by a static analysis of the infected test program's stub do not elude to what the payload does. However, these techniques could provide an organisation with key information to build a signature of their own to detect and block the strain. A polymorphic virus can spread rapidly and evade antivirus software for weeks, months or even years. While antivirus

vendors work on solutions to detect the threat, it is vital for the analyst to develop their own defences. Knowing how the malware spreads and evades detection can help an analyst to develop a signature of their own to stem the spread of an outbreak within their organisation.

Two methods were successful in obtaining the payload from the sample whilst it was executed in the lab. Appendix 12.2 provides a brief description and links to videos demonstrating the use of *Ollydbg* to obtain the payload.

The recorded demonstration of how to obtain an unpacked payload using Ollydbg is only a single method that was trialled when carrying out this study. Another method was to pause the virtual machine and dump out the running process's memory from a snapshot of the device. Both methods are perfect for quickly obtaining the unpacked payload for examination; however, pausing the machine and obtaining its memory from the host environment was the preferred choice for this case study. During a major incident, an analyst needs a copy of the unpacked payload as quickly as possible. As some samples can have advanced anti-debugging capabilities, obtaining the payload using Ollydbg would fail because the software requires the program to attach to the active malware sample as it is executing. Dumping out the memory from the lab that is outside of the virtual machine means that the malware does not detect the activity and therefore it is unable to take evasive action to prevent its unencrypted payload from being obtained. A tool called *Volatility* (http://www.volatilityfoundation.org) was used to extract just the process's memory to an executable. This was recorded and is available via the direct link at: https://youtu.be/1uk2eqpgmn8. Unfortunately, this process means that it loses the program's imports. As a result, it can no longer be executed. If a sample uses anti-debugging techniques to slow down an analyst, then the use of Volatility can help to "level the playing field". In a major incident, there is often no requirement to be able to execute the sample again after its payload has been extracted; instead, it is enough to obtain the payload and ascertain any risk it poses to the organisation.

Both methods of either connecting to a running process from within the virtual machine or interrogating the memory of the device from the host environment layer succeed in obtaining the unencrypted payload from the sample. This was confirmed by using IDA Pro to search for strings in the extracted sample and identifying the IRC details that were observed as being in use when executing the malware sample in section 7.3.1. The IRC details differ from the original address identified through the earlier packet capture. However, this is not an indication of the full payload's capabilities as some tasks in the malware's code may have been skipped, and other details that can be rotated, such as IRC server details, may not yet be exposed. Using a different IRC address when infecting new files means that shutting down one command-and-control server that the sample uses would not render the malware useless on new infections. Figure 7.17 shows the additional IRC domain used by the malware. Using a multitude of possible command-and-control addresses would mean that a malicious payload has a longer lifespan in the wild.

.data:00403000	0000000F	C	Goodbye world!
.data:0040300F	00000009	C	PROGRAM!
.data:00404919	00000017	C	\\Device\\PhysicalMemory
.data:00404B70	0000001D	C	%s:*:enabled:@shell32.dll,-1
.data:00404B8D	00000075	C	SYSTEM\\CurrentControlSet\\Services\\SharedAccess\\Parameters\\FirewallPolicy\\StandardP...
.data:00404C02	00000033	C	SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Explorer
.data:00404C35	0000000B	C	UpdateHost
.data:00404C54	00000014	C	proxim.ircgalaxy.pl
.data:00404C88	00000016	C	NICK jmjuezpw\nUSER j<

Figure 7.17: Additional domain used for IRC connections.

This case study resulted in the unencrypted payload being obtained from the sample by infecting a test program. This facilitated the separation of other malware that was also included in the downloaded sample, leaving just the polymorphic malware. The infected test program also provided

a known set of instructions that could be ignored when isolating the sample. The infected test program was executed and the unencrypted payload was obtained using a piece of software called Volatility. The methods used in this case study could therefore be utilised in a major incident to quickly obtain a malware sample's unencrypted payload to work out any threats that it may have posed to an organisation if a corporate device had been infected. Practical demonstrations of the techniques used during this case study are available via the YouTube links cited throughout this document.

8 Case study two – compilation and analysis of metamorphic malware

8.1 Background

The previous case study reverse engineered the techniques used by a polymorphic malware sample to obfuscate its payload and focused predominantly on dissecting the sample. It initially involved carrying out static analysis to confirm that the malware strain in scope was in the sample obtained for the investigation. This was a type of black-box testing that slowly provided clues as to what techniques were being used. What made this type of investigation possible was that the unpacked payload never changed between infections. Therefore, it was obvious when all the layers of protection were finally removed, and what the resulting payload should look like between new infections.

The obfuscation techniques detailed in section 4.4 describe metamorphic malware as not requiring encryption to secure a payload. As such, a broader range of metamorphic malware samples in the wild use a more diverse range of mutation techniques than polymorphic malware samples use to mutate their stub.

Metamorphic malware does not need to use encryption or packing to obfuscate the tasks carried out by its payload, and so the forensic techniques used in the previous case study will not work when trying to discover and document the techniques used by the metamorphic sample used in this current case study.

As a result, case study two approaches the examination of metamorphic malware from an alternative angle. It aims to provide an overview, as well as examples of the techniques used by the sample writer to build the first generation of the malware. An important skill to develop as a security analyst is to put oneself into the mindset of the malware writer. By understanding more about how the writer thinks, it becomes far easier to predict what the malware is doing, and ultimately speeding up the analysis of a sample.

Further generations of a sample formed by infecting new files can dramatically change a sample's code and can cause it to bloat in size beyond a manageable level for an analyst to examine. Ideally, an analyst needs to obtain a sample early on in the infection cycle before the program bloats in size. This way, techniques can be discovered easier and signatures can be developed more efficiently.

The sample used for this study is *Lexotan32*. Released in 2002, it was written by a malware writer called Vecna who has an extensive portfolio in writing malware samples and tools. This sample was selected as it has already been heavily commented on as being an elaborately implemented molecular-based malware sample. It demonstrates the wealth of experience gained by a malware writer who has evolved their techniques over time and has taken their inspiration from other aspects of science [11, section 1]. A particular advantage in examining this sample was that Vecna had released the source code and its first-generation compiled sample on a website called Virus Exchange (http://vxheaven.org/). Therefore, the sample was easy to obtain and it was in a state ready for analysis, compilation and, ultimately, further investigation.

Vecna's previous work and collaboration with other malware writers has been documented in newsletters from the hackers group called 29A (http://vxheaven.org/vx.php?fid=1388). These newsletters provided a source for examining the evolution of the techniques and tools used by the malware writer. When looking at all of the samples and tools available in the 29A newsletters, it is fair to say that the malware writers who formed part of 29A did not just write quick re-usable scripts, but instead they were creating their own art.

8.2 Scope

As mentioned in the background of this case study, decompiling a sample obtained from malware repositories will most likely be of a generation that is too old to provide a suitable and efficient examination of the malware. Therefore, this study will initially use the source code for *Lexotan32* to examine the techniques used to build it. Using this methodology, a security analyst could utilise the first generation of a malware sample to identify the mutation techniques it uses to spread, and in doing so, the analyst could build a signature capable of encompassing a malware developer's style of writing malicious code.

One of the reasons why this particular malware strain was selected was attributable to the fact that other material had been published online by the writer. This array of material could therefore be used to gain more of an insight into the techniques that were preferred by the writer and to profile how their skills have evolved over time. The purpose of this investigation is to highlight the importance of understanding how to think like the malware writer when trying to understand the techniques used by a sample.

Most malware samples will not have source code available to examine their obfuscation techniques. This document has been designed to aid both analysts and systems administrators; the first case study supported analysts by looking at malware techniques from a compiled state, and this case study aims to support analysts from the other end of the spectrum. The sample will be analysed from its pre-compiled state to help demonstrate metamorphic techniques as well as to appreciate the high level of understanding and the Assembly programming skills that malware writers can have.

As this malware sample is well documented elsewhere and comes complete with its source code, advanced debugging techniques will not be used here and will instead be utilised in the next case study when examining a blind sample that is freshly protected by online platforms.

8.3 Compilation of Lexotan32

The developer Vecna coded the malware directly in Assembly, and the main file containing the metamorphic engine is many lines long, even when excluding the code inserted as part of the pre-processor. Although it is possible to map it manually, an analyst may not have time for this step in the event of a major incident. Therefore, it was decided that it would be more efficient to compile the source code for the sample and then use a hex editor to observe the first generation copy of the sample that is run.

The code itself is well structured and uses macros written in Assembly. It also utilises *C* libraries, and it almost resembles a higher-level language. The techniques used, such as macros, clearly demonstrate the developer's understanding and experience. Earlier viruses that have been written by the author of this malware demonstrate some of these skills and can more or less be used as a curriculum vitae of the author's skill development. As an example, the *w32.Evol* virus uses many of the metamorphic examples used in *Lexotan32*. The malware itself suffered from bloating over time and it looks like this issue was then fixed in *Lexotan32*.

Lexotan32 itself was therefore the product of previous development work by the malware author. The *Lexotan32* strain was released around 2002 and a previous early strain called *Lexotan* (referred to as *the legacy sample* in this paper) was already being used back in 1998. It is important not to let

the age of this malware provide a false perception that its techniques are obsolete, as both the techniques and concepts used by this sample are still relevant today.

Not only was the source code for the legacy sample available to download, it was also easy to compile. The sample came with its own *Make* file and it therefore only required *Borland Turbo Assembler* version 2, released in 1990, to be downloaded before it could be compiled. The source code itself is available on this project's GitHub repository (https://github.com/devopscrazy/Lexotan) and it demonstrates the malware writer's early use of techniques, including macros. The *Make* file for the legacy sample relies on the code being compiled using old tools. As the sample was developed around the late 1990s, the tools required to compile them are from the early to mid-1990s. Extensive use of Borland Turbo Assembler tools provided an indication of the developer's preferred tool base and what they have learned to develop with. At this point, it is apparent that the investigation is already starting to enter the mindset of the developer.

Finding legacy copies of compilers can be challenging, and in some cases, it is easier to identify what the original tools used to build a sample did before finding a newer tool that can accomplish the same task. The website http://www.winworldpc.com is a good source for finding reliable discontinued tools to compile malware samples. To build the legacy sample, multiple versions of Borland Compiler needed to be installed. This was good practice for proceeding further into the mind of the developer and to get a feel for the tools that they may have used. To compile *Lexotan*, it needed version 5 of Borland Turbo Assembler followed by version 2 to carry out the linking process. Figure 8.1 shows the result of compiling the legacy sample.

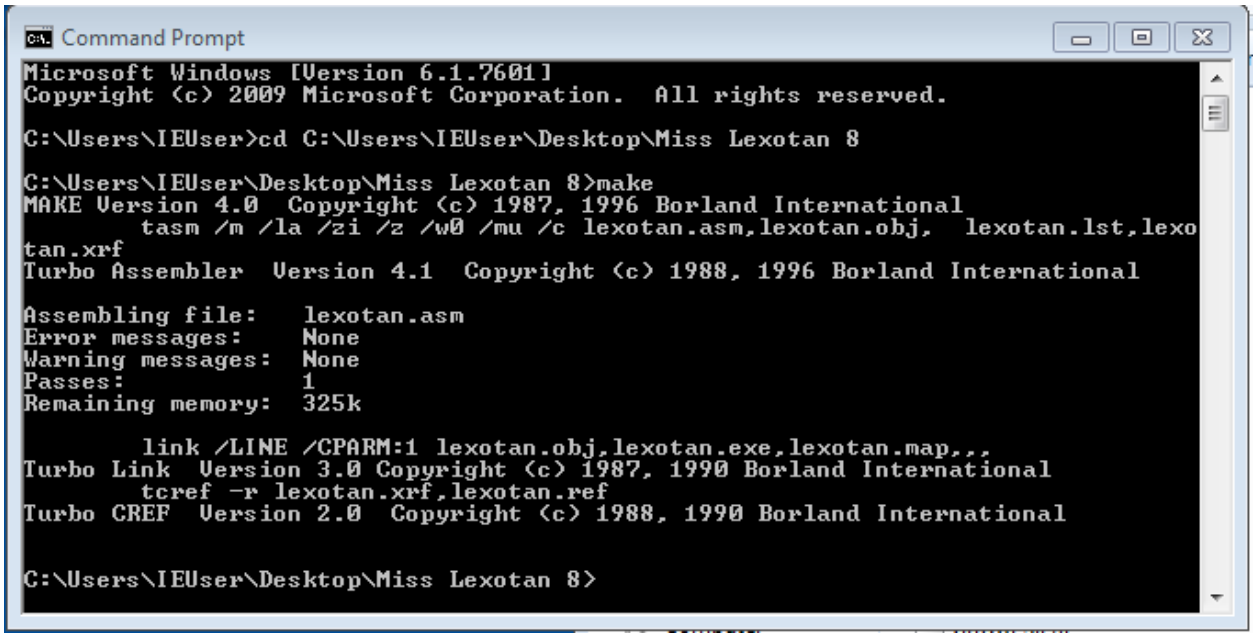


Figure 8.1: Result of compiling *Lexotan* using Borland Turbo Assembler version 5/version 2.

Lexotan32 takes a large leap in both the techniques used in the code and the complexity of the compilation required to build the malware sample. The code for *Lexotan32* also comes with a Make file, but the tools used to compile and link the sample, along with some of its required libraries, are missing from the source code. The sample of *Lexotan32* and the legacy version of *Lexotan* were both obtained from one of the newsletters from the group of hackers called 29A (https://vxheaven.org/29a/). Their newsletters contain samples of their work, source code and tools, and as mentioned earlier, they were useful in understanding a little more about the developer of *Lexotan32*.

Although all the 32bit tools required to compile *Lexotan32* are present in Borland Turbo Assembler version 5, there are missing components, such as the pre-processor, required to combine and flatten the multitude of files that comprise the malware's source code and external libraries. Many macros are sourced and then flattened during the pre-processing stage so that the compiler and linker have everything they need to build the portable executable file. The developer appeared to use a pre-processor that they referred to as *PrePro*. Sourcing this pre-processor has proven difficult and various searches provided no viable copy. It was therefore concluded that this pre-processor was built by the malware developer themselves, and an alternative pre-processor would be required instead. Entering further into the mindset of the developer, it was worth checking whether the pre-processor that was used was part of a Borland package. Every version of Borland software that could be obtained was therefore installed and then checked for appropriate pre-processors. As the pre-processor is used to not just flatten the code base and macros for compilation but also to pull in code from any C libraries used by the developer, this allows the use of high-level programming languages whilst keeping the main logic of the malware in Assembly. By downloading multiple versions of Borland C and C++ compilers, it was found that the pre-processor that came with version 4.52 may be compatible with the Make file's commands, but was unsuccessful at producing a viable artefact for further compilation. Compilation did not look promising and was further compounded by other libraries missing from the source code.

Necessary source files such as decoders and encoders were not present in the code. Due to the limited scope of this investigation, it was not possible to try and track down the missing files in this copy of the sample's code. The missing libraries were not found in either of the available resources from the malware developer or in the 29A groups newsletters. The result was a compilation error, as shown in figure 8.2.

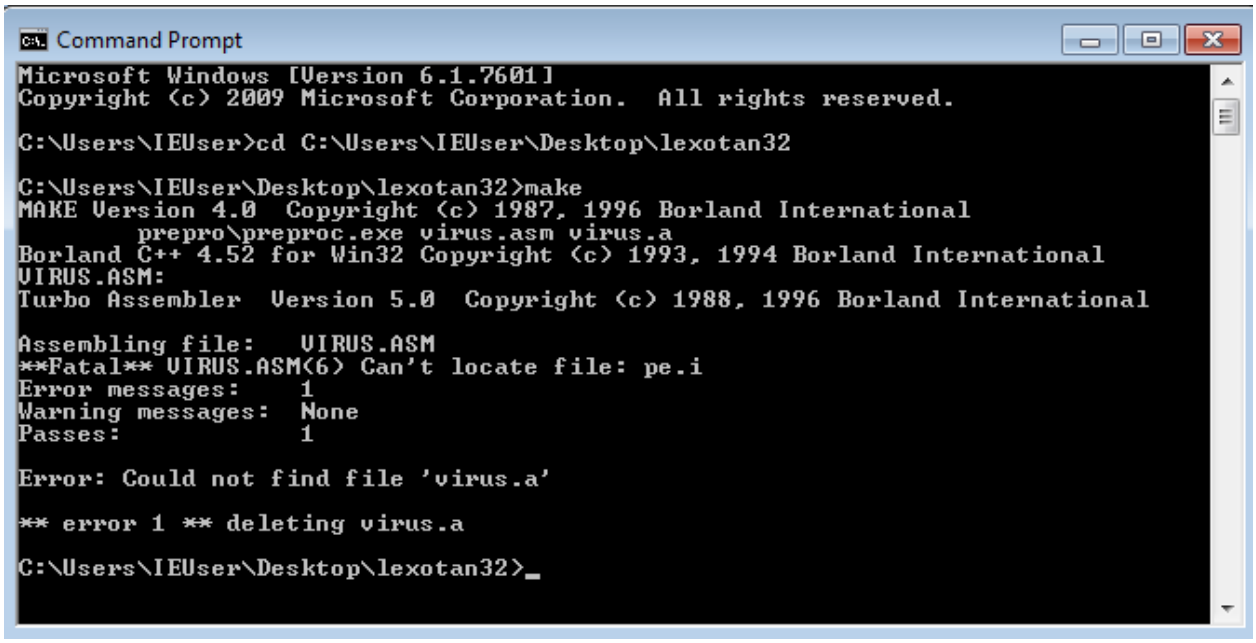


Figure 8.2: Error produced when trying to compile the original source code obtained for *Lexotan32*.

During earlier research, it was noted that the developer was part of a group of malware writers that published their findings and malware samples in a newsletter, and also conveniently published the tools that they used to build their samples. This investigation started with just profiling Vecna, however, it ended up researching the entire group's tools and documents released in their newsletters. The writer of *Lexotan32* may have utilised libraries developed by other members of 29A, and therefore the scope of the research rightly extended to the literature and tools released by the entire group.

The second newsletter published by the group included a tool called *pewrsec.exe* that was required by part of the build process. This allowed write access to the code for the first generation of the malware. Also, some missing *decoder* and *encoder* library files were obtained from the source code of other malware samples published by Vecna. It was interesting to be able to build a timeline of when the artefacts and tools used to build *Lexotan32* were first released.

Although compiling a malware sample requires a considerable amount of time and resourcefulness, it is sometimes worth the effort to obtain a first-generation copy of a metamorphic malware sample to speed up the investigation by an analyst and to more efficiently use their time to develop a signature for detection.

At this point of the investigation, a dead end had been reached with the compilation and continuing any further with the copy of the code base that had been obtained was not believed to be possible. However, it is important to understand that there is a community of talented analysts that are more than happy to help where possible, and that an analyst is never on their own in their investigations.

Interestingly, a paper by Orr [11] demonstrated some of the techniques used by the sample and claimed to have compiled *Lexotan32*, and so the author must have been able to piece together the missing aspects of the malware sample's code. Upon contacting Orr, they pointed out that the code released by the 29A newsletter was incomplete and that they had managed to obtain the full source code from Vecna's own website. Unfortunately, the website was down and no copy of the code was available from any website archiving sites such as *archive.org*. Orr had a copy of the code and kindly provided a link to the complete source code. It had all the required resources, such as libraries and pre-processors, and therefore compiled straight away in Borland Turbo Assembler version 5. The code is now also hosted on this project's supporting GitHub account at: https://github.com/devopscrazy/Lexotan32. After a lengthy investigation, a compiled first-generation copy of the malware has now been built and is ready for analysis. The tooling required to build a metamorphic sample in Assembly has already been explained earlier in this section. A description of the source code along with its compilation has been recorded and the video can be accessed directly at: https://youtu.be/RIxBzd64sfE.

8.4 Analysis of Lexotan32

The compilation of *Lexotan32* is complex, and in this case study it would not have been possible without the full code base provided by Orr. Both the malware samples obtained from VirusShare[2] and a first-generation copy[3] provided by building the source code are enough to provide a background of the types of obfuscation used by the malware. The vintage copy of *Lexotan* that also compiled successfully was useful when trying to understand some techniques used by the newer strain. As such, the older strain of *Lexotan* was also reflected upon as part of this analysis.

IDA Pro is good at cleaning up jumps and visualising workflows to make Assembly code easier to understand; however, even this sample is too bloated to make sense out of the jumps and render useful pseudo code for easier examination of the instruction logic. Supporting documentation by Orr [11, sections 3-8] was therefore used to identify the main steps used by the malware. These steps are:

- re-generation
- mutation

---

2 https://github.com/devopscrazy/MalwareAnalysis/tree/master/lexotan32/virusshare
(SHA-256 hash = c366f7ab4a90626a0ddf489524d22ffa299c2a1c174bdadd59151c795a4fc288, Samples' password = "infectedIYM011")
For more information refer to appendix 12.5.2

3 https://github.com/devopscrazy/MalwareAnalysis/tree/master/lexotan32/compiled-first-generation
(SHA-256 hash = e9ce8c89049936b42bb56c5c0e85dff8ad5b819292f8b7b4cfd21dc656e21505, Samples' password = "infectedIYM011")
For more information refer to appendix 12.5.2

- permutation
- garbage code creation
- relocation
- new genotype creation.

These main steps, shown in figure 8.3, enable the comparison and interpretation of memory dumps possible. Identifying these steps manually without extensive experience in reverse engineering samples would be extremely difficult and time-consuming, and would not have been possible within the limited time provided for this case study.

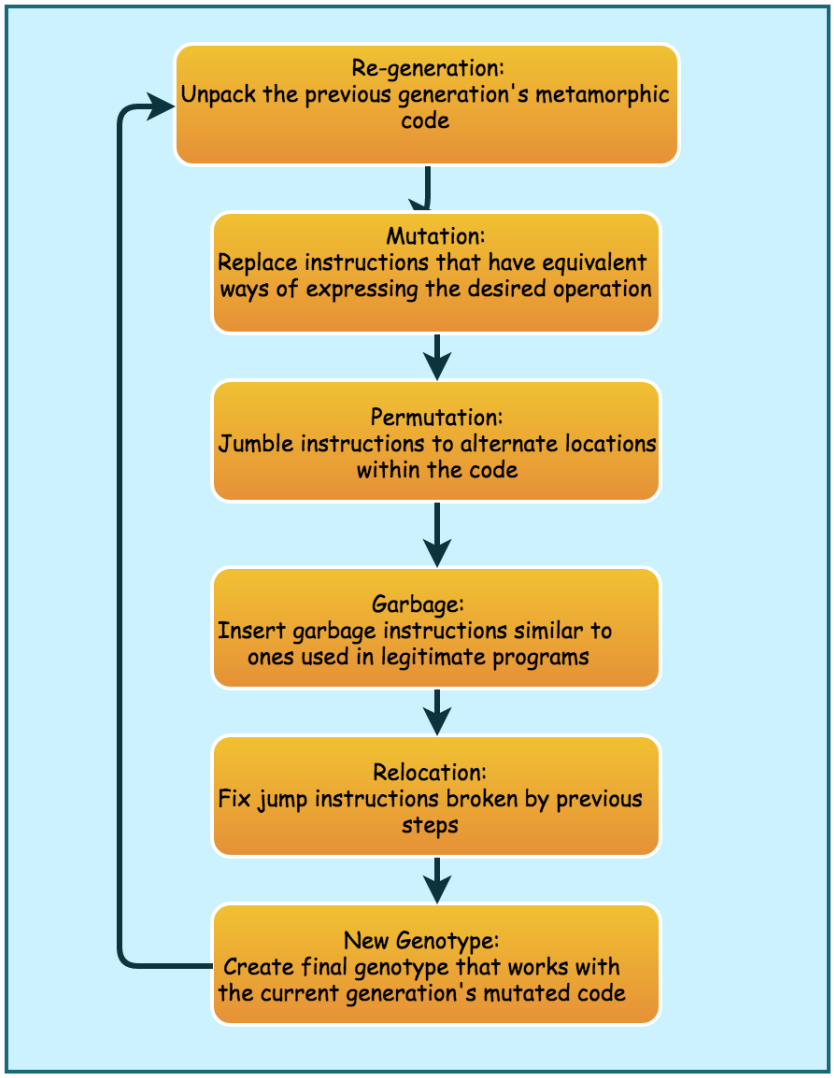


Figure 8.3: The main steps implemented by *Lexotan32*.

The original code creates a first-generation copy of the malware. This consists of a mutated set of code that, between the jumbled commands, contains the potential for the clean code. The other part of the malware is described in the code as a *genotype*. In genetic terms, a genotype contains sets of genes - some that are active and some that are recessive in nature. In terms of this malware sample, the genotype is the key to unlocking the cleaned code from the mutated code.

The malware sample uses the above-mentioned series of logical steps to unlock the clean code using rules specified by the genotype. These rules are run against the mutated code to extract the clean code.

The malware is then able to create the next generation of the virus by using the clean code and a newly generated genotype. Figure 8.4 shows that the clean code and the genotype are combined to create new mutated code that is different to the last.

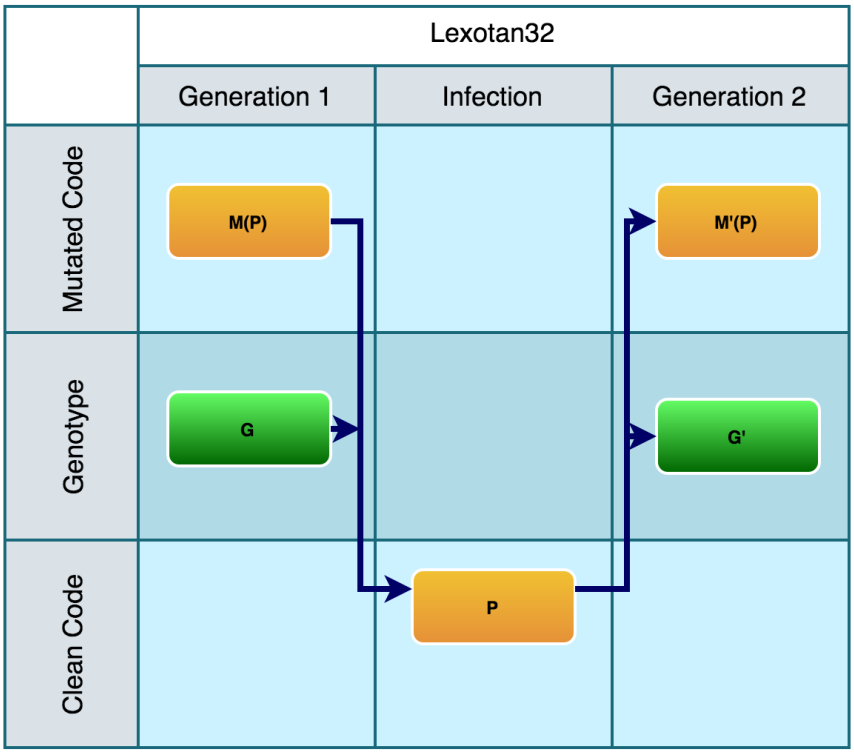


Figure 8.4: Creation of the next generation using a genotype.

The processes undertaken by the sample are like biological processes found in nature. From a very high level, the mutated code is equivalent to a gene and the part detailed as the genotype can be thought of as another gene; both elements together could be described as *DNA* (Deoxyribose Nucleic Acid). DNA uses other proteins, such as *RNA* (Ribose Nucleic Acid), to deliver instructions to cells. In this particular case, the RNA is the clean code.

When creating the next generation, the new mutated code and the new genotype in the next generation is similar to the re-splicing of DNA during biological reproduction. Although different to the previous generation, the new DNA is still capable of making the same clean code. When comparing this process to the one found in nature, it is similar to the same phenotype resulting in a certain eye colour. In the case of our metamorphic malware sample, it makes no difference what eye colour is given so long as the result has two eyes.

Not all the genes in DNA are used; "junk" genes are found in DNA and although they are dormant, they remain present due to millions of years of evolution. Similar to DNA, a large portion of a metamorphic malware sample contains mutated code that is not used, and is, to all intents and purposes, junk code. The clean code instructions have been swapped, scrambled and compressed. Therefore, it almost seems like the malware developer has used some processes that are commonly found in nature to help protect the malware sample's code, in similar ways to how our DNA tries to protect itself.

The *w32.Evol* virus was an early example of some of the metamorphic techniques used by the malware developer, demonstrating the insertion of junk instructions and the replacement of instructions with equivalent ones. However, this virus bloated over time and it consequently has a performance issue as well as a problem with resource as the generations continue to progress.

Vecna has further developed *Lexotan32* to work around this problem of bloating. Malware samples by other developers have implemented algorithms to either look-up the virus binary commands and store their location and lengths or to shrink the code. *Lexotan32* does not need to rely on these types of techniques because it implements a more genetic form of de-permutation instead.

This genetic type of de-permutation algorithm uses a specialised table, referred to as the genotype in figure 8.4, shown earlier. The entries in the genotype are used to extract the clean code from the metamorphic code.

It should be recognised that when looking at a piece of metamorphic code, it is easy to become lost down a rabbit-hole of jumps and random instructions in the code. Discovering why the code is jumping or what it is trying to accomplish is almost impossible without first finding the genotype to interpret the code. The jumps could be the result of a relocation, a re-generation or a mutation engine. The random instructions that are being run could have been exchanged via permutation or by a garbage code engine. It can therefore be confusing and intimidating on first impression. At this stage of the analysis, it is pointless to try to map out the jumps or do any kind of static analysis to this level of granularity because when the metamorphic engine uses the genotype against the code, it should significantly change the jump orders and the code logic.

Instead, it is far easier to understand and visualise the behaviour observed in the following sections by running the sample and using a hex editor to watch the movement of instructions during execution.

Each of these steps detailed in figure 8.3 are outlined below.

8.4.1 Re-generation

The malware developer uses the genotype to form the metamorphic code which, to all intents and purposes, is treated as a phenotype. This is because the way the genotype is used provides a key to lock away the clean code inside the metamorphic code. However, the combination of these two unique entities using the metamorphic engine generates the same clean code.

As previously mentioned, the genotype consists of a table and each entry in the table contains pointers to locations in the metamorphic code that are relative to the last pointer in the table. The pointers are stored as signed *WORD* values within the table. The first entry in the genotype will point to the first instruction in the metamorphic code. The second entry in the genotype, when added to the first pointer, will provide the pointer to the second instruction in the metamorphic code, and so on.

The genotype looks to have additional data for each pointer in the table, and on first impression, this appears to be padding; however, when it is executed, the malware sample uses this data as a flag to later work out additional steps to be conducted. Referring to the analysis conducted by Orr, this flag can be in one of two states: *-2 (0xFFFE)* executes the *FREE4GARBLE* directive and *-1 (0xFFFF)* indicates the end of the genotype list [11, section 3].

When the *FREE4GARBLE* directive looks at registers that are active players in the malware's code, these entries are then tagged for intelligent garbage instruction generation at a later stage depending on the operational instruction that they are to be translated into, such as *XOR*, *ESP* or *VALUE* instructions.

8.4.2 Mutation

The mutation engine is one of the most interesting because it almost mimics random selection or Darwinian evolution of the virus logic, and it provides a strong degree of entropy to help prevent a signature being built based on the logic of the underlying instructions.

By this phase, the genotype has been used to locate the instructions and their lengths within the mutated code. These locations are then used to transform the instructions based on random factors.

The mutation engine has a list of possible operations and it therefore identifies a semantically identical operation to exchange. When performing a lookup operation for the next generation, an already mutated set of instructions will simply be replaced by another mutated set of operations that syntactically conduct the same results.

Some of these mutated operations will be larger in size than the instructions they are replacing. However, this does not cause a longer-term expansion of the codebase as seen in other metamorphic malware, such as *w32.Evol*. This is because any operation that causes growth will be replaced in the next generation of the malware with an operation that is potentially compressed in comparison.

Some generations of malware may, by chance, form the same transformed instructions, therefore providing the possibility of a signature detecting them. As a result, it is practically the same as providing a degree of random selection at this stage for thwarting antivirus software.

8.4.3 Permutation

This phase is used to move instructions within the sample to random locations within the sample's code. The processes that this engine performs, although relatively simple to explain, can be difficult to understand when observing it execute. This is because there is a sudden bulk insertion of *No Operation* (NOPs) instructions into a buffer. Instructions from the plain code base are then randomly inserted into different locations within the buffer, replacing any NOPs that are in the way. The operation flow is maintained by linking the sections together with jumps.

The garbage instruction engine that was referenced earlier (*FREE4GARBLE*) is then used to insert junk code where applicable. The NOPs provide writable slack space for the garbage code to be inserted without affecting the destinations for any existing jump operations that were put in place during the earlier stages.

At this stage, the excess NOPs are no longer needed and so the operations are written to another buffer and the excess NOPs are filtered out during the transfer.

Figure 8.5 illustrates the remaining reordered copy of the metamorphic code, complete with additional garbage operations, and it is demonstrated that the new jump operations still work.

8.4.4 Garbage code creation

In the above section, garbage instruction insertion is briefly mentioned. The generation of these garbage instructions is handled by the garbage engine, and the main aim of this engine is to provide a blend of random instructions that do not alter the overall flow of the code.

Garbage code is often selected from a pool of available commands that are known not to affect the established workflow of a program if they were to be executed. This results in common entries that can be used by antivirus vendors to detect a malware sample's presence using a heuristic detection engine. *Lexotan32* works around this issue. The *FREE4GARBLE* engine that is used to generate garbage instructions targets common instructions that are used in legitimate programs, therefore reducing the chance of detection.

The file *VIRUS.ASM* contains the main code for the *Lexotan32* sample and it includes a series of directives that can be exchanged for commonly used operations within flagged code. The example given by Orr in their paper [11, section 6] is the following:

Original instruction: *XOR ESP, 100001b (21h)*

Exchanged with: *FREE4GARBLE _EAX+_EBP*

Figure 8.5 shows the result of the permutation and garbage instruction stages. The instructions in red highlight the garbage instructions being inserted. Note that the output also has jumps (shown in green), demonstrating the permutation stage that is tasked with re-ordering the sample's code.

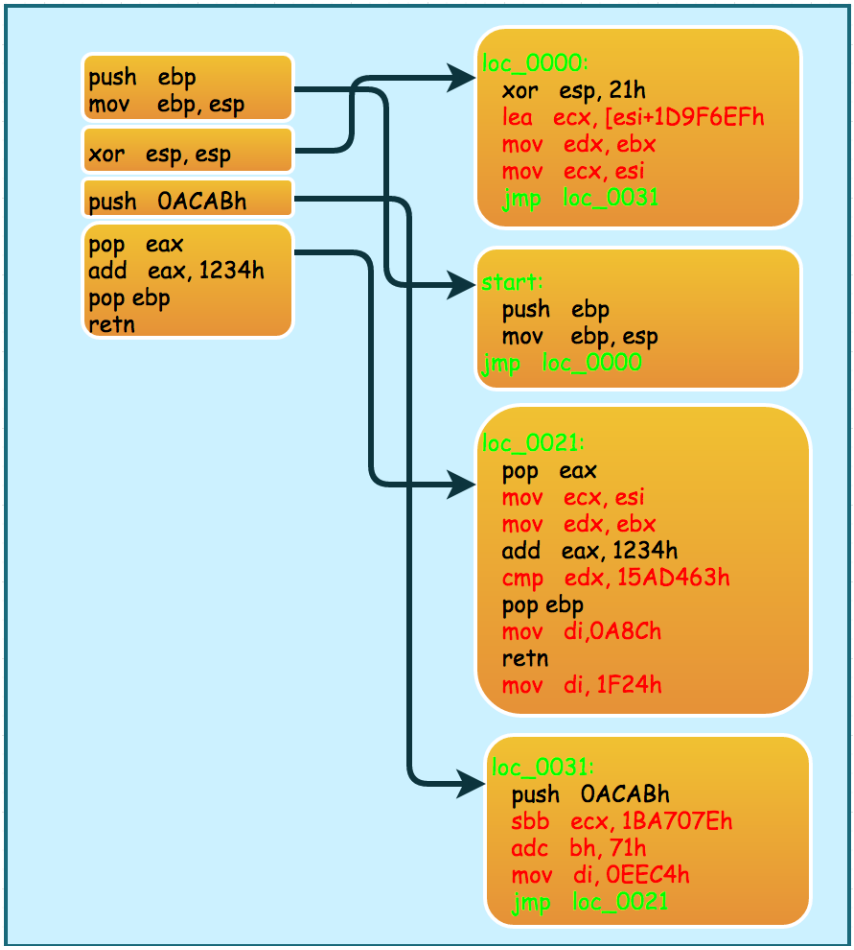


Figure 8.5: Permutation (green) and garbage (red) routines run by the metamorphic engine [11, section 6].

8.4.5 Relocations

This part of the malware engine is designed to fix all of the broken links between the newly created code sections. The creation of the new generation of the malware sample has moved code sections about, and at the same time, it has broken some of the jump operations. As the engine has moved operations around in various stages, it has kept track of this activity in a table. It is at this stage that the table is used to write those tracked changes to the mutated program and fix any broken links in the buffer. Jumps that were inserted as part of the permutation process were fixed at the end of that stage and have not been tracked for the relocation stage. The relocation stage has instead fixed jumps broken by other stages such as junk code insertions.

The table that has been used to track the changes contains the original location of the jump instructions as well as the new location relative to the original; each entry also includes the size of the section that was relocated. As a result, all this stage needs to do is iterate through the table and update the new locations.

8.4.6 New genotype creation

By this stage, all of the processes for mutating the clean code and producing the next generation of the sample are complete. The genotype has been built from all the tracked changes. If the new sample is executed, then the metamorphic engine will use the new genotype to convert the metamorphic code base back to clean code, and the process of building the next generation begins again.

Metamorphic malware is powerful; its engine is often capable of generating code with an exponential number of possible combinations. There does not need to be any encrypted components for antivirus software to build a signature on. Using heuristic detection will be very complicated due to the underlying instructions mutating and changing the APIs and operations that are used in each generation of the sample, and, in the end, using different combinations to perform the same semantical operations. Building a signature that encompasses all possible generations of a sample is a complex and daunting task for any analyst.

If an analyst is lucky, their organisation is only being forwarded a single generation of the metamorphic malware. Building a signature for one generation of the sample should be possible and this will enable an analyst to filter further floods of the infection at the perimeter of their organisation's network. However, if a device within their organisation has already been infected, then a new generation will have been built that their static signature will not detect. Alternatively, if the malicious party sends multiple generations of the same malware sample, then the analyst will need to build a signature that detects all the variants that are being received, hence metamorphic malware that has not been included in existing antivirus software databases will always present a challenge for organisations that are being targeted.

9 Case study three – mobile malware packed by online platforms

9.1 Background

People are starting to rely on resources provided via the internet more in their daily lives for tasks such as maintaining personal calendars, sending messages to contacts and looking up information online. Performing these common tasks can be slowed down by having to start up a machine, even for the shortest of tasks. Mobile devices are beginning to take over in popularity when conducting most tasks that require the internet, as these devices are often already powered on and available to the user [15, page 6]. Younger generations have grown up with mobile devices being integrated within their home and school life from a young age, and as such, it is common to find children choosing to predominantly use mobile devices instead of traditional personal computers (PCs).

This shift in popularity means that malware for mobile devices is becoming commonplace. Traditionally, Microsoft-based machines were one of the most popular in use and therefore the majority of malware was developed to target these machines. As the mobile market has grown, the popularity of Android has exceeded other market options and it has therefore attracted the attention of malware writers [16, page 9]. For this reason, it seems a logical progression to look at what is available to support polymorphic and metamorphic malware on mobile devices.

Android devices range from TVs, watches, mobile phones, tablets and more. The device's Operating System is based on Linux and so there is the potential for the same techniques that are used by traditional PC malware to be used when writing malicious payloads for Android-based devices. Android provides middleware between the specialised hardware used in devices, such as TV tuner cards, cameras and telephony hardware found in mobile phones. As such, Android devices are an attractive target for organised crime. They can be used as zombies in a botnet for attacks, or they can result in a financial loss for consumers and telecommunications providers by using an infected device to contact premium rate numbers to generate income for malicious parties [4, para 19].

Android executes *Dalvik* byte code, produced from Java code, and so most applications that are developed for Android are done so in *Java*. The main difference between a compiled Android application and a compiled Java application is that instead of multiple Java Class files being generated, the Dalvik code is compiled into a single *.dex* file. Although all applications run unprivileged on an Android device to provide a layer of protection to the Operating System, the applications themselves provide little assurance to the user installing them. It is not known if the individual or company developing the application is a reputable source. For an Android application to be installed on a device, it is necessary for it to have been signed by a set of keys that only the writer could possess, yet there is no central *Public Key Infrastructure* (PKI) used to provide a level of assurance regarding someone's identity. In other words, there is nothing to stop an individual developing an application and pretending to be someone else when signing it and submitting it to Android marketplaces [17, page 2].

A package's signature can be changed by either repackaging or disassembling and then reassembling the application; this procedure can be used to resubmit an application to the Android marketplaces using a different identity. Repackaging a sample modifies the identity of the person who has signed it. Disassembling the Dalvik code will provide source code that is not quite the same as the original. Hence, when reassembling the application again, it will produce a sample that is not identical to the original.

It therefore seems a suitable start to extend upon the concepts used in case study one and build an Android application that triggers antivirus software to alarm that it has detected malware. A third-party platform can then be used to pack the sample Android application using either polymorphic or metamorphic techniques to help it evade detection.

The first case study examined a payload that had already been built and packed using a polymorphic engine. Case study two focused on the techniques used by malware writers to build a metamorphic sample. A natural transition would be to look at how organised crime use Platforms as a Service (PaaS) to package up their payloads. For organised crime, these services mean that they can spend more time rotating their payloads and worrying less about developing packing engines and adjusting them to continually evade detection by antivirus software.

There are a multitude of legitimate PaaS providers for packing Android applications. This is because legitimate companies need to pack applications to protect their intellectual property from being easily decompiled and misused. Some of these PaaS providers also offer free trials or packing the first application without charge. This was a useful option for this study as selecting a viable PaaS provider was straightforward and did not cost anything. Some of the best providers seemed to be in the Far East and did not offer an English website. However, although challenging at times, Google translate can provide an output of the site in English that is good enough to establish how to use the product.

Most of the PaaS suppliers scan uploaded applications for virus and malware signatures to prevent scripted mass-produced viruses from going through the packer's engine. This was easily evaded as part of this investigation using very little effort.

9.2 Scope

It was initially intended to take the opportunity in this case study to unpack a malware sample that had been secured using standard packing software. However, thanks to readily available tools, this task is straightforward and does not take long. When it does become a challenge, it can simply be overcome by using the same techniques as detailed in previous case studies. As a result, analysing such packed samples would add little to this report.

Instead, the intention of this case study is to:

- create a sample Android application that triggers an alert in antivirus software
- identify a PaaS or software package to pack the sample
- use the PaaS to produce a packed sample that evades the aforementioned detection by antivirus software
- identify techniques used by the PaaS to secure the packed sample.

A video detailing how the sample was created and packed using the PaaS is available via the direct link at: https://youtu.be/sZtAFqfORzg.

9.3 Building a sample

Similar to the case study on polymorphic malware, a test application is required to run through the PaaS environments. Applications for Android devices are programmed in Java and packaged up in much the same way.

Appendix 12.3 provides information regarding the development software available for building an Android application and justifies the one used as part of this study.

For this sample, a test application was built that allowed commands to be passed to the Operating System; the results of these commands were returned to the application's user interface. This functionality alone was not enough to trigger as malware, but instead allowed for commands to be run that could be interpreted as malicious.

Rather than create a malicious application to then host on online code management sites such as GitHub, it was decided to use the *EICAR* pattern to trigger a false alert from antivirus systems. EICAR is a string that most antivirus vendors trigger on, and it is often used to test if antivirus solutions are functioning correctly. Rather than provide the string here and risk this document triggering antivirus solutions, the test string can be found on the EICAR site [18]. Line 35 of the main function of this case study's test program demonstrates how the EICAR string has been implemented: (https://github.com/devopscrazy/AndroidShell/blob/master/app/src/main/java/com/stentsure/androidshell/androidshell/MainActivity.java).

The test program was designed so that passing the command "malware" would trigger the EICAR string to be executed, thus triggering the antivirus solutions currently in use. The application was built and signed so that it could be executed legitimately on Android devices. The signed application was then submitted to VirusTotal and it triggered one vendor, as shown in figure 9.1, although strangely not all antivirus vendors detected the string being used.

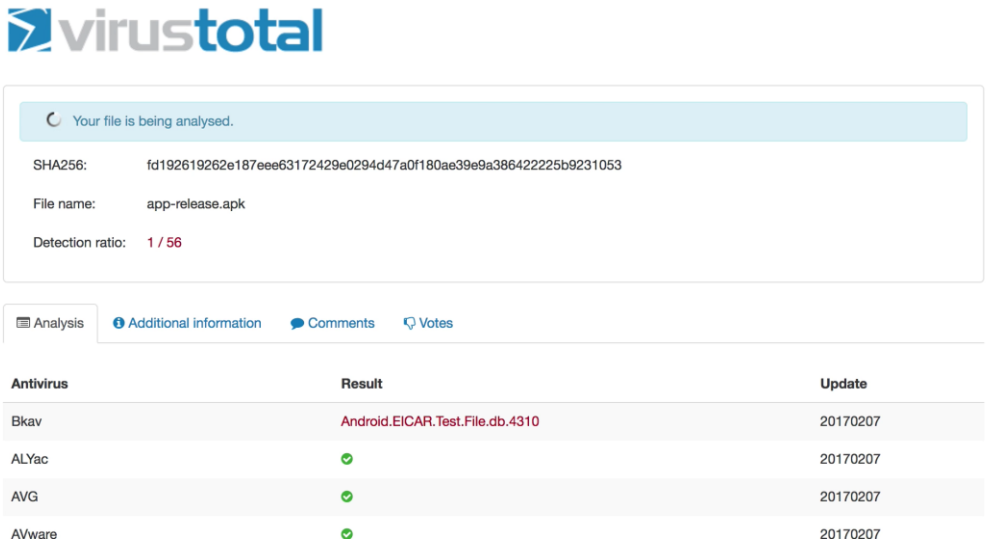


Figure 9.1: Bkav has detected the EICAR antivirus test signature in a sample Android application.

The application code developed for this case study can be located at the following GitHub link: https://github.com/devopscrazy/AndroidShell and can be loaded directly into Android Studio to then build and sign, free for personal use.

9.4 Packing the sample

Three off-the-shelf candidates were originally selected to conduct the packing of the sample; *ProGuard*, *APK Protect* and *Bangcle*. The main challenge was to select a candidate which created packed files that then constituted a polymorphic or metamorphic piece of malware.

The main thresholds for malware sample selection in each of the previous two case studies were that the sample could generate new signatures on infecting a new file and that it had the ability to mutate itself on execution to evade detection.

Android devices run applications in separated containers, and as such, the applications are isolated from key components of the Operating System and any resources outside of the container. Applications signed by a common key, possibly by the same developer, can interact with one another. However, an individual application does not have enough access to infect an unmodified factory-configured Android device outside of the container it is executed within. Therefore, creating mutated copies of itself on the device is not possible.

Malware on mobile devices cannot be classified as polymorphic or metamorphic. Although the original application can be packed and advanced techniques can be used to unpack it, either the stub (in the case of a polymorphic sample) or the metamorphic engine (in the case of metamorphic malware) will ultimately remain the same and be a viable constant for an antivirus vendor to develop a signature.

The candidates chosen for selection as part of this study are listed below.

- ProGuard – [19] free to download and runs locally for packing samples.
- APK Protect – [20] an online platform for protecting uploaded APK files.
- Bangcle – [21] another online platform, predominantly in Chinese. The Google translations of the page are enough to be able to understand how to register.

These candidates were selected due to their popularity and the ease of either obtaining their software or their sign-up process. Although these candidates were part of the original selection process, only one was used to conduct the packing for this sample and subsequent analysis. This was due to time constraints as well as the fact that only one of the candidates demonstrated suitable obfuscation techniques for this study.

9.4.1 ProGuard vs ALAN

Having a packer that was run from a host development machine would have been of major benefit to this study. Simply debugging the packing of the APK file and watching its obfuscation techniques would have been ideal; however, ProGuard was rejected as a viable candidate for this study as the obfuscation techniques that it uses do not classify it as oligamorphic, polymorphic or metamorphic. Research conducted as part of the Virus Bulletin released on July 2014 [22, section 2.1] detailed that ProGuard simply renamed paths as well as class names. As well as this, methods and their variables in the code were also renamed, and so building a signature would be as simple as looking for string patterns. Software is also freely available online for unpacking samples that have been transformed with either ProGuard or the commercial version of the software, called *DexGuard*.

Smali is a type of Assembler used by the *.dex* files that are inside the *.apk* file. Due to tight time constraints to carry out this case study, rather than considering the technical transformations being carried out on the Smali code, it was decided to look at open source projects that had implemented and provided code to understand the techniques faster.

A project called *ALAN*, [23] developed by the Department of Engineering at the University of Sannio, provided a great code base that demonstrated some of the types of Smali code transformations used by ProGuard.

Using the custom sample, already written in section 9.3, that had been successfully detected by antivirus vendors on VirusTotal, each of the transformations provided by ALAN were used one at a time to pack the sample, and between each transformation the sample was then re-submitted to VirusTotal. Most of the transformations masked the sample from being detected as malicious. The only transformation that failed to do so was *encoding*; this transformation caused more antivirus vendors to detect the payload as malicious. This can only be attributed to ALAN being used by genuine virus writers. Therefore, these types of transformations can render an automated tool ineffective if a signature has been developed by antivirus vendors to alert on the type of transformation being used instead of detecting a malicious sample that has used the automated tool to try and protect it.

After some testing, the end conclusion was that this tool did not conduct any transformation types that would classify as polymorphic or oligamorphic because the original payload was not being encrypted and then executed by a stub. The software did not constitute as metamorphic either as the transformations did not mutate the original sample using techniques such as operation substitution and generation mutation. Due to the lack of effective transformation techniques, original sections of the payload were still easily accessible and could potentially be used to form a signature for detection.

The transformations available through either the tool's user interface or by executing individual models are [23]:

- using APKTools to disassemble and reassemble a sample
- changing the name of the package
- renaming identifiers
- encoding data
- adding call indirections
- reordering code in the sample
- adding junk code to the existing code
- additionally, combining any of the above techniques together within a single sample.

The only operations above that would make a sample similar to metamorphic malware are the reordering of code instructions and the insertion of garbage code; however, these tasks are very simplistic compared to the tool ultimately selected for this case study. Neither ProGuard nor ALAN implemented debug or emulation mitigation and so forensic analysis is not deterred, unlike the tool that was selected to conduct the remainder of the case study.

9.4.2 APK Protect

Online platforms supporting APK Protection at the time of this study are unstable, and therefore access to these sites was not possible and its content had to be researched using online website archives instead. The report conducted as part of the Virus Bulletin paper from September 2014 [24, section 2.1] included a good overview of the protection provided by the platform. Further research

provided enough information to rule out investing time in finding alternative trusted sources for downloading APK Protect to then use for packing this case study's test sample.

The techniques implemented by the platform did not use anti-debugging or anti-virtualisation, and so to obtain the encrypted payload an analyst need only [22, section 2.3]:

- swap some bytes at the start and end of the sample
- decode the payload, as it is only base64 encoded
- run the result through an *XOR* operation with a key that is unique to the sample
- decrypt the result using a hard-coded key.

The key that is unique to the sample can be obtained using freely available tools found on the internet. One common way to analyse a sample protected by APK Protect is to use Android-unpacker [25]. Using this tool along with coded scripts allows an analyst to build their own tools for performing the above tasks.

One benefit to using APK Protect is that the contents of the *.apk* file are not bloated with extra files; only one additional file is added that could be used to detect that APK Protect has been used [24, table 1]. Ultimately though, this platform just implements obfuscation and junk code insertion, and as such, it is not a metamorphic packer.

APK Protect was still worthy of documenting in this case study as it provides more advanced protection than other tools documented so far, although reversing its protection is possible using automated tooling. However, packed samples only resemble an encrypted malware sample, and the stub used to decrypt the payload remains the same when packing a sample.

9.4.3 Bangcle

Bangcle was selected for packing the sample as part of this case study as it provides an exported sample that is particularly resistant to both static analysis via reverse engineering and dynamic analysis using a debugger.

Some of the techniques used by the platform to ensure that a resistant sample is created are the following [22, section 2.5].

- Functions that are exported from native libraries have obfuscated names. Therefore, a static analysis is slowed down when attempting to identify instructions from simple native libraries.
- Debuggers are fooled by hooking standard *libc* functions that are used in packages.
- Libraries within the sample are compiled with stack protection enabled, causing further difficulties for debuggers.
- The original sample is encrypted as a payload and only decrypted in memory at runtime. This means that any antivirus tools running on the device that only scan files written to disk are circumvented.

The platform's website is exclusively in Chinese and the Google translations of the page are a challenge, but workable. Despite the language barrier, this platform was actually the easiest to use. Once at the stage where the client side application can be exclusively used, then it is straightforward to use the platform. The client side tool uses English as its primary language and so it is relatively simple to navigate the interface.

The steps detailed in appendix 12.4 demonstrate how to create an account on the platform and secure the sample. They have also been recorded and are available at: https://youtu.be/sZtAFqfORzg.

9.5 Analysis of a packed sample

9.5.1 Basic static analysis

Often, the first stage of analysing a sample involves performing a static analysis, and when that stage is exhausted, a dynamic analysis of the sample is conducted. This section of the study follows this methodology during the investigation of the sample. It is easier when conducting static analysis of the packed Android sample to convert the code to Java. Although the interpretation of Dalvik code back to Java code is possible by decompressing the APK file and then decompiling the *.dex* file, this is a time-consuming phase. Decompiling the packed *.apk* file is demonstrated in the following video at: https://youtu.be/enKqmD_8VWw.

The decompilation was conducted using the below tools.

- *Apktool* – a reverse engineering tool.
- *Dex2jar* – reads Dalvik executable (*classes.dex*).
- *Java Decompiler* – used to quickly look at the contents of *.jar* files (http://jd.benow.ca/).

Alternatively, there are online services that will allow you to upload an *.apk* file and the platform will then perform the de-compilation tasks automatically. This can be a risky option and should only be used if the platform in question is trusted due to the source code that the platform returns potentially being modified to include malware that has been injected into the code by the platform. Recompiling infected code could result in an analyst's development environment becoming infected. Analysing a company's *.apk* file by uploading it to a platform could potentially mean distributing intellectual property owned by the business. None of the online de-compilers tested as part of this study could unpack the test sample that had been packed using Bangcle.

When viewing the contents of the new *.apk* file using Java Decompiler software, there were additional resources included, such as a */lib* directory. Using Dex2jar was simple (this process is also detailed in the aforementioned YouTube video) and is outlined below.

- Rename the APK file to *.zip* instead of *.apk* (making it easier to unzip).
- Unzip the renamed file and copy the *classes.dex* file into the same directory as Dex2jar.
- Run *d2j-dex2jar.bat classes.dex*.
- A *.jar* file will be created, called *classes-dex2jar.jar*.
- Then, use Java Decompiler to open the *.jar* file, as shown in figure 9.2.

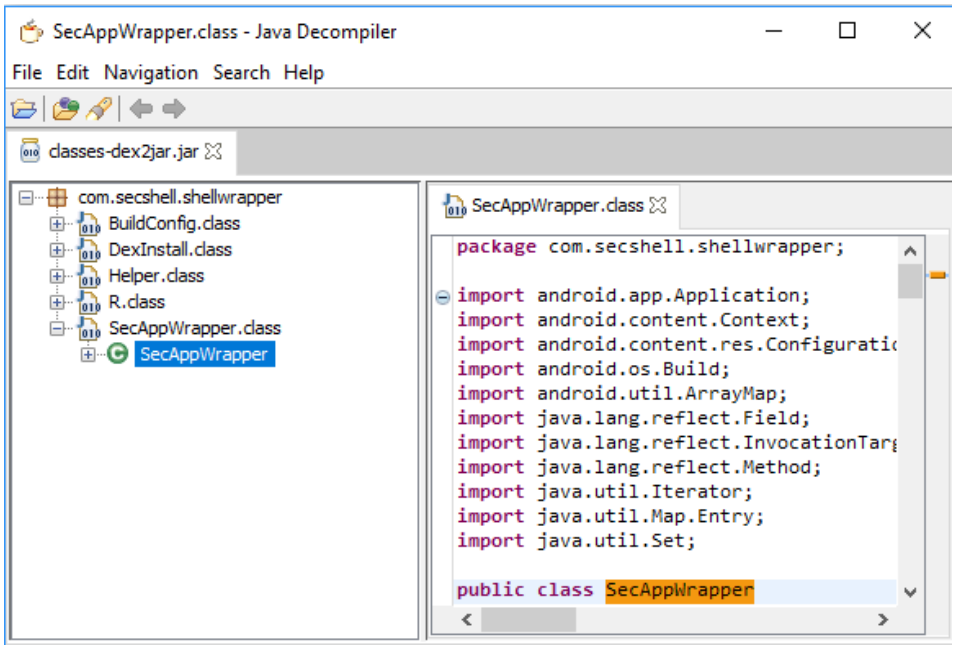


Figure 9.2: Contents of the decompiled jar file produced by decompiling the .dex file.

The main directory structure inside the *classes.dex* file starts with *com.secshell.shellwrapper*; searching for this structure on Google shows no results at all. As such, seeing this directory structure in a *.dex* file is a good indicator that either Bangcle has been used to pack the file or that a non-standard packer is in use.

When trying to reverse engineer the file, new class files were observed. Two of the files were of interest: *SecAppWrapper.class* and *helper.class*. The *SecAppWrapper.class* file appears to be the entry point to the code and the *helper.class* file has the only reference to the original test sample's code. The *SecAppWrapper.class* file uses the helper file to load the original sample into memory using an array map.

At this stage though, the original sample still has not been retrieved from the application packed with Bangcle. Reverse engineering the complete wrapper in a major incident would take too long. Therefore, other methods need to be considered to unpack the payload and extract it for further analysis.

As mentioned earlier in this case study, static analysis can reach a dead end where an analyst must weigh up how much time it will take to manually peel back the layers of protection used by a sample against simply executing the sample in a safe environment and dumping the un-obfuscated payload out of memory.

9.5.2 Anti-debugging

Instead of using the virtual lab used in previous case studies within this project, it would be more efficient to silo off a complete workstation and run Android Studio natively on the device. This would allow for better performance when running the virtual Android appliances. An analyst could also build their own Android devices for their virtual lab. However, Android Studio is designed to deploy applications directly to the virtual Android device. Running a virtual machine that utilises an *ARM* (https://www.arm.com/) based processor is very resource intensive, and as such, running this virtual machine natively on a host from Android Studio is a far more efficient use of resource and time.

Two methods were trialled for extracting the sample's payload directly from memory. The first method was to use a debugger installed on the Android virtual machine itself, dumping the contents of the process that contain the unpacked test sample. The second method used was to dump the entire memory of the Android virtual machine from the host development machine layer.

Two methods were chosen because malware can sometimes use techniques that prevent a debugger from attaching to a process to dump its memory contents. Therefore, it is essential that more than one method is used in case the first method fails or its extracted content is contaminated. If this first method fails, the second method of extracting the Android device's memory from the host devices layer can then be used; this method ensures that the sample is unaware of any attempts to extract the process memory. As detailed in the earlier case studies, this memory extraction does not always produce a complete payload. However, using both these techniques should produce enough of the original payload to allow for forensic analysis to be undertaken.

Loading the sample into IDA Pro and then trying to remotely debug the sample that is running on an Android virtual machine fails. IDA Pro connects to the remote debugger without any problem, however, the sample packed with Bangcle has anti-debugger protection. On further investigation, the debugger was refused a connection to the running processes as the Bangcle packed sample already had the maximum number of possible *ptrace* processes connected to it. On Android, there is a limit whereby a process can only be examined once [24, section 2.4, para 4]. Therefore, the current *ptrace* processes prevent IDA Pro from attaching a debugger to the processes in question.

Running the packed sample on an x86_64 version of the Android virtual machine causes the application to crash and exit. This is due to anti-debug code that checks for ARM based architectures being used on the virtual machine and causing unexpected behaviour when x86_64 architectures are detected. This check is most likely in place to slow down an analyst's investigation of a sample or to prevent automated tools from conducting any sandbox tests. Instead, further investigation was carried out on slower-performing ARM and ARM64 based Android virtual machines.

9.5.3 Dumping memory using Android-unpacker

It was clear that another approach was needed that did not require IDA Pro to attach to the process for analysis and only uses a debugger instead, installed locally on the Android device to dump relevant portions of memory once the original payload is located. The tool called Android-unpacker was designed with unpacking Bangcle samples in mind and can be found on GitHub for download [25]. The documentation surrounding the script is very good. The basic concept is that the sample of interest is deployed using the *Android debugging bridge* (ADB) package that comes with Android Studio. The ADB tool deploys *.apk* files to any Android emulators that are in a running state. The tool Android-unpacker utilises the ADB package to access the Android virtual machine and only requires the path construct for the classes that are being analysed. The tool lists the processes associated with the provided path and isolates the specific process spawned by it that has the injected payload from the original test sample. The tool then searches through the process for the start of the original *.dex* file and instructs the *GNU debugger* (GDB) to dump the memory that the tool has found.

In this study, however, running the tool was a bit more challenging and required a combination of specific tooling versions as well as a slight modification to the Android-unpacker tool. The steps followed to use this tool are listed below.

- Edit the script *playnice.sh* so that the format of the magic pattern is *dex\n* without any training numbers, making it more compatible and forgiving when looking for the start of the original samples *.dex* file in memory.

- For the best result, run an old version of the Android Operating System. Newer versions have *Position Independent Executable* (PIE) protection built in, and trying to execute the underlying scripts will result in an error (*error: only position independent executables (PIE) are supported*).
- Push the packed APK application to the Android emulator using the ADB tool.
- Upload the GDB tool to the location specified in the *playnice.sh* script.
- Run the script *playnice.sh*, as shown in figure 9.3. Providing that the original *.dex* sample was found in one of the processes examined, it should then be dumped to the file system.
- Download the dumped *.dex* file to the host development environment.
- Unpack and examine the downloaded file using the Java Decompiler software used earlier during static analysis.

```
$ ./playnice.sh com.stentsure.androidshell.androidshell
Attempting to find memory address for com.stentsure.androidshell.androidshell inside 2254
Found clone pid at 2315
Searching 0x8c360000 - 0x8c362000 for "dex\n"
Searching 0x8c362000 - 0x8c363000 for "dex\n"
Searching 0x8c363000 - 0x8c460000 for "dex\n"
Searching 0x8c460000 - 0x8c464000 for "dex\n"
Searching 0x8c5db000 - 0x8c69e000 for "dex\n"
Searching 0x8c6a4000 - 0x8caa5000 for "dex\n"
Searching 0x8caa5000 - 0x8caa7000 for "dex\n"
Searching 0x8caa7000 - 0x8caa8000 for "dex\n"
Searching 0x8caa8000 - 0x8cba5000 for "dex\n"
Searching 0x8cba5000 - 0x8cba6000 for "dex\n"
Searching 0x8cba6000 - 0x8cca3000 for "dex\n"
Searching 0x8cca3000 - 0x8cca4000 for "dex\n"
```

Figure 9.3: Execution of the *playnice.sh* script searches for the start of a *.dex* file in memory.

If this static analysis fails, then we must rely on other methods of dynamic analysis to analyse the packed sample. Due to the advanced anti-emulation and anti-debug capability of the packed sample, this does complicate its further analysis. Analysts are often reluctant to modify a sample to disable its anti-virtualisation and debugging capabilities because amending samples in such a way could have an effect on how and what instructions it runs. Without knowing more about the sample, it is difficult to know the full impact of editing the sample.

9.5.4 Using Volatility

The method for quickly obtaining an unpacked sample from an Android device during a major incident that is recommended from the research conducted by this case study is to use *LiME* [26] and Volatility. Volatility was used during case study one and is a powerful tool, scalable for this case study as it can use custom plugins to extend its functionality. LiME is a *kernel module* that enables memory acquisition from a target Android emulator, allowing memory extraction from a higher level within the device and not relying on debuggers to attach to a process. The setup phase of this section of the study however can be complex, time-consuming and immensely frustrating. During a major incident, it is not recommended that an analyst sets up a lab from scratch. If there is a risk highlighted within an organisation that can be reduced by ensuring an Android malware analysis lab is already set up to triage a likely future incident, then it is recommended to have this type of environment and tooling available ahead of a major incident. Alternatively, the lab used for this investigation and for the supporting videos has been made available at: https://github.com/devopscrazy/AndroidForensics. The repository not only contains the precompiled *Android kernel* that allows for custom modules to be loaded but also the LiME module already *cross-compiled* for the kernel. The repository includes a wrapper script that has been written in Python, used for this case study to speed up its investigation. It is recommended that the provided resources are used, otherwise preparing this tooling manually requires an environment with a reasonable level of resource and a lot of patience from the analyst.

The basic concept is to load a module into the Android kernel that allows for memory to be dumped on demand. Then, it is simply a case of running the sample on the emulator, dumping the memory using the kernel module and examining this memory dump using Volatility. Although this tool has been demonstrated in earlier case studies within this document, it will be examined again here as it is an essential tool for any analyst's tool box. A video of the below experiment has been captured and is available at: https://youtu.be/enKqmD_8VWw.

The biggest challenge of using a loadable kernel module is that it needs to be cross-compiled for the version of Android it will be running on. The Android kernel being used also needs to be compiled to allow for the execution of loadable kernel modules. If an analyst wants to set up this lab themselves, they will therefore need a development environment to compile Android and LiME. Most of the time in this study was spent on just preparing and rebuilding this development environment due to the prerequisites for compiling the Android kernel and the environment continually breaking for a variety of reasons.

There were multiple attempts to compile a working copy of the cross-compiled kernel module, and eventually a paper on *smartphone live forensics* [27, page 42] provided instruction on building a working environment. The components that were cross-compiled for use in this study were the following.

- An Android kernel was compiled that used a copy of the configuration file that was extracted from the emulator that it will be run on. The configuration file was modified to include the setting for enabling loadable kernel modules.
- A version of LiME that has been cross-compiled with the above kernel.
- A custom plugin was produced for Volatility that included mapped resources from the cross-compiling of LiME.

Setting up a development environment requires patience and various searches across the internet to fix obscure errors. Documenting the full set-up for this examination would offer nothing additional to the documentation regarding smartphone live forensics that has already been cited above, and it would more than likely become quickly out of date.

The custom script written for this project, available on the GitHub repository that was referenced earlier in this section, has functions to help automate:

- starting the emulator
- deploying custom applications
- deploying and running LiME
- dumping the memory from the device to a file on the development machine (memory dumps can be large and so it uses *netcat* to stream the data to the host instead of filling up the internal *sdcard* in the virtual device)
- running Volatility and passing custom plugin parameters
- dumping all of the running processes' memory contents.

Once the processes are extracted from the memory dump, it is simply a case of once more searching for the *.dex* pattern in the dump and extracting it, as demonstrated in section 9.5.3. The *.dex* file can then be decompiled again to extract the unpacked code. At the end of these steps, an analyst should have the original test sample that was injected into memory by the sample packed by Bangcle. Future versions of Bangcle may include steps to protect this injected *.dex* from further investigation. In this case, the extracted sample may be the starting point for further de-obfuscation steps before an examination of any malicious payloads can be conducted.

9.5.5 Obfuscation techniques observed

Bangcle could implement any of the below obfuscation techniques, as documented in a Virus Bulletin paper from July 2015 [22, section 3].

- Change class names to use very long ones, causing some debuggers to crash whilst still allowing a sample to execute correctly.
- Use NOPs to change the flow of bytecode (this has been documented in case study two but can also be used by Android applications).
- Path obfuscation can be used to fool antivirus signatures that look for strings in paths.
- Path phishing involves the renaming of application paths to mimic ones used by legitimate programs.
- Hide packages in assets and then inject their content directly into memory for execution when unpacking.
- Hiding bytecode, in general, can be used to abuse linear sweep disassemblers.
- Use string tables, a technique sometimes used by legitimate applications to help hide strings inside character arrays.
- Lastly, encoding or encrypting payloads within samples is common practice.

Due to time constraints around this case study, further analysis of the techniques used in the decompiled packed sample was not feasible. However, demonstrating how an analyst could obtain a payload's content for further analysis in a major incident was possible using the dynamic analysis techniques detailed in this case study.

The techniques required to obtain and analyse Android packed malware as part of this study were time consuming to research and prepare. Security professionals could be aided by more sophisticated and reliable tools to conduct forensic analysis on packed Android malware. In the future, as organised crime and APTs seek to exploit the potential gains from mobile devices, the deficit will need to be filled in by both commercially and community supported tools for analysing packed malware.

10 Comparisons, conclusion and reflections

10.1 Comparisons

Within this project, each case study has focused on a different aspect of forensic analysis that would be expected as part of a *root cause analysis* either after or during a major incident in an organisation. Case study one focused on the forensic dissection of a polymorphic sample to try and reverse engineer the techniques used to protect the sample. From the opposite end of the spectrum, case study two looked at building a first-generation sample from the source code of a malware sample, metaphorically walking a mile in the shoes of a malware writer. This exercise required contact from other members of the security community to complete the compilation task. The final case study focused on the ever-broadening field of mobile malware, and in contrast to the other case studies, it focused on combining everything that had already been learned from the previous two case studies. Just like case study two, a custom sample was built and protected using the same tools available to malicious parties. Then, using the same methodologies as in case study one, the tools used were adapted to obtain the de-obfuscated payload and document the techniques used by the PaaS to protect the payload.

All the case studies were distinct from one another; the first was a pre-compiled polymorphic sample, the next was a piece of assembly that, when compiled, created a metamorphic sample, and the last was a mobile-based sample. However, as previously mentioned, the skills acquired from

finishing one case study were then built upon in the next exercise. The case studies were distinct enough from one another to stand up on their own as a foundation for forensic malware investigation, but when run one after the other, each of their endeavours added to the next, thereby strengthening the previous study's findings.

The malware created in the last case study was neither polymorphic nor metamorphic, however the samples used in the first two case studies did fall within these classifications. The polymorphic sample and its techniques were once very successful in evading detection due to how they encrypted their payload and mutated the stub. However, the most impressive sample to watch was the one used as part of the metamorphic case study. The fact that the malware writer used processes found in science and nature to create a sample that could derive clean code from combining a genome with the malware body was incredible to watch. Both the polymorphic and metamorphic samples shared the ability to mutate. The polymorphic sample used mutating techniques to modify its stub to evade detection, but the metamorphic sample used these techniques on a larger scale, and when combined with the gene-based methodology, it was equally as effective at evading detection.

Analysing a metamorphic sample is far more difficult than a polymorphic sample. For the polymorphic malware, it was straightforward to bait the sample into infecting a controlled custom program. Then, interpreting the stub was simply a case of discarding any known sections present in the program that had been used as bait. As the metamorphic sample integrates obfuscation techniques across its entire source code, using a baited sample is just as much work to examine as the original. Again, it is possible to execute a polymorphic sample and extract its payload out of memory when it is decrypted into ram. However, not all metamorphic malware extracts clean code from their malware bodies. Therefore, analysis of a metamorphic malware sample involves more time-consuming work.

10.2 Conclusion

Understanding the impact that a malware infection has had on an organisation and if its payload has been developed as part of a targeted attack is vital intelligence for a security team. If an infection has remained undetected within an organisation for a period of time, then being able to report information about the APT to senior management is important. Senior management will be interested in understanding how the compromise occurred, what is currently being done to treat the incident and how future infections will be prevented. Organisations are under increased pressure to investigate and understand breaches in their security, and to prevent the same event occurring in the future by learning from the experience. Just cleaning up an infection and then closing a major incident is no longer a sustainable solution. Reverse engineering a malware sample provides a broader understanding of a payload's extent of infection and can provide security teams with signatures to detect and mitigate the threat. Each of the case studies examined in this project have focused on different aspects that are applicable to an organisation during a major incident involving polymorphic, metamorphic and Android-based malware.

The first case study looked at a piece of polymorphic malware and simulated the need for an organisation to forensically examine the sample and extract its payload. During a major incident, an organisation often just wants to know the impact an infection has had. In this case study, the focus was to quickly peel back the layers of protection that a polymorphic malware sample had and extract its payload for further analysis. The polymorphic malware sample analysed in this case study made a call out to external resources. There is a high probability that the IRC request was used to connect to a command-and-control server, which was subsequently used to download further instructions and execute them on the infected device. This could easily contain instructions to locate business-critical information which could then be transferred outside of the organisation. By unpacking the sample, the payload was obtained and could be used to assess the likelihood and risk

associated with its outbound IRC requests. An organisation can use this intelligence to build a signature to block attempts at outbound communication from other devices infected with the same sample.

The second case study focused on building techniques to walk a mile in the shoes of a malware writer. In this study, a metamorphic malware sample's source code was obtained and research was carried out to build a profile of the malware writer and the tools they utilised. This profile was then used to work out how to compile the sample. After obtaining a first-generation copy of the malware sample, the study looked at the metamorphic techniques used by the malware sample and it was discovered that it used techniques inspired by science to hide its payload. In the event of an organisation becoming the target of a malware writer, it may be the victim of several different strains of malware that have been written by the same developer. In this scenario, it would be an advantage to develop a signature that identifies a wide spectrum of malware written by the same developer. This case study highlighted the importance of researching the developer. The sample used in this study was written by a malware writer called Vecna who was a member of the hacking group 29A. When reviewing the group's published work, similar characteristics were observed between the members' samples and their tooling, such as the brand of compilers and the custom tools that they had individually developed. If the range of researched samples is too diverse to help build a signature, then obtaining a first-generation copy of the sample can allow an analyst to reverse engineer the sample before it becomes too complex to interpret. For instance, some samples that are several generations old can bloat in size.

The final case study looked at the growing trend of mobile devices within organisations. As more enterprise data and broader access is granted to such devices, they are becoming an attractive target for malware writers. A sample application for an Android device was built and then online platforms that are freely available to malicious parties were utilised to obfuscate the application. The obfuscated application was then used to peel back the layers of security and document the techniques used to secure the sample. Although Bangcle's website (the PaaS provider) was in Chinese and translations of it were challenging to comprehend, registering for and using the platform was easy. This type of platform can easily be used by malicious parties to protect applications that they intend to submit to device marketplaces. The application submitted to the PaaS was additionally provided with anti-virtualisation and anti-debugging protection as part of the packing process, meaning its forensic examination was complicated. An employee's Android device could become infected if no policies are put in place by an organisation that restrict the download of applications. As mobile devices are becoming integrated into corporate environments, this provides a weak entry point into an organisation's network. By examining the contents of a packed Android application using the techniques outlined in case study three, the intent of a mobile application's payload can be assessed and then approved for corporate devices.

All the case studies have started by conducting a static analysis of the malware sample to examine the obfuscation techniques that it utilises. Once the information extracted from a sample has been exhausted during a static analysis, a dynamic analysis has then been necessary to speed up the processes of peeling back the layers of obfuscation used by a sample. This project has provided documentation and videos of how to set up a lab environment that can be used to safely conduct dynamic analysis of a sample during its execution whilst ensuring an analyst's development environment is not infected by any malicious activity being observed. Most of the techniques used to obfuscate the malware samples were identified during the dynamic analysis stage.

The techniques demonstrated in the case studies and the videos that accompany them in this report should provide enough of a foundation for the forensic analysis of a malware sample to not be reserved for highly-paid security consultants. Systems administrators, security engineers and security analysts that already work for an organisation should be able to tackle obfuscated malware

and extract information required to answer questions from company directors as part of a root cause analysis surrounding an incident.

The documentation provided in all three case studies can be combined to provide techniques to obtain the de-obfuscated payload from a sample, research the malware writer if indicators are present and understand if the organisation is being directly targeted or if it is just a victim caught in a much wider campaign. Additionally, if an organisation falls foul of a malware infection on an Android device, then this document can also be used to peel back the layers of obfuscation used by the sample and ascertain if any organisational assets that are accessible by the device have been compromised.

The comparison in section 10.1 has already detailed how these case studies have complimented one another. What is apparent after conducting these studies is how effective these techniques are at evading antivirus software and that they have been used on a small scale for some years. Now, short-lived malware campaigns are profitable to organised crime and malicious parties, and so the use of these advanced techniques is not required. If a malware campaign (for instance, one using *Dridex* [8]) is only being run to harvest credit card and banking details, then the malware does not need to remain on the system once such details have been obtained. In these types of campaigns, it is acceptable for a signature to be developed after a few weeks and the strain to then be disinfected from systems as the campaign has remained profitable. The malware sample can simply be adjusted and re-packed using simple techniques to run in the next campaign.

As antivirus vendors close in on reducing the advantage that malicious parties possess and eventually manage to block such quick campaigns from being profitable, more advanced techniques will be required by malware writers, and polymorphic or metamorphic techniques may once again become popular.

Many organisations' systems are being developed with a web interface, and as such, browser extensions are often installed on client machines to help automate some tasks. These browser extensions, such as Chrome plugins, often install with high privilege levels on a user's machine and therefore they may become an attractive target in future malware campaigns. One such campaign that is current involves plugins being downloaded, and on infection, the plugin resubmits its malicious payload once again to the Chrome plugin store and puts a recommendation and comment on itself and other plugins available for the same variant, hoping that users will see a high number of positive comments and download it, unwittingly infecting themselves and helping to spread this variant of the sample. It is the opinion of this author that these types of infections in the future may use polymorphic and metamorphic techniques to mask its payloads, making samples from the same strain distinct enough between generations that re-publishing the next generation on the plugin marketplaces will mean that they appear completely unique from each other. The use of advanced malware techniques could also extend the duration of these campaigns.

Although the mobile malware investigated as part of this study was neither polymorphic nor metamorphic due to the advanced sandboxing used by Android, there is nothing stopping future malware using the technique detailed above that is used by Chrome plugins. An Android malware sample could re-package itself using polymorphic or metamorphic techniques on infection, and then it could submit itself to the marketplaces for mobile applications. It could then provide positive comments and attract users to its application so that they become infected themselves.

Due to time constraints, the scope of this project has been limited to the forensic investigation of polymorphic and metamorphic techniques in use by malware infecting Microsoft Windows environments, as well as obfuscation techniques used by platforms that are designed to protect Android applications. With more time, it would have been nice to extend the scope of the case studies and investigate the above hypothesised method of incorporating polymorphic and

metamorphic techniques in the obfuscation and spread of malicious Chrome plugins and Android applications. The case studies could be extended by building a polymorphic and metamorphic sample for Chrome and Android, and then submitting the samples to applicable digital marketplaces and observing if they are detected as malicious. It would also be interesting to analyse the current plugins and applications already available on digital marketplaces and attempt to detect if these hypothesised techniques are currently in use.

The videos provided by this project are designed to aid anyone tasked with conducting forensic analysis of a malware sample, however, they are not very interactive. They allow for comments to be published by viewers of the content, but provide little flexibility to adjust the content, and they lack the ability to give responsive feedback to viewers. It would be nice to migrate the videos' content to an online course format, allowing for content to be broken down and easily modified or updated depending on students' feedback. Multiple-choice tests could be integrated to allow students to track their progress and understanding of the content. There are multiple online learning services, such as the *MIT Open CourseWare* (https://ocw.mit.edu/index.htm) and *Open Mooc* (http://mooc.org/) sites, for which any interactive content developed from this project could be made into a proposal to one of the website's partners.

As the landscape changes and more advanced techniques are used, organisations and their current employees will need to be able to dissect samples and use the forensic techniques detailed in this document to answer the bare minimum of questions being posed to professionals tasked with security.

Case study two required filling in the blanks left in a piece of malware code to try and compile it to a first-generation copy of the malware. The exercise itself was a good example of how to research the tooling used by such parties to create these samples, as well as demonstrating how their techniques have evolved during their career in the field. The most important lesson learned from this case study though is that there is a security community ready to support and provide advice when it comes to examining such puzzles. Case study two could have been a disaster to try and make sense of the source code for the sample without obtaining the full code base provided by another security analyst. It is important that as a security community, analysts feel that they can take part in the challenges posed by conducting malware analysis and can ask for help when they need it. It is just as important to provide help to others if they can, and even if they don't have the skills to provide a solution, at least they can feel that they can join in with solving more puzzles.

10.3 Reflections

One of a security analyst's main tools is a hex debugger, and another is simply having a creative imagination. Watching a sample run and working out what and why something is moving around in a device is a strong skill to sharpen. An analyst will find themselves making sense of a primordial soup of instructions flowing through registers and gaining insights into the inner workings of other developer's creations.

This paper's author has found themselves understanding more about malware samples by trying to get into the head of a sample's developer. After a while, the developer's programming styles and the evolution of the techniques and tools they use become more visible. Understanding the techniques used by a payload is often similar to watching fog part. A week would pass, analysing or trying to compile a sample with no results, and then suddenly everything would seem to make sense and fall into place all at once.

There is no set way that a security analyst needs to think and no "run-book" they need to follow when analysing any malware sample. They need to build up an intuition and a tenacious attitude to working out the solution to complex puzzles. It was important during this study to enjoy the

challenge, and looking at malware as a puzzle helps drive an analyst to not give up and to instead strive to find these solutions.

As part of the metamorphic case study, it was necessary to contact an analyst called Orr who had written a great paper on the sample used as part of this investigation. They were a great help, and not only did they provide links to the original source code that made compilation of the sample possible, but they also provided names of further samples to analyse for other incredibly ingenious techniques. Unfortunately, there is not enough time to document those samples here, but the community within the security analyst spectrum is very supportive. So, another important thing learned from this study is that when a dead end is reached with an investigation, don't give up. Ask others; if they haven't already worked through the puzzle themselves, they are often open to helping with another challenge.

I want to say thank you to the various malware developers that wrote the original samples for this study. Without them I would have had no puzzles to drill into and attempt to solve. I also want to thank Orr for helping unblock my investigations into metamorphic malware when they were reaching a dead end. Orr's original study of *Lexotan32* made it much quicker to know what to look out for when running the sample.

One interesting quote that has stuck with me throughout this project is a quote I read in a Virus Bulletin paper from September 2014 [24, section 2]. It is a quote from Sun Tzu:

> "It is said that if you know your enemies and know yourself, you will not be imperilled in a hundred battles; if you do not know your enemies but do know yourself, you will win one and lose one; if you do not know your enemies nor yourself, you will be imperilled in every single battle." [28]

As an analyst trying to defend assets, the attacker will always have the advantage. However, by ensuring that an analyst does all that they can to prepare and obtain the knowledge required to tackle a puzzle in the same way that an attacker constructs them, then they at least know themselves, and it gives them the tools they need to build a picture of their adversary. If organisations continue to simply disinfect machines and look no further, then the victory of closing the major incident is undone by the defeat of giving up on understanding the true risk landscape.

11 Bibliography

11.1 References cited

[1] Suarez-Tangil, G., Tapiador, J. E., Peris-Lopez, P. *Stegomalware: Playing Hide and Seek with Malicious Components in Smartphone Apps.* In Lin, D., Yung, M., Zhou, J. (eds). Information Security and Cryptology. Inscrypt 2014, *Lecture Notes in Computer* Science, vol 8957: 496-515. Available from: http://www0.cs.ucl.ac.uk/staff/G.SuarezdeTangil/papers/2014inscrypt-stegomalware.pdf [Accessed 22nd March 2017].

[2] Hardikar, A. *Malware 101 – Viruses*. SANS Institute InfoSec Reading Room. April 2008. Available from: https://uk.sans.org/reading-room/whitepapers/incident/malware-101-viruses-32848 [Accessed 22nd March 2017].

[3] Douligeris, C., Mitrokotsa, A. 2003. "DDoS attacks and defense mechanisms: classification and state-of-the-art." *Computer Networks*. 2004; 44(5): 643-666. Available from: http://cys.ewi.tudelft.nl/sites/default/files/comnet.pdf [Accessed 22nd March 2017].

[4] Wisniewski, C. *The Money behind the Malware*. June 2012. Available from: https://www.sophos.com/en-us/security-news-trends/security-trends/money-behind-malware-threats.aspx [Accessed 7th March 2017].

[5] Deque. *Handbook about Crypters and Binders*. Version 1.2.2, April 2014. Available from: https://github.com/devopscrazy/MalwareAnalysis/blob/master/crypters.pdf [Accessed 11th February 2017].

[6] Langer, R. *To Kill a Centrifuge: A Technical Analysis of what Stuxnet's Creators Tried to Achieve*. November 2013. Available from: http://www.langner.com/en/wp-content/uploads/2013/11/To-kill-a-centrifuge.pdf [Accessed 22nd October 2016].

[7] Moon, P. *The Use of Packers, Obfuscators and Encryptors in Modern Malware*. MSc Thesis, London: Royal Holloway University of London, March 2015. Available from: https://www.ma.rhul.ac.uk/static/techrep/2015/RHUL-MA-2015-10.pdf [Accessed 7th March 2017].

[8] Wikipedia. *Dridex*. Available from: https://en.wikipedia.org/wiki/Dridex [Last updated 25th March 2017, accessed 25th March 2017].

[9] Department for Culture, Media and Sport (National Cyber Security Programme). *Cyber Security Breaches Survey 2016*. 2016. Available from: https://www.gov.uk/government/uploads/system/uploads/attachment_data/file/521465/Cyber_Security_Breaches_Survey_2016_main_report_FINAL.pdf [Accessed 7th March 2017].

[10] Graveland, B. *Fully Undetectable Cryptors and the Antivirus Detection Arms Race*. January 2011. Available from: https://www.symantec.com/connect/blogs/fully-undetectable-cryptors-and-antivirus-detection-arms-race [Accessed 22nd March 2017].

[11] Orr. *The Molecular Virology of Lexotan32: Metamorphism Illustrated*. August 2007. Available from: http://www.openrce.org/articles/full_view/29 [Accessed 22nd October 2016].

[12] Zakorzhevsky, V. *Review of the Virus. Win.32.Virut.ce Malware Sample*. June 2010. Available from: https://securelist.com/analysis/publications/36305/review-of-the-virus-win32-virut-ce-malware-sample/ [Accessed 22nd January 2017].

[13] Sikorski, M. Honig, A. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. San Francisco: No Starch Press, Inc.; 2012. [ISBN: 978-1-59327-290-6].

[14] Serban, L. *Analysis of W32/Virut.ce*. March 2011. Available from: https://aviratechblog.files.wordpress.com/2011/03/analysis_w32-virut_-ce_.pdf [Accessed 22nd January 2017].

[15] Google. *How People Use Their Devices: What Marketers Need to Know*. September 2016. Available from: https://storage.googleapis.com/think/docs/twg-how-people-use-their-devices-2016.pdf [Accessed 10th March 2017].

[16] Symantec. *Internet Security Threat Report*. Volume 21, April 2016. Available from: https://www.symantec.com/content/dam/symantec/docs/reports/istr-21-2016-en.pdf [Accessed 10th March 2017].

[17] Rastogi, V., Chen Y., Jiang, X. *DroidChameleon: Evaluating Android Anti-Malware Against Transformation Attacks*. Northwestern University Technical Report NU-EECS-13-01. March 2013. Available from: http://pages.cs.wisc.edu/~vrastogi/static/papers/rcj13b.pdf [Accessed 22nd January 2017].

[18] EICAR – European Institute for Computer Anti-Virus Research. *EICAR homepage*. Available from: http://www.eicar.org/ [Accessed 22nd January 2017].

[19] GuardSquare. *ProGuard homepage*. Available from: https://www.guardsquare.com/en/proguard [Accessed 22nd January 2017].

[20] APK Protect. *APK Protect download page*. Available from: https://sourceforge.net/projects/apkprotect/ [Accessed 22nd January 2017].

[21] Bangcle. *Bangcle homepage*. Available from: https://www.bangcle.com/ [Last updated 15th March 2017, accessed 25th March 2017].

[22] Apvrille, A., Nigam, R. *Obfuscation in Android Malware, and How to Fight Back*. July 2014. Available from: https://www.virusbulletin.com/virusbulletin/2014/07/obfuscation-android-malware-and-how-fight-back [Accessed 22nd January 2017].

[23] Mr. Trojans. *ALAN – Android Malware Evaluating Tools Released*. Weblog, February 2015. Available from: http://seclist.us/alan-android-malware-evaluating-tools-released.html [Accessed 22nd January 2017].

[24] Yu, R. *Android Packers: Facing the Challenges, Building Solutions*. January 2016. Available from: https://www.virusbulletin.com/virusbulletin/2016/01/paper-android-packers-facing-challenges-building-solutions/ [Accessed 22nd January 2017].

[25] GitHub. *Android-Unpacker homepage*. Available from: https://github.com/strazzere/android-unpacker [Last updated 30th June 2016, accessed 22nd January 2017].

[26] GitHub. *LiME memory extractor homepage*. Available from: https://github.com/504ensicsLabs/LiME/tree/master/doc [Last updated 25th January 2017, accessed 7th February 2017].

[27] Wächter, P. *Practical Infeasibility of Android Smartphone Live Forensics: Applicability Constraints of LiME and Volatility*. MSc Thesis, Nürnberg: Friedrich-Alexander-Universität Erlangen, April 2015. Available from: https://www1.cs.fau.de/filepool/gruhn/thesis_waechter.pdf [Accessed 7th February 2017].

[28] Wikiquote. Translation of quote from Sun Tzu's *The Art of War*. Available from: https://en.wikiquote.org/wiki/Sun_Tzu [Last updated 7th February 2017, accessed 10th February 2017].

11.2 Documents used as additional sources

[29] Ablon, L., Libicki, M. C., Golay, A. A. *Markets for Cybercrime Tools and Stolen Data: Hacker's Bazaar*. Santa Monica, California: RAND Corporation, 2014. Available from: http://www.rand.org/pubs/research_reports/RR610.html [Accessed 9th February 2017].

[30] Alam, S., Horspool, R. N., Traore, I., Sogukpinar, I. "A framework for metamorphic malware analysis and real-time detection." *Computers &* Security. 2015; 48: 212-233. Available from: http://www.sciencedirect.com/science/article/pii/S0167404814001576 doi: 10.1016/j.cose.2014.10.011. [Accessed 22nd January 2017].

[31] Android. *Android development page*. Available from: https://source.android.com/source/initializing.html [Accessed 22nd January 2017].

[32] Anubis. *Anubis homepage*. Available from: http://anubis.iseclab.org/ [discontinued, accessed 22nd January 2017].

[33] cOrRuPt G3n3t!x. Batch Run Techniques. July 2009. Available from: http://vxheaven.org/lib/vcg02.html [Accessed 25th March 2017].

[34] Dang, B., Gazet, A., Bachaalany, E. *Practical Reverse Engineering*. Indianapolis: John Wiley & Sons, Inc.; 2014. [ISBN 978-1-118-78731-1].

[35] Ferrie, P., Shannon, H. *Virus Bulletin*. May 2005. Available from: https://www.virusbulletin.com/uploads/pdf/magazine/2005/200505.pdf [Accessed 22nd October 2016].

[36] Filliol, E. *Applied Cryptoanalysis of Cryptosystems and Computer Attacks through Hidden Cyphertexts Computer Viruses*. INRIA Institut National de Recherche en Informatique et en Automatique Research Report RR-4359. January 2002. Available from: https://hal.archives-ouvertes.fr/inria-00072229/document [Accessed 22nd January 2017].

[37] Gandotra, E., Bansal, D., Sofat, S. "Malware analysis and classification: a survey." *Journal of Information Security*, 2014; 5: 56-64. Available from: http://file.scirp.org/pdf/JIS_2014040110394271.pdf doi: 10.4236/jis.2014.52006. [Accessed 10th February 2017].

[38] GitHub. *ALAN homepage*. Available from: https://github.com/faber03/AndroidMalwareEvaluatingTools [Last updated 22nd May 2016, accessed 22nd January 2017].

[39] GitHub. *Android kernel compilation instructions*. Available from: https://github.com/volatilityfoundation/volatility/wiki/Android [Last updated 12th September 2016, accessed 22nd January 2017].

[40] GLiTCH. *GLiTCH's Polymorphic Batch Tutorial*. No date. Available from: http://vxheaven.org/lib/vgl01.html [Accessed 25th March 2017].

[41] Hh86. *Hidden in .NET*. August 2011. Available from: http://vxheaven.org/lib/vhh07.html [Accessed 25th March 2017].

[42] Hosmer, C. *Polymorphic and Metamorphic Malware*. Presentation, Black Hat. August 2008. Available from: https://www.blackhat.com/presentations/bh-usa-08/Hosmer/BH_US_08_Hosmer_Polymorphic_Malware.pdf [Accessed 25th September 2016].

[43] Iliopoulos, D., Ször, P., Adami, C. "Darwin inside the machines: malware evolution and the consequences for computer security." *Proceedings of Virus Bulletin Conference.* 2008: 187-194. Available from: https://arxiv.org/pdf/1111.2503.pdf [Accessed 22nd January 2017].

[44] Jacob, G., Filiol, E., Debar, H. 2008. "Functional polymorphic engines: formalisation, implementation and use cases." *Journal in Computer Virology.* 2009; 5(3): 247-261. Available from: https://link.springer.com/article/10.1007/s11416-008-0095-z doi:10.1007/s11416-008-0095-z [Accessed 22nd January 2017].

[45] Java Decompiler. *Java Decompiler homepage*. Available from: http://jd.benow.ca/ [Accessed 9th February 2017].

[46] Kaspersky Lab. *Unveiling "Careto" – The Masked APT*. February 2014. Available from: https://kasperskycontenthub.com/wp-content/uploads/sites/43/vlpdfs/unveilingthemask_v1.0.pdf [Accessed 22nd January 2017].

[47] Kendall, K., McMillan, C. *Practical Malware Analysis*. Presentation, Black Hat. August 2007. Available from: https://www.blackhat.com/presentations/bh-dc-07/Kendall_McMillan/Presentation/bh-dc-07-Kendall_McMillan.pdf [Accessed 22nd October 2016].

[48] Kruegel, C., Kirda, E., Mutz, D., Robertson, W., Vigna, G. *Polymorphic Worm Detection Using Structural Information of Executables*. In RAID Proceedings of the 8th international conference on Recent Advances in Intrusion Detection. September 2005. Available from: https://cs.ucsb.edu/~chris/research/doc/raid05_polyworm.pdf [Accessed 22nd January 2017].

[49] Leder, F., Steinbock, B., Martini, P. 2009. *Classification and Detection of Metamorphic Malware Using Value Set Analysis.* In Malicious and Unwanted Software (MALWARE) 2009 4th International Conference. Available from: http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.570.216&rep=rep1&type=pdf [Accessed 22nd October 2016].

[50] Lord Julus. *Polymorphism – Analysis on the Decryptor Generator 1.5*. February 1998. Available from: http://vxheaven.org/lib/vlj04.html [Accessed 25th March 2017].

[51] Nelson, A., Cavallaro, L. *Sandnet++ – A Framework for Analysing and Visualising Network Traffic from Malware*. Royal Holloway University of London Technical Report RHUL-ISG-2016-11. 2016. Available from: https://www.royalholloway.ac.uk/isg/documents/pdf/technicalreports/2016/computer-weekly-articles/anthonygnelsoncw.pdf [Accessed 22nd January 2017].

[52] Pearce, S. *Viral Polymorphism*. SANS Institute. November 2003. Available from: https://cyber-defense.sans.org/resources/papers/gsec/viral-polymorphism-105483 [Accessed 22nd January 2017].

[53] Perriot, F., Ször, P, Ferrie, P. *Striking Similarites: Win32/Simile and Metamorphic Virus Code*. 2002. Available from: https://www.symantec.com/avcenter/reference/striking.similarities.pdf [Accessed 22nd January 2017].

[54] Raynal, F., Delugré, G., Aumaitre, D. "Malicious origami in PDF." *Journal in Computer Virology*. 2010; 6(4): 289-315. Available from: https://pdfs.semanticscholar.org/1387/08890788b9f43cbfd3d09041f8361a4a3cf3.pdf doi: 10.1007/s11416-009-0128-2 [Accessed 22nd January 2017].

[55] Santamarta, R. *Generic Detection and Classification of Polymorphic Malware Using Neural Pattern Recognition ReverseMode*. 2006. Available from: https://pdfs.semanticscholar.org/5cda/37f3fe61f1fa156752be27fdb7cc40983e84.pdf [Accessed 22nd January 2017].

[56] Sharma, A., Sahay, S. K. "Evolution and detection of polymorphic and metamorphic malwares: a survey." *International Journal of Computer Applications*. 2014; 90(2): 7-11. Available from: http://research.ijcaonline.org/volume90/number2/pxc3894098.pdf doi: 10.5120/15544-4098. [Accessed 10th February 2017].

[57] Szor, P. *The Art of Computer Virus Research and Defense*. New Jersey: Addison-Wesley Professional (Symantec Press); 2005. [ISBN 978-0-3213-0454-4].

[58] Tapiador, J. *Malware Morphisms*. Presentation, 2014. Available from: http://www.seg.inf.uc3m.es/~jet/#papers [Accessed 25th September 2016].

[59] Tarkan Yetiser. *Polymorphic Viruses – Implementation, Detection, and Protection*. January 1993. Available from: http://vxheaven.org/lib/ayt01.html [Accessed 25th March 2017].

[60] Wyke, J. *Extracting Actionable Data from Modern Financially Motivated Malware on Windows Personal Computers*. Royal Holloway University of London Technical Report RHUL–ISG–2016–14. April 2016. Available from:

https://www.royalholloway.ac.uk/isg/documents/pdf/technicalreports/2016/rhul-isg-2016-14-james-wyke.pdf [Accessed 22nd January 2017].

12 Appendix

12.1 The use of VMware ESXi for hosting the malware lab

VMware's ESXi is a full hypervisor and this was also tested as a candidate to host the lab used for this project's case studies by using spare hardware. It is a full Operating System that is installed directly onto hardware and allows for a client side tool to connect to the hypervisor and provision the same lab architecture as documented in section 6.2. The benefits of this are that the lab does not use resources from an analyst's machine, the lab can be shared by other analysts in a security team and the software licencing cost can potentially be reduced if large teams require multiple versions of the desktop software.

12.2 Using Ollydbg to extract a payload from memory

Obtaining an unencrypted payload from a malware sample was possible in case study one by:

- using a newly infected test program
- executing the test program
- using Ollydbg (http://www.ollydbg.de) to hook into the executing processes
- dumping out the running program from a device's memory.

By examining the extracted memory dump after the payload has been unpacked during the sample's execution, the de-obfuscated payload is obtained. This procedure, as well as the earlier baiting of the malware in section 7.3.3, was recorded and a video of its steps are available via the direct link at: https://youtu.be/Goqd2RJXcaA.

12.3 Case study three development environments

Although *Eclipse* could be used to make a test Android application, much like any other Java-based application, it is easier to use tools provided by Android. Android studio is an intuitive IDE that can be used to build applications very quickly. It is based on *InteliJ* instead of Eclipse and runs quickly.

Android studio comes packaged with Android emulators. It provides an environment to build applications and debug and test them with little effort. The virtual machines use *QEMU* as the hypervisor, and as such have the option to virtualise both x86/x64 and ARM virtual appliances. Running ARM appliances is slow on most machines, and so it was easier to develop applications using the x86 appliances and then switch to ARM when extracting memory for later analysis.

12.4 Using Bangcle to protect an Android application

The first step is to register with the Bangcle [21] online service to obtain a user account. Once logged into the portal, it is possible to download the Bangcle assistant tool that is run client side. As with any other tool downloaded as part of this project, it is safer to run such third-party tools within the lab as it is segregated from the rest of an analyst's development environment.

The Bangle assistant tool allows the upload of an *.apk* sample, and after upload, the platform executes a shielding process. The sample passes through the shielding process and is then available for download. During this time, the sample is uploaded to the cloud platform and scanned for malicious payloads; if found, samples are then rejected from further processing.

Basic techniques can get past the detection of malicious payloads by the platform, and it was observed that some of the techniques in the ALAN[23] tool, detailed earlier, could be used to obfuscate a payload enough to pass through the platform's detection engine successfully.

Once shielded, the Bangcle assistant tool provides the option to sign the *.apk* file so that it can be uploaded directly into an Android device or marketplace after download. Providing an application with access to encryption keys is not always good practice, and in the example of this lab, the encryption keys used were only made available on the main development environment's host and not on the virtual machine used to run the Bangcle assistant tool.

Downloading the shielded sample using the client side tool was fast and simple, and is shown in figure 12.1.

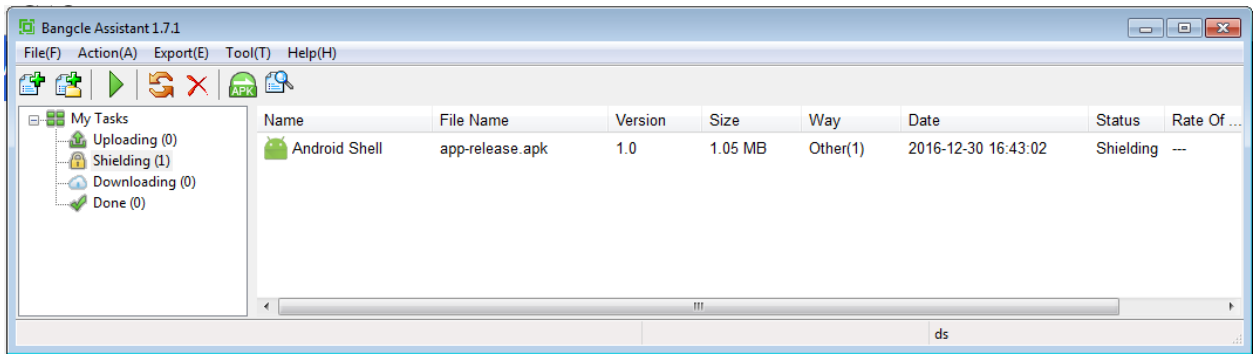


Figure 12.1: The Bangcle assistant client side tool used for uploading and downloading samples.

When comparing a simple hash of the original sample against the downloaded sample, it shows that it has indeed been edited in some way. Figure 12.2 demonstrates that the file has been modified, but as later analysis proves, the sample has been edited in such a way as to deter forensic analysis.

```
$ ls
app-release.apk  app-release_protected.apk

$ md5sum.exe app-release.apk
85e6a7382da1c194f86de15228bd304a *app-release.apk

$ md5sum.exe app-release_protected.apk
3efa461d8f2671fc122977c2bfe24445 *app-release_protected.apk
```

Figure 12.2: Original sample has been modified as part of the Bangcle obfuscation process, demonstrated by the change in hash.

As mentioned, providing a signing key to un-tested client side applications is not always safe when dealing with third-party platforms. Without the knowledge of their storage policies and internal security measures, there is a level of risk around the integrity of those signing keys after they have been uploaded to any client side tools that are being used. As part of this study, it was therefore decided to download the packed sample and sign the sample manually using the APK tools provided as part of the ALAN project, which has been covered in section 9.4.1.

12.5 Malware samples used as part of the study

12.5.1 Virut.ce

https://github.com/devopscrazy/MalwareAnalysis/blob/master/virtu.ce/offensivecomputing/offensivecomputing-virutece.zip
Sample password: "infectedIYM011"
File: offensivecomputing-virutece.zip
MD5: d0141d592741e8ecf5ee1777dd7ff5de
SHA1: 9f644022b7713c36ebaec15ea5fd53154b7c313b
SHA256: 57243f589c28a012f9bfaef3504cf5cec1f19f14e5ff4793a21d00e034c77755

https://github.com/devopscrazy/MalwareAnalysis/blob/master/virtu.ce/virusshare/virusshare-virutce.zip
Sample password: "infectedIYM011"
File: virusshare-virutece.zip
MD5: d0141d592741e8ecf5ee1777dd7ff5de
SHA1: 9f644022b7713c36ebaec15ea5fd53154b7c313b
SHA256: 57243f589c28a012f9bfaef3504cf5cec1f19f14e5ff4793a21d00e034c77755

12.5.2 Lexotan32

https://github.com/devopscrazy/MalwareAnalysis/blob/master/lexotan32/compiled-first-generation/malware-lexotan32.zip
Sample password: "infectedIYM011"
File: malware-lexotan32.zip
MD5: 95f57cad9a4d439113904f195b6cb374
SHA1: c9c531b404d61b02d73dd0bb1dad94edf619e1ec
SHA256: e9ce8c89049936b42bb56c5c0e85dff8ad5b819292f8b7b4cfd21dc656e21505

https://github.com/devopscrazy/MalwareAnalysis/blob/master/lexotan32/virusshare/virusshare-lexotan32.zip
Sample password: "infectedIYM011"
File: virusshare-lexotan32.zip
MD5: 19ec109e78db8e377ea790850890fc15
SHA1: 8b9d75ea0d977114d24707d24ae4e63b9beb4721
SHA256: c366f7ab4a90626a0ddf489524d22ffa299c2a1c174bdadd59151c795a4fc288